

Module M11.C01 — La fonction de fenêtre : `OVER`, `PARTITION BY`, `ORDER BY`, et pourquoi `GROUP BY` ne suffit plus

Analyse de données appliquée & Business Intelligence
9 partie(s) assemblée(s) · document de production

Table des matières

MODULE M11.C01 — LA FONCTION DE FENÊTRE : OVER, PARTITION BY, ORDER BY, ET POURQUOI GROUP BY NE SUFFIT PLUS	8
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le tableau d’affichage du stade	9
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 L’ordre d’exécution : pourquoi une fenêtre n’est pas un agrégat	10
5.2 OVER () : la fenêtre sur toute la table	11
5.3 PARTITION BY : le facteur de comparaison	11
5.4 ORDER BY dans la fenêtre : l’ordre change la valeur	11
5.5 Le cadre : cumul et moyenne glissante	12
5.6 La même question, deux écritures : fenêtre contre sous-requête scalaire	12
5.7 Les trois pièges, mesurés	13
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — la part de chaque magasin, et ce qu’elle décide	14
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — le total de contrôle	14
7.2 Étape 2 — la part, avec la fenêtre	15
7.3 Étape 3 — la même réponse par sous-requête scalaire	15
7.4 Étape 4 — une part calculée dans son année	15
7.5 Étape 5 — le piège : le filtre et la fenêtre dans le même SELECT	15
7.6 Étape 6 — la fenêtre n’a pas réduit la table	16
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C02 — CLASSER, CUMULER, DÉCALER : ROW_NUMBER, RANK, DENSE_RANK, NTILE, SUM() OVER, LAG, LEAD	21
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le podium et les dossards	22
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 Trois fonctions de rang, une seule mesure	23
5.2 Le départage : ce qui rend un classement reproductible	23

5.3 NTILE : découper en tranches, pas classer	24
5.4 Le top N par groupe : le motif qui exige un filtre après	24
5.5 SUM() OVER : le cumul, et sa clôture	25
5.6 LAG et LEAD : la variation, et son premier trou	25
5.7 Le classement qui contredit le total	25
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — trois classements, un seul socle	26
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — mesurer l'ampleur des égalités avant de classer	27
7.2 Étape 2 — les trois rangs côte à côte	27
7.3 Étape 3 — l'écart maximal entre les deux classements	27
7.4 Étape 4 — le cumul, et son contrôle de clôture	27
7.5 Étape 5 — la variation mensuelle, puis annuelle	27
7.6 Étape 6 — la comparaison annuelle, et le piège du filtre	28
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C03 — LE TEMPS EN SQL : TABLE CALENDRIER, ANNÉE SUR ANNÉE, MOIS GLISSANTS, ET LE MOIS QUI MANQUE	33
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — l'agenda et les trous	34
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 Fabriquer l'axe du temps	35
5.2 Le trou qu'on ne voit pas : l'objectif manquant	35
5.3 Année sur année : le périmètre d'abord	36
5.4 Moyennes mobiles : la fenêtre pleine et son compteur	37
5.5 La saison, à connaître avant d'interpréter	37
5.6 Le mois partiel : la dernière ligne du tableau	37
5.7 Ce que ce chapitre ne couvre pas	38
6. Exemple concret — trois lectures de la même année	38
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — compter l'axe du temps	39
7.2 Étape 2 — le CA mensuel, et la preuve qu'il n'y a pas de trou	39
7.3 Étape 3 — les objectifs, et leurs deux trous	39
7.4 Étape 4 — année sur année à périmètre égal	39
7.5 Étape 5 — moyennes mobiles avec compteurs	39
7.6 Étape 6 — la saison, pour interpréter	39

8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C04 — COHORTES, RÉTENTION ET RÉCURRENCE : RFM, ET LE CLIENT QUI N'A PAS DE NOM	44
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — la promotion d'une année, et le verre à moitié plein	45
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 La cohorte d'entrée : 44 lignes, une seule par client	46
5.2 La matrice : un triangle, jamais un rectangle	46
5.3 Le piège des deux lectures	47
5.4 Une rétention qui ne baisse pas	48
5.5 Le RFM : trois mesures, une décision	49
5.6 Fréquence, récurrence, récence : les trois chiffres qui tiennent	49
5.7 Le client 0 : 18,3 % du CA sans porteur	50
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — trois façons de répondre à la même question	50
7. Démonstration pas à pas — cinq étapes sur le socle	51
7.1 Étape 1 — la cohorte d'entrée	51
7.2 Étape 2 — la matrice et son triangle	51
7.3 Étape 3 — les deux lectures, côte à côte	51
7.4 Étape 4 — le RFM, avec le client 0	51
7.5 Étape 5 — les mesures de contrôle	51
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C05 — REQUÊTES AVANCÉES : CTE, TABLEAU CROISÉ, UNNEST, QUALIFY, RÉCAPITULATIFS	56
1. Objectifs du chapitre	8

2. Pourquoi cette notion est importante	8
3. Explication simple — la recette, le tableau, l'étagère	57
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 La CTE : nommer les étapes, et réutiliser la bonne	58
5.2 Le tableau croisé : PIVOT et l'agrégat conditionnel	59
5.3 QUALIFY : classer et filtrer dans la même requête	59
5.4 UNNEST : inverser le grain	60
5.5 Les récapitulatifs : ROLLUP, CUBE, GROUPING SETS	60
5.6 La récursion : pour les hiérarchies, pas pour les calendriers	60
5.7 Ce que chaque moteur sait faire	61
5.8 Ce que ce chapitre ne couvre pas	13
6. Exemple concret — le tableau que la direction avait demandé	61
7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — le repliement mesuré	62
7.2 Étape 2 — la CTE réutilisée	62
7.3 Étape 3 — le tableau croisé	62
7.4 Étape 4 — QUALIFY avec et sans repliement	62
7.5 Étape 5 — UNNEST et le grain	62
7.6 Étape 6 — le récapitulatif	62
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C06 — LE RAPPORT LENT : VUES, INDEX, PLANS D'EXÉCUTION, SELECT *, PARTITIONS	66
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — le classeur, le sommaire et le chariot	67
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 La vue : ce qu'elle relit, à chaque fois	68
5.2 SELECT * : le lire dans le plan, pas seulement au chronomètre	69
5.3 Lire un plan : trois étages, de bas en haut	69
5.4 L'index : mesuré, puis contrôlé dans un second moteur	69
5.5 Lisibilité contre performance : le cas de la fenêtre	70
5.6 Matérialiser ou non : le plancher d'un fichier	70
5.7 Les partitions	71
5.8 La note de méthode d'un rapport de performance	71
6. Exemple concret — la facture d'un rapport mal écrit	71

7. Démonstration pas à pas — six étapes sur le socle	14
7.1 Étape 1 — le protocole d’abord	71
7.2 Étape 2 — la vue contre la table	71
7.3 Étape 3 — le plan du SELECT *	72
7.4 Étape 4 — l’index, mesuré puis contrôlé	72
7.5 Étape 5 — fenêtre contre sous-requête	72
7.6 Étape 6 — le plancher de matérialisation	72
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19
15. À retenir	19
16. Évaluation formative	20
MODULE M11.C07 — LA REQUÊTE COMME LIVRABLE : STYLE, CONVENTION, TESTS DE NON-RÉGRESSION	76
1. Objectifs du chapitre	8
2. Pourquoi cette notion est importante	8
3. Explication simple — la fiche de recette et le contrôle qualité	77
4. Vocabulaire essentiel	10
5. Cours approfondi	10
5.1 L’en-tête de requête : la question avant le code	78
5.2 Nommer : les étapes, les alias, les colonnes	78
5.3 Un style qui se relit	79
5.4 Les trois tests de non-régression du module	79
5.5 Le contrôle entre deux sources : expliquer un écart, jamais l’additionner	80
5.6 Un test rejoue la requête à l’identique	80
5.7 Ce que ce chapitre ne couvre pas	38
6. Exemple concret — la revue d’un rapport avant envoi	81
7. Démonstration pas à pas — cinq étapes sur le socle	51
7.1 Étape 1 — écrire l’en-tête	81
7.2 Étape 2 — nommer les étapes	81
7.3 Étape 3 — rejouer les 18 contrôles	81
7.4 Étape 4 — comparer les deux socles	81
7.5 Étape 5 — documenter ce qui n’a pas été testé	81
8. Erreurs fréquentes	16
9. Bonnes pratiques professionnelles	16
10. Exercice guidé	17
11. Exercices autonomes	17
12. Correction détaillée	18
13. Mini-projet de chapitre	19
14. Résumé du chapitre	19

15. À retenir	19
16. Évaluation formative	20
PROJET M11.P — « LES 12 RAPPORTS SQL DE LA CELLULE COMMERCIALE »	86
1. Énoncé	86
La commande	86
Le matériel	86
Les six totaux de contrôle	86
2. Les 12 questions métier (le catalogue imposé)	87
Le format d'en-tête, imposé	87
3. Les 4 livrables	88
P1 — Les 12 requêtes (8 points)	88
P2 — Le jeu de tests (5 points)	88
P3 — La note de lecture (4 points · 2 pages)	89
P4 — La fiche de reprise (3 points)	89
4. Barème	89
5. Ce qui fait rejeter un livrable	90
6. Déroulé conseillé (4 séances, environ 8 h)	90
7. Comment vous corriger vous-même	90
ÉVALUATION M11 — « SQL AVANCÉ POUR LA BI »	91
A · Questions de récupération (non notées)	91
B · Quiz (20 questions · 20 points · 1 point par question)	91
Bloc 1 — Quelle requête pour quelle question (C01—C05) (Q1 à Q5)	91
Bloc 2 — Prédisez le résultat (Q6 à Q10)	92
Bloc 3 — Diagnostic (Q11 à Q15)	92
Bloc 4 — Vocabulaire (Q16 à Q20)	92
C · Exercices à rendre (20 points · 5 exercices, dont 2 de type E4)	93
E1 — Le top 3 par famille, deux fois (niveau 3 · 3 points)	93
E2 — Les deux périmètres de la concentration (niveau 3 · 3 points)	93
E3 — Le suivi des objectifs, trous compris (niveau 3 · 4 points)	93
E4 — « Ce rapport annonce un chiffre d'affaires net » (type E4 · niveau 4 · 5 points)	93
E5 — « Ce classement change de réponse à chaque exécution » (type E4 · niveau 4 · 5 points)	94
D · Auto-test SQL (10 points · 18 requêtes · auto-corrigé)	94
E · Étude de cas — « 10 % des clients font-ils 60 % du CA ? » (20 points · seuil 12)	94
La commande	86
Les quatre productions attendues	94
Le résultat attendu (à retrouver, pas à recopier)	95
Barème de l'étude de cas	95
Correction détaillée	95
A · Réponses de récupération	95
B · Corrigé du quiz (1 point par question)	95
C · Corrigé des exercices à rendre	96
D · Corrigé de l'auto-test	97
E · Corrigé de l'étude de cas — les quatre attendus	97
Barème global et seuils	97

Module M11.C01 — La fonction de fenêtre : `OVER`, `PARTITION BY`, `ORDER BY`, et pourquoi `GROUP BY` ne suffit plus

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5). **Durée indicative :** 5 h. **Niveau :** N3 → N4. **Pré-requis :** M07 (tout : `SELECT`, agrégats, `GROUP BY`, jointures, sous-requêtes) ; M06.C04 (DuckDB, SQLite, PostgreSQL) ; M02 (moyenne, médiane, dispersion).

L'idée du chapitre. M07 a appris à **réduire** : `GROUP BY` transforme des milliers de lignes en quelques totaux. C'est exactement ce qu'il faut quand la question est « combien ». Mais la question d'un responsable est presque toujours « **par rapport à quoi** » — quelle part du total, quel écart à la moyenne, où se situe cette ligne dans son groupe. Et là, `GROUP BY` a un défaut rédhibitoire : il **supprime la ligne** dont on parle. La fonction de fenêtre fait l'inverse : elle calcule le contexte **et laisse la ligne en place**. Ce chapitre pose le mécanisme (`OVER`, `PARTITION BY`, `ORDER BY`, cadre de fenêtre) et surtout la question préalable à toute requête de BI — **comparer à quoi ?** — avec un contre-exemple mesuré qui coûte cher : oublier `PARTITION BY` ne provoque aucune erreur, et divise la réponse par cinq.

Matériel de l'atelier — DuckDB 1.5.5 · Python 3.13 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). Le socle est un **script** de 3 595 octets : 5 petites tables (produits, magasins, vendeurs, objectifs, calendrier) et 2 vues sur les fichiers du socle M01-M03. Il se rejoue en mémoire en **0,4 s** et ne crée **aucune donnée nouvelle** : c'est le même fichier de ventes que M01 à M03 — **240 000** lignes, **237 191** ventes nettes (**2 809** retours), **44** mois, **5** magasins, **23 497** clients, **15 595 154 955** FCFA de chiffre d'affaires net. Toutes les sorties publiées dans ce chapitre sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Expliquer la différence d'usage** entre un agrégat (`GROUP BY`) et une **fonction de fenêtre** (window function) : l'un réduit la table, l'autre ajoute une colonne — et dire laquelle des deux répond à « quelle part du total ? ».
2. **Écrire et lire une fenêtre** : `OVER (`, `PARTITION BY`, `ORDER BY` à l'intérieur de la fenêtre, et un **cadre de fenêtre** (`ROWS BETWEEN ... AND ...`).
3. **Déterminer la base de comparaison** d'un chiffre — par magasin, par année, sur toute la table — et **prouver** votre choix en montrant que les parts somment à 100 % au niveau où elles doivent sommer.
4. **Reconnaître les trois pièges du chapitre** sur un résultat réel : la fenêtre évaluée après le `WHERE`, la jointure sur une clé incomplète, et la partition qui vaut le groupe (qui affiche toujours 100 %).
5. **Choisir entre une fenêtre et une sous-requête scalaire** pour ramener un « contexte » dans une requête agrégée, et justifier le choix autrement que par le goût.

2. Pourquoi cette notion est importante

Un responsable de magasin ne demande pas « quel est le chiffre d'affaires de la société ? ». Il demande « **quelle est ma part ?** ». Un responsable de rayon demande « **ce mois est-il**

bon ? », ce qui suppose un mois moyen à côté. Un directeur commercial demande « **qui sont mes dix meilleurs vendeurs ?** », ce qui suppose un classement **dans** un groupe. Ces trois questions ont un point commun : elles comparent une **ligne** à un **ensemble de lignes**. C’est le métier de la fenêtre.

Le socle du module rend l’enjeu immédiatement visible. Prenez les 5 magasins : un simple `GROUP BY` donne leurs totaux, et rien de plus. Pour obtenir leur **part**, il faut le total général — et le réflexe de M07 est d’écrire une sous-requête scalaire qui le ramène :

```
SELECT m.nom, SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc)
             / (SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour), 2) AS part_pct
FROM ventes v JOIN magasin m USING (id_magasin)
WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca DESC;
```

Cette requête est **juste**, et c’est celle que M07 vous aurait fait écrire. Elle a deux défauts, tous les deux mesurables : elle réclame **le total de la table** à chaque fois qu’on veut un pourcentage (ici une seconde lecture complète des 237 191 lignes), et elle devient illisible dès qu’on veut **deux** contextes — la part de la société et la part du magasin dans son propre réseau. Le même résultat s’écrit avec une fenêtre, et l’atelier a chronométré les deux écritures : **150 ms** pour la fenêtre, **292 ms** pour la sous-requête scalaire, soit **×1,9** pour la même sortie, ligne pour ligne.

Mais la vraie raison d’apprendre la fenêtre n’est pas la vitesse — les deux requêtes donnent exactement le même tableau. C’est que la fenêtre rend **visible** la question « comparer à quoi ? », parce qu’on doit l’écrire noir sur blanc : `OVER ()` dit toute la table, `OVER (PARTITION BY id_magasin)` dit dans son magasin, `OVER (PARTITION BY annee)` dit dans son année. La sous-requête scalaire cache ce choix dans une sous-requête qu’on ne relit pas. Et un choix caché, en BI, c’est un chiffre faux qui a l’air juste.

Le chiffre qui ouvre ce chapitre. Sur ce socle, le plus gros client identifié de la base n’est pas un client : c’est un identifiant **0** qui totalise **2 852 612 447** FCFA et **18,3 %** du chiffre d’affaires net du réseau — tout en étant **absent** du référentiel clients. Celui qui écrit une fenêtre de classement sans le savoir met en tête de son rapport commercial un client « non identifié » : la fenêtre ne protège de rien, elle exécute ce qu’on lui demande. C’est la leçon que le module déroulera jusqu’à l’étude de cas.

3. Explication simple — le tableau d’affichage du stade

Imaginez un match. Chaque spectateur (une ligne de la table) a un numéro de place, un prix payé, une tribune. À un moment, l’écran géant affiche trois informations : **le total du stade, le total de votre tribune, le nombre de spectateurs de votre tribune**.

- `GROUP BY`, c’est la **feuille de caisse** : on plie la foule en quelques tas, et les individus disparaissent. On sait qu’il y a 5 tas, on ne sait plus qui est dans lequel.
- La **fenêtre**, c’est l’**écran du stade** : chaque spectateur reste assis à sa place et **voit** l’information de son groupe. Le nombre de places ne change pas : 237 191 personnes restent 237 191 personnes, on a simplement ajouté trois colonnes à leur billet.

Trois conséquences, qui sont tout le chapitre :

1. **Le nombre de lignes ne change pas.** Si votre requête avec fenêtre rend 5 lignes là où vous attendiez 237 191, vous avez écrit un `GROUP BY`, pas une fenêtre.
2. **Ce que voit chaque ligne dépend de la fenêtre.** `OVER ()` = tout le stade ; `OVER (PARTITION BY tribune)` = votre tribune. Changez la fenêtre, changez le chiffre — sans changer une seule donnée.

3. **L'ordre compte dès qu'on demande un cumul.** Un cumul (« total depuis l'ouverture ») n'existe que **dans un ordre** : il faut dire lequel (`ORDER BY`), sinon le moteur choisit, et vous ne saurez pas quoi.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
fonction de fenêtre	window function	Un calcul qui regarde un ensemble de lignes liées à la ligne courante sans réduire le nombre de lignes du résultat.
fenêtre	window	L'ensemble de lignes que la fonction voit : décrit par <code>OVER (...)</code> .
partition	partition	La découpe de la fenêtre en groupes indépendants (<code>PARTITION BY id_magasin</code>) : le facteur de comparaison .
ordre de fenêtre	window order	L'ordre à l'intérieur de la partition (<code>ORDER BY date_vente</code>) : il rend possibles cumuls, rangs et décalages.
cadre de fenêtre	window frame	Les lignes effectivement prises en compte autour de la ligne courante (<code>ROWS BETWEEN 2 PRECEDING AND CURRENT ROW</code>).
part relative	share of total	Le rapport d'une ligne au total de sa fenêtre ; la somme des parts vaut 100 % au niveau de la partition .
sous-requête scalaire	scalar subquery	Une sous-requête qui rend une seule valeur , utilisée comme un nombre dans une expression — l'ancêtre de la fenêtre.
ordre d'exécution	logical order of execution	L'ordre dans lequel le moteur interprète les clauses (<code>FROM</code> → <code>WHERE</code> → <code>GROUP BY</code> → fenêtres → <code>SELECT</code> → <code>ORDER BY</code>).
grain	grain	Ce que représente une ligne du résultat : le vérifier avant de joindre, de fenêtrer et de conclure.
total de contrôle	control total	Le chiffre qu'un rapport doit retrouver coûte que coûte (ici 15 595 154 955 FCFA) et qui démasque toute multiplication de lignes.

Définition. Une **fonction de fenêtre** — window function — est un calcul qui prend en entrée un **ensemble de lignes** (la fenêtre) et rend **une valeur par ligne**. Elle s'écrit toujours après un nom de fonction, avec le mot-clé `OVER` : `SUM(x) OVER (...)`, `AVG(x) OVER (...)`, `COUNT(*) OVER ()`. Sans `OVER`, la même fonction est un agrégat ordinaire et réduit la table.

Définition. La **partition** — partition — est la subdivision de la fenêtre en groupes **indépendants les uns des autres**. Deux partitions ne se voient pas : un `SUM` de fenêtre partitionné par magasin ne connaît jamais le total de la société. C'est exactement ce qu'on veut quand la bonne question est « quelle part de son marché ? ».

5. Cours approfondi

5.1 L'ordre d'exécution : pourquoi une fenêtre n'est pas un agrégat

Le moteur lit une requête dans un ordre qui n'est pas celui de l'écriture :

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → FENÊTRES → `SELECT` → `DISTINCT` → `ORDER BY` → `LIMIT`

Deux conséquences se paient comptant, et elles sont mesurées au §5.7 :

1. **Une fenêtre s'évalue après `GROUP BY`.** Dans une requête agrégée, elle ne peut donc partitionner que sur ce qui **existe** à cet étage : une colonne de regroupement ou une expression agrégée. Écrire `SUM(SUM(x)) OVER (PARTITION BY colonne_non_groupee)` fait échouer la requête — et le message d'erreur de DuckDB dit précisément pourquoi (voir §8, erreur n°1).

2. **Une fenêtre s'évalue après WHERE.** Filtrer une ligne avant de calculer une part ne laisse plus qu'une ligne dans la fenêtre : la part devient **100,0 %**. Mesuré au §7.5 sur janvier 2023 : **8,3 %** quand le filtre est appliqué **après** le calcul, **100,0 %** quand il est appliqué avant.

5.2 OVER () : la fenêtre sur toute la table

La fenêtre vide dit « **toutes les lignes du résultat** ». C'est la forme qui ramène le total général à côté de chaque ligne :

```
SELECT m.nom AS magasin,
       SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc) / SUM(SUM(v.montant_ttc)) OVER (), 2) AS part_pct
FROM ventes v JOIN magasin m USING (id_magasin)
WHERE NOT v.est_retour
GROUP BY 1 ORDER BY ca DESC;
```

La double écriture `SUM(SUM(v.montant_ttc)) OVER ()` surprend la première fois : **le SUM intérieur est l'agrégat du GROUP BY** (il fabrique le total du magasin), **le SUM extérieur est la fenêtre** (il additionne ces totaux, donc toute la table). On lit la requête de l'intérieur vers l'extérieur : regrouper par magasin, puis comparer chaque total au total des totaux.

Dans les faits. L'atelier a exécuté les deux écritures sur le socle et comparé les sorties ligne à ligne : **identiques**. Seul le chronomètre les distingue — **150 ms** pour la fenêtre, **292 ms** pour la sous-requête scalaire. La fenêtre ne « calcule pas mieux » : elle évite de redemander le total à chaque ligne. Et elle a un avantage décisif en relecture : le total y est **visible** dans le texte de la requête.

5.3 PARTITION BY : le facteur de comparaison

`PARTITION BY` découpe la fenêtre. C'est **le** paramètre du chapitre, celui qui décide du sens du chiffre. La même mesure, trois partitions, trois lectures :

Ce qu'on écrit	La question à laquelle on répond	Ce que la somme des parts vaut
<code>OVER ()</code>	quelle part de toute la société ?	100 % sur l'ensemble du résultat
<code>OVER (PARTITION BY id_magasin)</code>	quelle part de son magasin ?	100 % dans chaque magasin
<code>OVER (PARTITION BY annee)</code>	quelle part de son année ?	100 % dans chaque année

Le tableau ci-dessus est aussi la **procédure de vérification** : après avoir écrit une part, on l'additionne au niveau de sa partition. Si l'on n'obtient pas 100 %, la partition ne décrit pas ce qu'on raconte.

À retenir. Dans une requête où le `GROUP BY` et le `PARTITION BY` portent sur la même colonne, la part vaut **toujours 100,0 %** : chaque groupe compare son total à lui-même. C'est le résultat le plus vide qui existe, il a l'air parfaitement crédible sur un rapport, et c'est l'erreur n°3 du §8.

5.4 ORDER BY dans la fenêtre : l'ordre change la valeur

Sans `ORDER BY`, la fenêtre est « toutes les lignes de la partition » : `SUM(...) OVER (PARTITION BY m)` donne le total du magasin sur **chaque** ligne du magasin. Avec `ORDER BY` **dans** la fenêtre, la même fonction devient **cumulative** : elle additionne de la première ligne jusqu'à la ligne courante. Attention, cet `ORDER BY` **n'est pas** celui du résultat : c'est lui qui définit la progression du calcul. Une requête peut très bien cumuler par date (`ORDER BY date_vente` dans la fenêtre) et **rendre** les lignes triées par montant décroissant (`ORDER BY ca DESC` à la fin).

Définition. Le **cadre de fenêtre** — window frame — précise **quelles lignes** autour de la ligne courante la fonction voit : `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` (les deux précédentes et la courante), `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` (depuis le début), `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` (toute la partition). Dès qu'on écrit `ORDER BY` dans une fenêtre, le cadre par défaut devient « du début à la ligne courante » : c'est lui qui transforme un `SUM` en cumul.

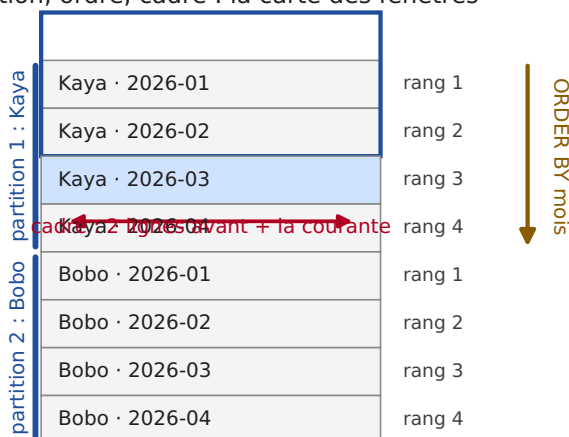
5.5 Le cadre : cumul et moyenne glissante

Sur les cinq premières ventes du magasin 1, l'atelier a publié le cumul et la moyenne glissante de trois lignes. Les valeurs du cumul sont **53 690 ; 55 729 ; 76 969 ; 134 081 ; 171 605** FCFA : chaque ligne ajoute son montant à la précédente. La moyenne glissante de trois lignes, elle, vaut **53 690 ; 27 865 ; 25 656 ; 26 797 ; 38 625** FCFA : elle ne monte pas, elle **suit** — la première ligne n'a qu'elle-même, la deuxième en a deux, les suivantes en ont trois.

C'est toute la différence entre un indicateur de **niveau** (le cumul, qui dépend du passé entier) et un indicateur de **tendance** (la moyenne glissante, qui oublie). Les deux se calculent avec la même fonction `SUM/AVG` : seul le **cadre** change.

Attention. Le cumul dépend de l'`ORDER BY` de la fenêtre, donc du **départage** entre lignes de même date. Le socle contient des milliers de ventes par jour : `ORDER BY date_vente` seul laisse le moteur choisir l'ordre des lignes d'une même journée, et le cumul de midi n'est plus celui du matin. Écrivez `ORDER BY date_vente, id_vente` — la clé de départage, ici l'identifiant unique de la ligne.

Partition, ordre, cadre : la carte des fenetres



La partition separe les groupes de lignes.

Le cadre decide ce qui entre dans le calcul.

5.6 La même question, deux écritures : fenêtre contre sous-requête scalaire

Définition. Une **sous-requête scalaire** — scalar subquery — est une sous-requête placée dans une expression et qui rend **une seule valeur** (une ligne, une colonne) : `SUM(x) / (SELECT SUM(y) FROM t)`. C'est la façon dont on écrivait un contexte avant les fenêtres. Elle reste légitime pour une constante — un seuil, un taux de référence — et devient illisible dès qu'il faut deux contextes différents.

Critère	Fonction de fenêtre	Sous-requête scalaire
Sortie	identique	identique
Coût mesuré ici	150 ms	292 ms
Le choix de comparaison est-il visible ?	oui, dans <code>OVER (...)</code>	non, enfoui dans la sous-requête
Deux contextes dans la même requête ?	oui (<code>OVER ()</code>) et <code>OVER (PARTITION BY ...)</code>	oui, mais deux sous-requêtes
S'écrit sur une requête agrégée ?	oui (<code>SUM(SUM(x)) OVER (...)</code>)	oui (<code>SUM(x) / (SELECT ...)</code>)
S'ajoute à une table non agrégée ?	oui, directement	oui, mais une sous-requête par contexte

La règle de travail du module : **dès qu'un chiffre exprime une comparaison, la fenêtre est la forme attendue** — parce que la relecture doit pouvoir vérifier le facteur de comparaison sans exécuter la requête.

Attention. La vitesse ne justifie pas un choix de syntaxe. Les **150 ms** contre **292 ms** de l'atelier dépendent de la machine, de la version du moteur et de la taille du socle : sur dix lignes, la sous-requête scalaire sera plus rapide. Écrivez la fenêtre parce que le **facteur de comparaison y est explicite** — c'est un argument de relecture, pas un argument de performance. Le seul argument de performance du module qui survive à un changement de machine est celui du chapitre C06, et il est mesuré sur un plan d'exécution.

5.7 Les trois pièges, mesurés

1. **La fenêtre s'évalue après le WHERE** : janvier 2023 vaut **8,3 %** de son année quand la fenêtre est calculée puis filtrée ; **100,0 %** quand filtre et fenêtre sont dans le même `SELECT`. Le résultat « 100,0 % » est faux et n'émet aucun avertissement.
2. **La jointure sur une clé incomplète** : joindre les ventes à une table de totaux **mensuels par magasin** en oubliant `id_magasin` dans la condition multiplie chaque ligne par le nombre de magasins du mois. Le rapport annonce alors **77 975 774 775 FCFA** sur **1 185 955** lignes — soit cinq fois le total du socle, et le contrôle des 5 magasins « fonctionne » toujours.
3. **La partition qui vaut le groupe** : `GROUP BY YEAR(date_vente)` avec `OVER (PARTITION BY YEAR(date_vente))` affiche 100,0 % pour chaque année. La requête est valide, la réponse est vide.

Conseil professionnel. Terminez chaque rapport de comparaison par un **total de contrôle** : un `SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour` qui doit rendre **15 595 154 955 FCFA**. C'est cinq secondes d'écriture, et c'est la seule ligne qui attrape les pièges 2 et 3 — les deux seuls du chapitre qui produisent un nombre **plus grand** ou **trop rond** sans jamais lever d'erreur.

5.8 Ce que ce chapitre ne couvre pas

Les **rang**s (`ROW_NUMBER`, `RANK`, `NTILE`), les **décalages** (`LAG`, `LEAD`) et les **variations** d'une période à l'autre sont le sujet de C02 : ce sont les mêmes fenêtres, appliquées à l'ordonnancement. Le traitement du **temps** (mois sans vente à fabriquer, comparaison inter-annuelle, moyennes mobiles de 12 mois) est celui de C03. Ici, on ne fait que comparer une ligne à un ensemble — et on apprend à dire lequel.

6. Exemple concret — la part de chaque magasin, et ce qu'elle décide

Le directeur commercial demande le poids de chaque magasin. La requête de référence rend **5 lignes** :

Magasin	CA net (FCFA)	Part du réseau
Sahel Distribution — Ouaga 2000	5 312 205 459	34,06 %
Sahel Distribution — Gounghin	3 424 978 244	21,96 %
Sahel Distribution — Bobo Kibidwé	3 099 946 956	19,88 %
Sahel Distribution — Koudougou Centre	2 024 794 213	12,98 %
Sahel Distribution — Kaya Marché	1 733 230 083	11,11 %
Total	15 595 154 955	100,00 %

Trois lectures que ce tableau autorise — et une qu'il interdit :

- **Le premier magasin fait plus du tiers du réseau** (34,06 %). C'est un fait de structure, pas de performance : il est plus grand (Ouaga 2000), il vend plus, et sa part est stable.
- **Le rapport entre le premier et le dernier vaut 3,06**. Toute action commerciale « pour égaliser » les magasins se heurte donc à un écart de taille, pas à un écart de talent — la surface, le bassin de population et l'assortiment ne sont pas dans la table.
- **La somme des parts vaut 100,00 %** : la partition est bien le réseau entier. C'est la preuve exigée au §5.3, et elle tient en une addition.

La lecture **interdite** est celle qu'un rapport pressé écrit tout de suite : « Ouaga 2000 porte le réseau ». Faux : c'est le magasin le plus gros d'un réseau de cinq, et rien dans ces cinq lignes ne dit qu'il tire les autres. Une part n'est pas une contribution au résultat — question qui exigera la marge (C06 du module M13 et les coûts d'achat du socle).

7. Démonstration pas à pas — six étapes sur le socle

Toutes les commandes de cette section ont été exécutées dans l'atelier. Le socle se reconstruit d'abord :

```
import sys; sys.path.insert(0, "03_exercices/dossier_M11")
from connexion import ouvrir
con = ouvrir()                                # 0,4 s : le script du socle est rejoué en mémoire
```

7.1 Étape 1 — le total de contrôle

```
SELECT COUNT(*), COUNT(*) FILTER (WHERE NOT est_retour), SUM(montant_ttc) FILTER (WHERE NOT
est_retour)
FROM ventes;
```

```
240000 | 237191 | 15595154955
```

C'est la ligne à retrouver à la fin de **chaque** rapport du module.

7.2 Étape 2 — la part, avec la fenêtre

```
SELECT m.nom AS magasin, SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc) / SUM(SUM(v.montant_ttc)) OVER (), 2) AS part_pct
FROM ventes v JOIN magasin m USING (id_magasin)
WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca DESC;
```

Sahel Distribution — Ouaga 2000		5312205459		34.06
Sahel Distribution — Gounghin		3424978244		21.96
Sahel Distribution — Bobo Kibidwé		3099946956		19.88
Sahel Distribution — Koudougou Centre		2024794213		12.98
Sahel Distribution — Kaya Marché		1733230083		11.11

7.3 Étape 3 — la même réponse par sous-requête scalaire

```
SELECT m.nom, SUM(v.montant_ttc) AS ca,
       ROUND(100.0 * SUM(v.montant_ttc) / (SELECT SUM(montant_ttc) FROM ventes WHERE NOT
est_retour), 2)
FROM ventes v JOIN magasin m USING (id_magasin) WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca
DESC;
```

Sortie **identique**, colonne par colonne. La comparaison des deux écritures porte donc sur la lisibilité et le coût (**150 ms** contre **292 ms**), pas sur la justesse.

7.4 Étape 4 — une part calculée dans son année

```
SELECT strftime(date_trunc('month', date_vente), '%Y-%m') AS mois, YEAR(date_vente) AS annee,
       ROUND(SUM(montant_ttc)) AS ca,
       ROUND(100.0 * SUM(montant_ttc) / SUM(SUM(montant_ttc)) OVER (PARTITION BY annee), 1) AS
part_annee_pct
FROM ventes WHERE NOT est_retour GROUP BY 1, 2 ORDER BY 1;
```

2023-01		2023		284400304		8.3
2023-02		2023		263183276		7.6
...						
2023-12		2023		333819159		9.7

44 lignes, une par mois, chacune portant la part de son mois **dans son année**. Janvier 2023 pèse **8,3 %** de 2023.

7.5 Étape 5 — le piège : le filtre et la fenêtre dans le même SELECT

```
WITH m AS (SELECT strftime(date_trunc('month', date_vente), '%Y-%m') am, YEAR(date_vente) an,
                  SUM(montant_ttc) ca FROM ventes WHERE NOT est_retour GROUP BY 1, 2)
SELECT ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY an), 1) FROM m WHERE am = '2023-01';
```

100.0

Le **WHERE** s'exécute **avant** la fenêtre : au moment du calcul, la partition ne contient plus qu'une seule ligne, celle qu'on vient de garder. La bonne écriture filtre **après**, dans une requête englobante :

```
WITH m AS (...)
SELECT part FROM (SELECT am, ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY an), 1) part FROM m)
WHERE am = '2023-01';
```

8.3

Deux requêtes, deux réponses, un seul mot d'écart : l'emplacement du filtre. **8,3 %** est la part du mois dans son année ; **100,0 %** est un artefact.

7.6 Étape 6 — la fenêtre n'a pas réduit la table

```
SELECT id_vente, id_magasin, ROUND(montant_ttc) AS montant,
       COUNT(*) OVER () AS lignes_du_socle, ROUND(AVG(montant_ttc) OVER ()) AS moyenne_generale
FROM ventes WHERE NOT est_retour LIMIT 5;
```

47820	1	53690	237191	65749
92080	1	2039	237191	65749
92628	1	21240	237191	65749
99471	1	57112	237191	65749
119868	1	37524	237191	65749

Chaque ligne a **conservé** son identifiant, son magasin et son montant, et **reçu en plus** deux informations de contexte : **237 191** lignes au total et **65 749** FCFA de montant moyen par ligne. Une requête `GROUP BY` ne peut pas publier cela : elle aurait rendu **1** ligne.

8. Erreurs fréquentes

1. **Partitionner sur une colonne qui n'est plus là après `GROUP BY`.** Le moteur refuse : `column "date_vente" must appear in the GROUP BY clause or must be part of an aggregate function`. Ce n'est pas une brimade : à l'étage de la fenêtre, la colonne a déjà été remplacée par un groupe. La correction est d'ajouter l'**expression** au `GROUP BY` (ici `YEAR(date_vente) AS annee`) puis de partitionner sur l'alias. Retenez la règle : **une fenêtre ne partitionne que sur ce qui existe à son étage**.
2. **Filtrer avant de fenêtrer.** Le `WHERE` étant évalué avant les fenêtres, filtrer un mois puis calculer sa part sur l'année laisse une seule ligne dans la partition : **100,0 %** au lieu de **8,3 %**. Toute comparaison « cette ligne par rapport à son ensemble » doit filtrer **après** le calcul — donc dans une requête englobante.
3. **Partitionner sur sa propre clé de regroupement.** `GROUP BY annee + PARTITION BY annee` rend 100 % partout. Le chiffre est exact, la conclusion est nulle : si votre rapport de parts ne contient que des 100 %, la partition n'a pas été choisie, elle a été recopiée.
4. **Joindre une table de totaux sans vérifier son grain.** La table des totaux mensuels par magasin contient **220** lignes (une par couple magasin-mois). Jointe sur le mois **seul**, elle apporte **5** lignes de totaux à chaque vente, et le rapport publie **77 975 774 775** FCFA au lieu de **15 595 154 955** : **5** fois trop, **1 185 955** lignes au lieu de **237 191**, et aucun message d'erreur. Le grain de la table jointe est **la** chose à vérifier avant la jointure.
5. **Croire qu'un cumul est un indicateur de niveau.** Un cumul dépend de la date de début, de l'ordre choisi et du départage des ex æquo : deux analystes honnêtes peuvent en publier deux valeurs différentes du même socle. Préciser `ORDER BY` **et** la clé de départage, sinon le cumul n'est pas reproductible.

9. Bonnes pratiques professionnelles

1. **Écrire les fenêtres dans cet ordre : le grain, la partition, l'ordre, le cadre.** Une requête de fenêtre se relit toujours dans le même sens : quelle ligne par ligne ? puis comparée à quoi ? puis dans quel ordre ? puis sur quelle profondeur ?. Si l'une des quatre réponses manque, la requête est incomplète même si elle tourne.

2. **Appliquer la preuve de la partition.** Après toute part relative, contrôler que la somme vaut 100 % au niveau voulu : sur tout le résultat (`OVER ()`), par magasin, par année. Sans ce contrôle, l'erreur n°3 passe en production.
3. **Choisir la fenêtre pour ce qui se compare, la sous-requête pour ce qui s'isole.** Une constante (le taux de TVA de référence, un seuil) se lit par sous-requête ou par paramètre ; un **contexte changeant** (le total d'un groupe, la moyenne d'une famille) s'écrit en fenêtre, parce que le contexte doit être **vu**.
4. **Toujours nommer les colonnes de contexte.** `COUNT(*) OVER ()` se nomme `lignes_du_socle`, pas `c1`. Un rapport de BI est lu six mois plus tard par quelqu'un qui n'a pas la requête sous les yeux.
5. **Terminer par le total de contrôle. 15 595 154 955 FCFA.** Un écart de ce total signale une jointure multiplicatrice (erreur n°4) avant que la conclusion ne soit écrite.

10. Exercice guidé

Situation. Vous produisez le rapport « poids des vendeurs » pour le comité de direction. Le socle contient **22** vendeurs. Votre stagiaire a publié ce tableau, qui a beaucoup plu :

Vendeur	Part du CA	Rang
Vendeur 7	8,68 %	1
Vendeur 21	8,44 %	2
Vendeur 12	7,89 %	3

Consigne. Dans les 20 minutes (chrono), sans ouvrir `ATTENDU.json` :

1. Écrivez la requête qui produit exactement ces trois colonnes, avec la part en fenêtre. (3 pts)
2. Ajoutez la **preuve** que la partition choisie est la bonne : une colonne ou une requête qui montre à quoi les parts additionnent. (3 pts)
3. Le stagiaire jure avoir testé : « la somme des colonnes fait 100 % ». Expliquez en deux lignes pourquoi son test ne prouve rien ici. (2 pts)
4. Le comité demande maintenant la part **par magasin** pour chaque vendeur (un vendeur appartient à un magasin). Modifiez la fenêtre. (2 pts)
5. Écrivez le **total de contrôle** que vous joindrez au rapport, et dites ce qu'un écart révélerait. (2 pts)

11. Exercices autonomes

E1 — Le mois dans son année, dans son magasin, dans son réseau (35 min). Sur le socle, produisez une seule requête rendant pour chaque couple **magasin-mois** (220 lignes) : le CA, sa part dans l'année du magasin, sa part dans le réseau pour ce mois. Les trois colonnes doivent sommer à 100 % **au niveau de leur partition** — joignez à votre rendu le contrôle qui le prouve. Contrainte : aucune sous-requête scalaire, uniquement des fenêtres.

E2 — Le contrôle de grain (25 min). Construisez la table des totaux mensuels par magasin, puis écrivez **deux** jointures : l'une sur le mois seul, l'autre sur les deux clés. Publiez les deux totaux, expliquez l'écart par le nombre de lignes, et déduisez-en la phrase que vous diriez à un collègue pour l'aider à détecter ce piège dans son propre rapport.

12. Correction détaillée

Exercice guidé.

1. **La requête.** Le classement des vendeurs se calcule sur les ventes nettes, groupées par vendeur :

```
sql SELECT v.id_vendeur, SUM(v.montant_ttc) AS ca, ROUND(100.0 * SUM(v.montant_ttc) /
SUM(SUM(v.montant_ttc)) OVER (), 2) AS part_pct FROM ventes v WHERE NOT v.est_retour GROUP BY 1
ORDER BY ca DESC LIMIT 3;
```

La fenêtre est `OVER ()` : la partition est le **réseau entier**, ce qui correspond à la question posée (« poids de chaque vendeur dans la société »). 2. **La preuve.** Une part ne se valide pas sur son maximum, elle se valide sur sa **somme** au niveau de la partition :

```
sql WITH p AS (SELECT id_vendeur, SUM(montant_ttc) ca FROM ventes WHERE NOT est_retour GROUP BY
1) SELECT ROUND(SUM(100.0 * ca / SUM(ca) OVER ()), 2) FROM p; -- 100.00
```

Vingt-deux lignes, une somme, un verdict. 3. **Pourquoi le test du stagiaire ne prouve rien.** Il a vérifié que **ses trois lignes publiées** totalisent moins de 100 % — ou pire, il a additionné les parts de tout le monde sans le dire. Un contrôle de part se fait **sur la partition complète**, jamais sur l'extrait qu'on publie : un classement tronqué au top 3 somme par construction à 24,99 %, et cela ne dit rien de la justesse de la partition. 4. **La part dans le magasin.** On ajoute `id_magasin` au `GROUP BY` (il faut qu'il existe à l'étage de la fenêtre), puis on partitionne dessus :

```
sql SELECT id_magasin, id_vendeur, SUM(montant_ttc) AS ca, ROUND(100.0 * SUM(montant_ttc) /
SUM(SUM(montant_ttc)) OVER (PARTITION BY id_magasin), 2) FROM ventes WHERE NOT est_retour GROUP
BY 1, 2 ORDER BY 1, ca DESC;
```

Le même vendeur change de part **sans qu'aucune donnée n'ait changé** : c'est le sens du chapitre. 5. **Le total de contrôle.** `SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour;` doit rendre **15 595 154 955** FCFA. Un écart signalerait une jointure multiplicatrice (chaque ligne comptée plusieurs fois) ou un filtre sur les retours oublié — les deux erreurs qui font un chiffre trop gros.

E1 (les trois parts emboîtées). La structure attendue : le `GROUP BY` fabrique les couples magasin-mois avec leur CA, puis **trois fenêtres** calculent trois contextes différents sur cette même table agrégée.

```
WITH m AS (SELECT id_magasin, YEAR(date_vente) an,
                strftime(date_trunc('month', date_vente), '%Y-%m') am, SUM(montant_ttc) ca
            FROM ventes WHERE NOT est_retour GROUP BY 1, 2, 3)
SELECT id_magasin, am, ca,
       ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY id_magasin, an), 2) AS part_dans_l_annee,
       ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY am), 2) AS part_du_mois_reseau
FROM m ORDER BY am, id_magasin;
```

Le contrôle : `SUM(part_dans_l_annee)` sur un magasin et une année vaut 100,00 (les 12 mois, ou 8 en 2026) ; `SUM(part_du_mois_reseau)` sur un mois vaut 100,00 (les 5 magasins). Le même CA, deux partitions, deux lectures — et un tableau qui ne somme à 100 % à aucun autre niveau.

E2 (le contrôle de grain). La table des totaux compte **220** lignes. Jointe sur le mois **seul**, chaque vente rencontre 5 lignes de totaux : le résultat passe à **1 185 955** lignes et le total annoncé à **77 975 774 775** FCFA, cinq fois **15 595 154 955**. Jointe sur les deux clés, on retrouve **237 191** lignes et le total juste. La phrase à dire à un collègue : « avant de joindre une table de totaux, écris `COUNT(*)` **avant** et **après** la jointure : si le nombre de lignes augmente, ta clé de jointure est incomplète ».

13. Mini-projet de chapitre

La note de trois pages : « la part et l'écart ». Vous produisez, pour le comité de la quincaillerie, une note de 3 pages maximum portant sur le socle M11 (240 000 lignes, 44 mois, 5 magasins).

Livrables et barème.

Critère	Ce qui est vérifié	Points
La requête de part	une fenêtre <code>OVER ()</code> , avec le total de contrôle 15 595 154 955 FCFA	6
La part par magasin et dans l'année	deux partitions dans la même note, chacune avec sa preuve à 100 %	5
Le piège documenté	le filtre avant/après fenêtre, chiffré (8,3 % contre 100,0 %)	4
La phrase de conclusion	ce que la note autorise à dire, et ce qu'elle interdit de dire	3
Total		18

14. Résumé du chapitre

`GROUP BY` réduit une table à ses groupes ; une **fonction de fenêtre** ajoute à chaque ligne une information calculée sur un **ensemble** de lignes, sans en supprimer aucune. Cet ensemble se déclare : `OVER ()` pour toute la table, `OVER (PARTITION BY ...)` pour un groupe, avec un `ORDER BY` interne dès qu'il s'agit de cumuler, et un **cadre** (`ROWS BETWEEN ...`) pour décider de la profondeur du regard. Le facteur de comparaison est **le** contenu de la requête : `OVER ()` répond « part de la société », `PARTITION BY id_magasin` répond « part de son magasin », et la même mesure change de sens sans qu'une donnée change. Trois pièges, tous mesurés ici : le `WHERE` qui s'exécute **avant** la fenêtre (**100,0 %** au lieu de **8,3 %**), la jointure sur une clé incomplète (**77 975 774 775** FCFA au lieu de **15 595 154 955**), la partition recopiée du `GROUP BY` (100 % partout). Aucun des trois ne lève d'erreur — d'où la règle de la maison : **terminer chaque rapport par son total de contrôle**.

15. À retenir

- `GROUP BY` répond « combien », la fenêtre répond « par rapport à quoi ».** Si votre question contient « part », « écart », « rang » ou « cumul », il vous faut une fenêtre.
- Le nombre de lignes ne change pas.** Une requête à fenêtre sur 237 191 lignes rend 237 191 lignes ; si vous en obtenez 5, vous avez écrit un `GROUP BY`.
- La partition est une décision, pas une syntaxe.** Elle doit être écrite dans la requête, prouvée par une somme à 100 %, et relisible six mois plus tard.
- La fenêtre s'évalue après le `WHERE` et après le `GROUP BY`.** Filtrer avant de comparer fausse la comparaison ; partitionner sur une colonne non regroupée fait échouer la requête.
- Un rapport de comparaison sans total de contrôle est un pari.** Le total du socle est **15 595 154 955** FCFA, et il tient en une ligne de SQL.

À retenir. Une fonction de fenêtre ne change pas les données, elle change ce qu'on peut **dire** des données : elle rend une ligne comparable à un ensemble, explicitement.

À retenir. La question à se poser avant d'écrire `OVER ()` n'est jamais « quelle syntaxe ? » mais « **comparé à quoi ?** » — et la réponse doit figurer dans la requête, pas dans la tête de l'analyste.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Quelle est la différence, en une phrase, entre `SUM(x)` et `SUM(x) OVER ()` dans une requête **sans** `GROUP BY` ?
2. Combien de lignes rend `SELECT id_magasin, COUNT(*) OVER () FROM ventes WHERE NOT est_retour` ?
3. Dans `SUM(SUM(x)) OVER (PARTITION BY annee)`, que fait le `SUM` intérieur ? Que fait celui de la fenêtre ?
4. Vous lisez un rapport de parts où chaque ligne affiche 100,0 %. Quelle est la cause la plus probable ?
5. Vous voulez la part de janvier 2023 dans l'année 2023. Où placez-vous le filtre sur le mois, et pourquoi ?
6. Que vaut la somme des parts de tous les mois d'une année si la partition est `PARTITION BY annee` ?
7. Que se passe-t-il si vous joignez une table de totaux mensuels par magasin sur le mois seul ?
8. Un cumul est-il reproductible sans clé de départage ? Justifiez en trois mots.
9. Quelle est la forme la plus courte du total de contrôle du module ?
10. Pourquoi une part publiée sur un top 3 ne peut-elle pas être validée en additionnant ses trois lignes ?

Réponses. 1. `SUM(x)` sans `GROUP BY` réduit toute la table à une valeur ; `SUM(x) OVER ()` rend une valeur **par ligne**, identique partout. 2. **237 191** lignes — une par vente nette ; la fenêtre ne réduit pas. 3. Le `SUM` intérieur agrège le groupe (le mois) ; la fenêtre additionne ces totaux d'année. 4. Le `PARTITION BY` porte sur la même colonne que le `GROUP BY` : chaque groupe se compare à lui-même ; c'est l'erreur n°3. 5. **Après** le calcul de la part, dans une requête englobante — sinon la partition ne contient plus qu'une ligne et la part vaut 100,0 % au lieu de **8,3** %. 6. 100,00 %, et c'est la preuve attendue. 7. Chaque vente rencontre les totaux de tous les magasins du mois : **5** fois trop de lignes (**1 185 955** au lieu de **237 191**) et **77 975 774 775** FCFA au lieu de **15 595 154 955**. 8. Non : sans clé de départage, l'ordre des ex æquo est choisi par le moteur. 9. `SELECT SUM(montant_ttc) FROM ventes WHERE NOT est_retour` → **15 595 154 955** FCFA. 10. Parce qu'un classement tronqué ne somme à 100 % à aucun niveau : la preuve se fait sur la **partition complète**, pas sur l'extrait publié.

Module M11.C02 — Classer, cumuler, décaler :

`ROW_NUMBER`, `RANK`, `DENSE_RANK`, `NTILE`, `SUM() OVER`, `LAG`, `LEAD`

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5). **Durée indicative :** 5 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 (fenêtres, `OVER`, `PARTITION BY`, cadre) ; M07.C03 (`ORDER BY`, `LIMIT`), M07.C05 (`GROUP BY`) ; M02 (moyenne, médiane).

L'idée du chapitre. C01 a montré comment **comparer** une ligne à un ensemble. C02 en fait un usage qui produit des chiffres faux à une fréquence inquiétante : **ranger**. Un classement est une affirmation — « ce vendeur est le premier », « ce client fait partie des 10 % les plus fidèles » — et toute affirmation de rang se paie d'une décision : celle du **départage**. Le socle du module rend le problème visible : parmi les 23 497 clients, il n'existe que **21** nombres de tickets différents, et **3 316** clients ont exactement le même (**8** tickets). Selon la fonction employée, le même socle place le dernier client au rang **23 444** ou au rang **21** — deux nombres justes, un seul interprétable. Et le classement peut contredire le total : les 5 magasins du réseau sont rangés **exactement à l'envers** selon qu'on les classe par chiffre d'affaires ou par panier moyen.

Matériel de l'atelier — DuckDB 1.5.5 · Python 3.13 · socle M11 (03_exercices/dossier_M11/socle_m11.sql, rejoué en 0,4 s). Rappel du socle : **240 000** lignes, **237 191** ventes nettes (**2 809** retours), **44** mois, **5** magasins, **145 212** tickets, **23 497** clients, **15 595 154 955** FCFA de chiffre d'affaires net. Toutes les sorties de ce chapitre sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Choisir entre `ROW_NUMBER`, `RANK` et `DENSE_RANK`** sur un cas réel comportant des ex æquo, et **expliquer** ce que chacun affirme — numéroter, classer, ou compter les niveaux.
2. **Départager un classement** : reconnaître qu'une mesure comporte des égalités massives et écrire la clé qui rend le résultat reproductible.
3. **Découper en groupes de taille égale** avec `NTILE`, et savoir pourquoi un découpage en tiers n'est pas un classement.
4. **Extraire un top N par groupe** (les 3 premiers produits de chaque famille) et **filtrer après** le calcul du rang, jamais avant.
5. **Calculer un cumul et une variation** avec `SUM() OVER (ORDER BY ...)` et `LAG/LEAD`, en traitant explicitement le premier trou du décalage.
6. **Lire un classement comme une affirmation discutable** : vérifier qu'il répond bien à la question posée, et pas à une autre.

2. Pourquoi cette notion est importante

Trois des demandes les plus fréquentes d'un responsable se formulent en rang :

- « sortez-moi le **top 20** des produits » : un classement tronqué ;
- « le chiffre **cumulé** depuis janvier » : une somme en mouvement ;
- « la **progression** par rapport au mois dernier » : un décalage.

Chacune se trompe d'une façon différente, et aucune ne produit de message d'erreur. Un top tronqué au mauvais rang publie cinq lignes pour un cinquième rang : les 5 clients à **20** tickets du socle occupent les rangs 2, 3, 4, 5 et 6 en numérotation, mais **tous le rang 2** en classement — un rapport qui annonce « le 5e client » raconte une hiérarchie qui n'existe pas. Un cumul sans ordre explicite change de valeur selon l'heure de la journée. Une variation calculée sur le premier mois d'une série compare à un vide et publie `NULL`, que la plupart des tableurs affichent comme une cellule vide — donc comme un zéro.

Le chiffre qui structure ce chapitre. Le socle contient **23 497** clients et seulement **21** valeurs distinctes de « nombre de tickets ». La mesure est donc **massivement** faite d'égalités : **3 316** clients partagent 8 tickets. Sur cette mesure, `RANK` descend jusqu'au rang **23 444** tandis que `DENSE_RANK` s'arrête à **21**. Les deux requêtes sont exactes ; l'une des deux n'a aucun intérêt pour un décideur, et il faut savoir laquelle on publie avant que le comité de direction ne compare deux rapports incompatibles.

3. Explication simple — le podium et les dossards

À l'arrivée d'une course, trois documents différents peuvent être imprimés, et ils ne disent pas la même chose :

- **`ROW_NUMBER` = les dossards de classement.** Chaque coureur reçoit un numéro **unique**, de 1 à n, dans l'ordre d'arrivée. S'il y a égalité, on tranche arbitrairement (un tirage) ou par une règle explicite (« à égalité, celui dont l'identifiant est le plus petit »). Personne ne partage un dossard.
- **`RANK` = le podium.** Deux coureurs à égalité reçoivent la **même** place, et les places suivantes sont **sautées**. Trois arrivants ex æquo en 2e position : ils sont tous 2e, et le suivant est 5e. C'est la règle des médailles olympiques.
- **`DENSE_RANK` = le nombre de niveaux.** Deux coureurs à égalité partagent la 2e place, et le suivant prend la **3e** : on ne compte que les niveaux de performance réellement distincts.

Et **`NTILE(n)` = la photo de famille.** On coupe la troupe en n groupes de taille aussi égale que possible. Ce n'est pas un classement : deux coureurs du même groupe peuvent avoir des performances très différentes, et surtout, à la frontière, deux ex æquo peuvent tomber **de part et d'autre** de la ligne de coupe. C'est ce que le chapitre mesure sur le socle : un tiers comptant **2 718, 2 720, 2 728** ou **2 738** clients selon l'exécution — le même socle, le même code, quatre chiffres.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
numérotation	row numbering	Attribuer un numéro unique à chaque ligne d'une partition (<code>ROW_NUMBER</code>) : aucune égalité possible.
classement égalité	à rank	Attribuer la même place aux lignes de même valeur, en sautant les places suivantes (<code>RANK</code>).
rang dense	dense rank	Attribuer la même place aux ex æquo sans sauter de place (<code>DENSE_RANK</code>) : il compte les niveaux .
ex æquo	tie	Deux lignes de valeur identique sur la mesure classée : le cas qui décide du choix de la fonction.
clé de départage	tie-breaker	La colonne ajoutée à l' <code>ORDER BY</code> d'une fenêtre pour rendre un classement reproductible (presque toujours l'identifiant).
quantile	quantile / bucket	Découpage en groupes de taille comparable (<code>NTILE(n)</code>) : tiers, quarts, déciles.

Terme (français)	Équivalent anglais	Définition en une phrase
top N par groupe	top-N per group	Le motif <code>PARTITION BY groupe ORDER BY mesure DESC</code> puis filtre sur le rang — le classement le plus demandé en BI.
cumul	running total	Somme qui s'arrête à la ligne courante (<code>SUM(x) OVER (ORDER BY ...)</code>) ; dépend de l'ordre et du départage.
décalage	lag / lead	La valeur de la ligne précédente ou suivante dans l'ordre (<code>LAG/LEAD</code>) : c'est ce qui rend possible une variation.
trou de série	series gap	La ligne qui n'a pas de prédécesseur (premier mois) : elle rend <code>NULL</code> , et ce <code>NULL</code> doit être déclaré .

Définition. Les trois fonctions de rang répondent à trois questions différentes sur la même mesure : « **donne-moi un numéro unique** » (`ROW_NUMBER`), « **quelle place mérite-t-elle ?** » (`RANK`), « **combien de niveaux de performance y a-t-il jusqu'ici ?** » (`DENSE_RANK`). Choisir, c'est décider de ce que le rapport affirmera — pas de ce qui est le plus pratique.

Définition. Une **clé de départage** — tie-breaker — est une colonne ajoutée après la mesure dans l'`ORDER BY` d'une fenêtre (presque toujours la clé primaire de la table) pour que deux exécutions du même code produisent **le même** résultat. Sans elle, le moteur est libre : le classement n'est pas faux, il est **instable**.

5. Cours approfondi

5.1 Trois fonctions de rang, une seule mesure

```
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT id_client, t,
       ROW_NUMBER() OVER (ORDER BY t DESC, id_client) AS numero,
       RANK() OVER (ORDER BY t DESC) AS place,
       DENSE_RANK() OVER (ORDER BY t DESC) AS niveau
FROM c ORDER BY t DESC, id_client;
```

Sur le socle, les premières lignes montrent le client **0** (identifiant non référencé) seul au sommet, puis un groupe de clients à **20** tickets :

id_client	tickets	numero	place	niveau
0	38 984	1	1	1
3545	20	2	2	2
14859	20	3	2	2
21036	20	4	2	2
21392	20	5	2	2

Cinq clients ont exactement **20** tickets. `NUMERO` les numérote 2, 3, 4, 5, 6 (l'ordre entre eux vient du départage par identifiant) ; `PLACE` leur donne à tous **2** ; `NIVEAU` aussi **2**. Les trois colonnes sont justes — elles répondent à trois questions que le demandeur n'a pas distinguées. « Le 5e meilleur client » est une phrase vide : il n'existe pas de 5e meilleur client, il existe cinq clients deuxièmes.

5.2 Le départage : ce qui rend un classement reproductible

Sur la même mesure, l'écart entre `RANK` et `DENSE_RANK` s'amplifie à mesure qu'on descend : le `RANK` maximum atteint **23 444** (il saute les places perdues par les 3 316 clients à 8 tickets, et

par tous les autres ex æquo), tandis que le `DENSE_RANK` maximum vaut **21** — soit exactement le nombre de niveaux de fréquence observés.

Dans les faits. L'atelier a exécuté **15** fois la même requête de segmentation RFM (C04) **sans** clé de départage : le segment principal a compté **2 718, 2 720, 2 728** puis **2 738** clients. Le socle n'a pas changé d'une ligne. Ce n'est pas une erreur de mesure : c'est `NTILE` qui, à la frontière d'un tiers, choisit librement parmi des ex æquo. **3 316** clients partageant 8 tickets suffisent à déplacer cette frontière — et un segment qui bouge de 20 clients est un segment qu'on ne peut pas comparer à celui du trimestre dernier.

La règle est donc simple et non négociable : **toute fenêtre d'ordonnement reçoit un départage**. `ORDER BY t DESC, id_client` au lieu de `ORDER BY t DESC`. C'est une colonne de plus, et c'est la différence entre un indicateur et un tirage au sort.

Attention. Le départage ne change pas la **valeur** de la mesure, il change la **frontière**. Un top 3 par groupe avec ou sans départage peut désigner deux produits différents quand trois produits sont ex æquo — et le rapport, dans les deux cas, présentera trois lignes avec l'aplomb d'un résultat définitif.

5.3 `NTILE` : découper en tranches, pas classer

`NTILE(3) OVER (ORDER BY m DESC)` range les lignes en trois groupes de taille aussi égale que possible : c'est le découpage utilisé partout pour « les 10 % les plus gros clients ». Deux précautions, mesurées au §5.2 : le **départage** (sinon la frontière bouge) et la **définition** du découpage. Sur le socle, les deux écritures les plus courantes du « top 10 % » ne gardent pas le même nombre de lignes : `NTILE(100) <= 10` retient 2 350 clients, `ROW_NUMBER() <= 0,10 × 23 497` en retient 2 349. La part de chiffre d'affaires publiée est alors de **38,1 %** ou de **38,0 %** — un dixième de point, mais deux rapports qui ne se réconcilient plus.

Définition. Un **quantile** — quantile — est une frontière qui coupe une population ordonnée en parties de taille comparable : les tiers (`NTILE(3)`), les déciles (`NTILE(10)`), les centiles (`NTILE(100)`). Un quantile n'est pas un seuil : il ne demande pas « combien valent-ils ? » mais « combien sont-ils ? ».

5.4 Le top N par groupe : le motif qui exige un filtre après

Le classement le plus demandé en BI est le top N **par** groupe : les 3 meilleurs produits par famille, les 5 meilleurs clients par magasin. Le motif s'écrit en deux temps, et l'ordre des temps compte :

```
SELECT * FROM (
  SELECT p.categorie, p.designation, SUM(v.montant_ttc) AS ca,
         ROW_NUMBER() OVER (PARTITION BY p.categorie ORDER BY SUM(v.montant_ttc) DESC) AS rg
  FROM ventes v JOIN produit p USING (id_produit)
  WHERE NOT v.est_retour
  GROUP BY 1, 2
) WHERE rg <= 3 ORDER BY categorie, ca DESC;
```

Le filtre `rg <= 3` est à **l'extérieur** de la requête qui calcule le rang. Écrit dans la même requête, il s'appliquerait avant la fenêtre : `RANK` vaudrait alors **1** pour **toutes** les lignes restantes, et le rapport publierait un « top 3 » qui contient tout ce qu'on a laissé passer. C'est exactement le piège du `WHERE` avant la fenêtre (C01, §5.7), appliqué au cas le plus fréquent de la BI.

Exécuté sur le socle, ce top 3 rend **46** lignes — alors qu'un top 3 sur les **7** familles réelles du catalogue en rend **21**. L'écart n'est pas un problème de requête : il vient des **16** libellés de catégorie du référentiel produits (**9** écritures après casse et espaces, **7** familles après accents et pluriel), un défaut de qualité que C05 puis C07 traitent à fond. Retenez pour l'instant que le top N par groupe est l'endroit où une donnée mal normée devient **visible** : 46 lignes de réponse pour 7 familles, sans le moindre message d'alerte.

5.5 SUM() OVER : le cumul, et sa clôture

Un cumul s'écrit avec le même `SUM` que C01, plus un `ORDER BY` **dans** la fenêtre :

```
SELECT an, mm, ca, SUM(ca) OVER (PARTITION BY an ORDER BY mm) AS cumul
FROM (SELECT YEAR(date_vente) an, date_trunc('month', date_vente) mm, SUM(montant_ttc) ca
      FROM ventes WHERE NOT est_retour GROUP BY 1, 2);
```

Sur 2023, le cumul passe par **284 400 304 ; 547 583 580 ; 881 958 439 ; 1 196 335 034** FCFA, et la part cumulée par **8,3 ; 15,9 ; 25,6 ; 34,7** %. Deux contrôles gratuits :

1. **la clôture** : au dernier mois de l'année, le cumul doit égaler le total de l'année — **3 443 581 547** FCFA pour 2023 ;
2. **la monotonie** : un cumul de chiffre d'affaires ne peut pas diminuer. S'il diminue, une ligne a été comptée deux fois ou une annulation n'a pas été traitée.

5.6 LAG et LEAD : la variation, et son premier trou

`LAG(ca) OVER (ORDER BY mm)` rend la valeur du mois **précédent**, `LEAD` celle du mois **suivant**. La variation s'écrit alors directement, sans auto-jointure — la raison pour laquelle `LAG` a remplacé une bonne moitié des jointures dites « auto-jointures » des rapports d'avant.

Un décalage de **12** lignes donne la comparaison **annuelle** à périmètre égal : c'est la façon la plus honnête de comparer deux années, parce qu'elle compare **le même mois**. Sur les **32** mois du socle qui ont un équivalent un an plus tôt, la progression annuelle moyenne vaut **+16,8** % — et les premiers mois de 2024 valent **+11,0** %, **+23,5** %, **+20,6** %, **+27,7** %, **+11,5** %.

Définition. Un **trou de série** — series gap — est une ligne dont le décalage n'a pas de prédécesseur : le premier mois d'une série, ou le premier mois après un filtre trop zélé. `LAG` rend alors `NULL`, et l'arithmétique propage ce `NULL` (`ca - NULL` vaut `NULL`). Ce n'est pas un bug : c'est la réponse exacte à une question impossible — « par rapport à quoi ? ».

Attention. Le `WHERE` s'exécute avant la fenêtre. Une requête écrite `SELECT am, ca, LAG(ca, 12) OVER (ORDER BY am) ... WHERE am >= '2024-01'` filtre **puis** décale : les douze premiers mois manquent à l'appel et la première variation annuelle publiée est `NULL`. L'atelier a reproduit ce résultat avant de le corriger en calculant la fenêtre dans une sous-requête, puis en filtrant au-dessus.

5.7 Le classement qui contredit le total

Le socle fournit la démonstration la plus nette de ce qui distingue un **total** d'un **classement**. Les cinq magasins, rangés par chiffre d'affaires puis par panier moyen :

Magasin	CA net (FCFA)	Rang CA	Panier moyen (FCFA)	Rang panier
Ouaga 2000	5 312 205 459	1	107 374	3
Gounghin	3 424 978 244	2	106 621	4
Bobo Kibidwé	3 099 946 956	3	105 724	5
Koudougou Centre	2 024 794 213	4	108 035	2

Magasin	CA net (FCFA)	Rang CA	Panier moyen (FCFA)	Rang panier
Kaya Marché	1 733 230 083	5	111 447	1

Le classement par panier est **l'inverse** du classement par chiffre d'affaires : le dernier magasin du réseau a le plus gros panier moyen (**111 447** FCFA contre **107 374** pour le premier). Aucun des deux tableaux n'est faux, et aucune des deux conclusions ne s'ensuit : un petit magasin sert moins de clients, mais des clients qui achètent plus par visite. Publier « Kaya vend mal » ou « Kaya vend le mieux » est, dans les deux cas, une erreur de lecture — la bonne phrase porte sur ce qui est mesuré : le panier moyen, pas l'activité.

Conseil professionnel. Avant de publier un classement, écrivez en une phrase la question à laquelle il répond, et vérifiez qu'elle contient bien la mesure classée : « les magasins **par chiffre d'affaires** », « les magasins **par panier moyen** ». Un classement publié sans sa mesure est un classement qui sera lu comme un jugement de valeur.

5.8 Ce que ce chapitre ne couvre pas

Le temps **calendaire** — le mois sans vente qu'il faut fabriquer, la table de dates, l'absence de comparaison quand la période est incomplète — est le sujet de C03 : LAG compare des **lignes**, pas des **dates**, et deux mois qui se suivent dans un résultat ne sont pas forcément deux mois voisins au calendrier. La segmentation par **cohortes** et le **RFM** complet sont ceux de C04.

6. Exemple concret — trois classements, un seul socle

Le directeur commercial demande « les meilleurs clients ». La question, telle quelle, ne détermine pas la mesure : en tickets, en montant, en récence ? Le tableau de bord du module retient la **fréquence** (nombre de tickets) et publie **trois** colonnes de rang — c'est le seul moyen de montrer la différence en production :

Identifiant	Tickets	NUMERO	PLACE	NIVEAU
0	38 984	1	1	1
3545	20	2	2	2
14859	20	3	2	2
21036	20	4	2	2
21392	20	5	2	2
5	8	10 435	10 434	14

La dernière ligne est la leçon du chapitre. Un client à **8** tickets — la valeur la plus fréquente du socle, partagée par **3 316** clients — porte le numéro **10 435** (unique, donc faux comme information puisque 3 315 autres clients sont derrière lui à la même valeur), la place **10 434** (juste, illisible), et le niveau **14** (lisible : il existe 14 niveaux de fréquence au-dessus du sien sur **21** possibles).

Et le client **0** ? Il est premier partout, avec **2 852 612 447** FCFA et **18,3 %** du chiffre d'affaires du réseau — et il **n'existe pas** dans le référentiel clients : c'est l'identifiant des ventes non identifiées. Le classement l'ignore superbement, parce qu'un classement ne vérifie rien : il ordonne ce qu'on lui donne. Ce cas traverse tout le module et devient l'étude de cas de son évaluation.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — mesurer l'ampleur des égalités avant de classer

```
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT COUNT(*) AS clients, COUNT(DISTINCT t) AS valeurs_distinctes, MAX(t) AS maximum FROM c;
```

```
23497 | 21 | 38984
```

23 497 clients pour **21** valeurs distinctes : le classement par fréquence est un classement d'égalités. Toute la suite du chapitre en découle.

7.2 Étape 2 — les trois rangs côte à côte

C'est la requête du §5.1. Sa sortie montre le client 0 seul au sommet, puis les cinq clients à 20 tickets qui partagent la place 2 (§6).

7.3 Étape 3 — l'écart maximal entre les deux classements

```
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1),
      r AS (SELECT RANK() OVER (ORDER BY t DESC) rk, DENSE_RANK() OVER (ORDER BY t DESC) dr FROM c)
SELECT MAX(rk), MAX(dr) FROM r;
```

```
23444 | 21
```

Le dernier client du socle est **23 444e** ou **21e** selon la fonction. Dit autrement : « le nombre de niveaux de fréquence » est une information, « le 23 444e client » n'en est pas une.

7.4 Étape 4 — le cumul, et son contrôle de clôture

```
WITH m AS (SELECT YEAR(date_vente) an, date_trunc('month', date_vente) mm, SUM(montant_ttc) ca
            FROM ventes WHERE NOT est_retour GROUP BY 1, 2),
      c AS (SELECT mm, SUM(ca) OVER (PARTITION BY an ORDER BY mm) cumul,
            SUM(ca) OVER (PARTITION BY an) total FROM m WHERE an = 2023)
SELECT mm, cumul, total, cumul = total AS cloture FROM c ORDER BY mm;
```

```
2023-01-01 | 284400304 | 3443581547 | false
2023-02-01 | 547583580 | 3443581547 | false
2023-03-01 | 881958439 | 3443581547 | false
...
2023-12-01 | 3443581547 | 3443581547 | true
```

Une seule ligne de contrôle (`cumul = total`) suffit à valider les douze mois.

7.5 Étape 5 — la variation mensuelle, puis annuelle

```
WITH m AS (SELECT date_trunc('month', date_vente) mm, SUM(montant_ttc) ca FROM ventes
            WHERE NOT est_retour GROUP BY 1)
SELECT mm, ca, LAG(ca) OVER (ORDER BY mm) AS mois_precedent,
       ROUND(100.0 * (ca - LAG(ca) OVER (ORDER BY mm)) / LAG(ca) OVER (ORDER BY mm), 1) AS var_pct
FROM m ORDER BY mm LIMIT 5;
```

2023-01-01	284400304	NULL	NULL
2023-02-01	263183276	284400304	-7.5
2023-03-01	334374859	263183276	27.1
2023-04-01	314376595	334374859	-6.0
2023-05-01	275723327	314376595	-12.3

Le premier mois rend `NULL` : il n'a pas de mois précédent. La variation mensuelle d'un socle de quincaillerie est **bruyante** (de -12,3 % à +27,1 % en cinq mois) : c'est exactement pourquoi le chapitre C03 lui préférera des moyennes mobiles.

7.6 Étape 6 — la comparaison annuelle, et le piège du filtre

Avec `LAG(ca, 12)`, la comparaison porte sur le **même mois** de l'année précédente : **+11,0 %**, **+23,5 %**, **+20,6 %**, **+27,7 %**, **+11,5 %** pour les cinq premiers mois de 2024, et **+16,8 %** en moyenne sur les **32** mois comparables. Écrite avec le filtre dans la même requête que la fenêtre, la même requête publie `NULL` — le filtre a supprimé les douze mois dont `LAG` avait besoin.

8. Erreurs fréquentes

1. **Numéroter quand on voulait classer.** `ROW_NUMBER` attribue un numéro unique : sur une mesure à égalités, il **invente** une hiérarchie (5 clients à 20 tickets deviennent 2e, 3e, 4e, 5e et 6e). Le rapport qui en découle annonce « le 5e meilleur client » alors qu'il n'en existe que deux niveaux.
2. **Classer sans départage.** Sans colonne de départage, deux exécutions peuvent rendre deux frontières différentes : l'atelier a mesuré **2 718**, **2 720**, **2 728** et **2 738** clients pour un même segment RFM. Un indicateur qui bouge sans que la donnée bouge n'est pas un indicateur.
3. **Filtrer le top N dans la même requête que le rang.** `WHERE rg <= 3` s'applique **avant** la fenêtre : le rang vaut alors 1 pour toutes les lignes survivantes et le « top 3 » contient tout ce qui a passé le filtre. Le filtre sur un rang se met **toujours** dans une requête englobante.
4. **Publier une variation sur le premier mois.** `LAG` rend `NULL` au premier mois, et la division propage le `NULL`. Afficher ce `NULL` comme « 0 % » est la faute la plus coûteuse des séries temporelles : elle transforme un trou en information.
5. **Confondre le rang et le total.** Le premier magasin du réseau par chiffre d'affaires n'est que **3e** par panier moyen, et le dernier est **1er**. Les deux classements sont justes ; un rapport qui n'annonce pas sa mesure sera lu comme un jugement sur les équipes.

9. Bonnes pratiques professionnelles

1. **Mesurer les égalités avant de classer.** `COUNT(DISTINCT mesure)` à côté de `COUNT(*)` : si le rapport est faible (ici **21** valeurs pour **23 497** lignes), le choix de la fonction de rang n'est plus un détail, c'est le sujet.
2. **Toujours départager, et écrire pourquoi.** `ORDER BY mesure DESC, id_client` : la clé de départage est l'identifiant le plus petit, et cela se documente en commentaire dans la requête — un lecteur qui ne comprend pas la règle d'égalité ne peut pas reproduire le rapport.
3. **Choisir la fonction selon la phrase, pas l'inverse.** « Il est 2e » → `RANK`. « Il existe deux niveaux au-dessus » → `DENSE_RANK`. « Il est le 3e de la liste » → `ROW_NUMBER` **avec** départage assumé.
4. **Contrôler un cumul par sa clôture et sa monotonie.** Le dernier cumul égale le total de la période ; un cumul qui diminue signale un doublon ou une annulation non traitée.
5. **Déclarer les trous.** Quand une série commence, le premier décalage est `NULL` : on l'écrit dans la note de méthode du rapport (« les variations de janvier 2023 ne sont pas calculables : pas d'antériorité dans la base ») plutôt que de laisser la cellule vide.

10. Exercice guidé

Situation. Le comité de direction a reçu, la semaine dernière, deux rapports sur la fidélité des clients, produits par deux analystes de l'équipe. Le premier conclut : « **5** clients dépassent 20 tickets ; le 5e est à 20 tickets ». Le second conclut : « le seuil des plus fidèles est de **20** tickets ; **cinq** clients l'atteignent ». Les deux rapports sont **incompatibles** et la direction demande pour quoi.

Consigne. Dans les 20 minutes (chrono) :

1. Écrivez la requête qui reproduit le classement du premier rapport, et celle du second. (3 pts)
2. Expliquez, en nommant les fonctions, pourquoi elles ne disent pas la même chose sur les **mêmes** données. (3 pts)
3. Montrez, chiffres en main, où les deux rapports divergent **le plus** : quel est l'écart maximal entre RANK et DENSE_RANK sur le socle ? (2 pts)
4. Écrivez la phrase que vous enverriez aux deux analystes pour qu'ils produisent à l'avenir des rapports comparables. (2 pts)

11. Exercices autonomes

E1 — Le top N par famille, proprement (35 min). Produisez les **3** produits les plus vendus de chaque **famille** réelle du catalogue — au sens de **C05**, qui définit la famille comme le libellé replié (casse, espaces, accents, pluriel) — et non de chaque libellé présent dans le fichier. Pour chaque produit son rang, son chiffre d'affaires et la part qu'il représente **dans sa famille**. Contrainte : le filtre sur le rang doit être **à l'extérieur** de la requête qui le calcule, et vous joindrez la preuve du nombre de lignes attendu ($3 \times$ nombre de familles).

E2 — Le cumul et la variation d'un magasin (30 min). Pour le magasin de votre choix, publiez mois par mois : le chiffre d'affaires, le cumul depuis janvier 2023, la variation mensuelle et la variation annuelle à périmètre égal. Contrôles exigés : le cumul du dernier mois égale le total du magasin, et les mois sans antériorité sont **déclarés NULL** (pas zéro).

12. Correction détaillée

Exercice guidé.

1. **Les deux requêtes.** Le premier rapport classe avec ROW_NUMBER (ou avec LIMIT 5 sur un tri) :

```
sql WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour
GROUP BY 1) SELECT id_client, t, ROW_NUMBER() OVER (ORDER BY t DESC, id_client) AS numero FROM
c ORDER BY t DESC, id_client LIMIT 5;
```

Le second annonce une **égalité**, donc un RANK (ou un DENSE_RANK) :

```
sql WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour
GROUP BY 1) SELECT id_client, t, RANK() OVER (ORDER BY t DESC) AS place FROM c WHERE place <= 2
ORDER BY t DESC, id_client;
```

2. **Pourquoi les deux conclusions diffèrent.** Le premier affirme une **hiérarchie** (« le 5e est à 20 tickets ») : ROW_NUMBER numérote des lignes, y compris quand elles ont la même valeur. Le second affirme un **niveau** (« cinq clients atteignent 20 tickets ») : RANK attribue la même place aux ex æquo. Sur des données sans égalité, les deux coïncident — c'est pourquoi l'erreur passe longtemps inaperçue. 3. **L'écart maximal.** Sur le socle, MAX(RANK) vaut **23 444** et MAX(DENSE_RANK) vaut **21**. L'écart vient des **3 316** clients à 8 tickets, qui font sauter 3 315 places à eux seuls. 4. **La phrase à envoyer.** « Merci de préciser dans chaque rapport la fonction de rang employée et la clé de départage : RANK pour parler de places (avec égalités partagées), DENSE_RANK pour parler de niveaux, ROW_NUMBER seulement quand un rang unique est explicitement demandé — et dans ce cas, écrire la règle de départage. »

E1 (le top 3 par famille). Il faut d'abord **normaliser** le libellé : `UPPER(TRIM(categorie))` réunit casse et espaces (**9** écritures), et le repliement des accents et du pluriel ramène à **7** familles (voir C05). Puis classer par famille :

```
WITH p AS (SELECT id_produit, UPPER(TRIM(categorie)) AS famille FROM produit),
v AS (SELECT p.famille, p.id_produit, SUM(x.montant_ttc) AS ca
      FROM ventes x JOIN p USING (id_produit) WHERE NOT x.est_retour GROUP BY 1, 2)
SELECT * FROM (
  SELECT famille, id_produit, ca,
         ROW_NUMBER() OVER (PARTITION BY famille ORDER BY ca DESC, id_produit) AS rg,
         ROUND(100.0 * ca / SUM(ca) OVER (PARTITION BY famille), 1) AS part_famille_pct
  FROM v
) WHERE rg <= 3 ORDER BY famille, rg;
```

Contrôle attendu : **21** lignes (3×7 familles). Sans la normalisation, la même requête sur les libellés bruts rend **46** lignes — l'écart est la trace exacte du défaut de qualité du référentiel.

E2 (le cumul d'un magasin). La structure attendue est celle du §7.4 avec `PARTITION BY id_magasin`, plus un `LAG(ca, 12) OVER (PARTITION BY id_magasin ORDER BY am)` pour la variation annuelle. Les deux contrôles : le cumul du dernier mois égale le total du magasin (**5 312 205 459** FCFA pour Ouaga 2000, **1 733 230 083** FCFA pour Kaya Marché), et les douze premiers mois affichent une variation annuelle `NULL` — déclarée, jamais remplacée par zéro.

13. Mini-projet de chapitre

La note « trois classements, une mesure ». Vous produisez 3 pages maximum pour le comité commercial : trois classements du même socle, avec la mesure annoncée pour chacun et la preuve du départage.

Livrables et barème.

Critère	Ce qui est vérifié	Points
Les trois classements	<code>ROW_NUMBER</code> , <code>RANK</code> , <code>DENSE_RANK</code> sur la même mesure, sorties publiées	6
La preuve du départage	la clé de départage écrite et justifiée ; une deuxième exécution rend le même résultat	5
Les chiffres d'égalité	23 497 clients, 21 valeurs distinctes, 3 316 clients à 8 tickets, écart 23 444 contre 21	4
La phrase de conclusion	une phrase par classement, avec la mesure annoncée et ce qu'elle interdit de dire	3
Total		18

14. Résumé du chapitre

Classer, c'est décider **ce qu'on affirme** : `ROW_NUMBER` numérote des lignes (hiérarchie inventée si égalités), `RANK` attribue des places (les suivantes sont sautées), `DENSE_RANK` compte les niveaux. Sur le socle, la mesure de fréquence ne compte que **21** niveaux pour **23 497** clients, dont **3 316** partagent 8 tickets : le choix de la fonction n'est jamais neutre. `NTILE` découpe en tranches de taille égale — utile pour « les 10 % les plus gros », dangereux sans **clé de départage** (l'atelier a mesuré des frontières à **2 718**, **2 720**, **2 728** et **2 738** clients pour la même requête). Le **top N par groupe** place le filtre **après** le rang, jamais avant. Le **cumul** se contrôle par sa clôture (le cumul de décembre égale **3 443 581 547** FCFA pour 2023) et par sa monotonie. Le **décalage** (`LAG`, `LEAD`) calcule variations mensuelles et annuelles — **+16,8 %** en moyenne sur les **32** mois comparables du socle — à condition de **déclarer** le premier trou. Et un classement ne vaut que par sa mesure : les cinq magasins sont rangés exactement à l'envers selon qu'on classe par chiffre d'affaires ou par panier moyen.

15. À retenir

1. **ROW_NUMBER numérote, RANK classe, DENSE_RANK compte les niveaux.** Trois questions différentes, trois colonnes justes, un seul rapport recevable — celui qui annonce la sienne.
2. **Aucune fenêtre d'ordonnancement sans clé de départage.** Sans elle, le classement change sans que la donnée change.
3. **Le filtre sur un rang se met dans une requête englobante.** Dans la même requête, il s'applique avant la fenêtre et le rang vaut 1 partout.
4. **Un cumul se contrôle deux fois** : sa clôture (dernier cumul = total de la période) et sa monotonie.
5. **LAG du premier mois rend NULL, et ce NULL se déclare.** Le remplacer par zéro est une invention.

À retenir. Un classement est une **phrase**, pas une colonne : « les magasins par panier moyen » n'affirme pas la même chose que « les magasins par chiffre d'affaires », même quand les deux tableaux contiennent les mêmes cinq noms.

À retenir. L'égalité n'est pas un cas limite : c'est le cas normal d'une mesure de comptage. Un classement écrit sans penser aux ex æquo est un classement écrit pour un autre jeu de données.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Cinq clients ont 20 tickets. Que valent leur ROW_NUMBER, leur RANK et leur DENSE_RANK s'ils sont précédés d'un seul client à 38 984 tickets ?
2. Sur le socle, MAX(RANK) et MAX(DENSE_RANK) valent respectivement... ? Pourquoi un tel écart ?
3. Pourquoi NTILE a-t-il besoin d'une clé de départage, alors qu'il ne publie aucun rang ?
4. Où placer le filtre `rg <= 3` d'un top 3 par groupe, et que se passe-t-il sinon ?
5. Que vaut **2 718** dans ce chapitre ? Et **2 738** ?
6. Pourquoi le cumul de décembre doit-il évaluer le total de l'année ?
7. Que rend `LAG(ca, 12)` pour le premier mois d'une série, et que publie-t-on à la place ?
8. Le premier magasin en chiffre d'affaires est 3e en panier moyen : quelle phrase peut-on écrire, et laquelle est interdite ?
9. Combien de lignes rend un top 3 par catégorie sur les libellés du fichier, et combien sur les familles réelles ?
10. Pourquoi la variation mensuelle du socle est-elle un mauvais indicateur de tendance, avant même le chapitre C03 ?

Réponses. 1. ROW_NUMBER : 2, 3, 4, 5, 6 ; RANK : 2 pour les cinq ; DENSE_RANK : 2 pour les cinq. 2. **23 444** et **21** : RANK saute les places des ex æquo (les **3 316** clients à 8 tickets en font sauter 3 315 à eux seuls), DENSE_RANK compte les niveaux. 3. Parce que la frontière des tranches tombe au milieu d'ex æquo : sans départage, le moteur choisit qui passe (mesuré : **2 718, 2 720, 2 728, 2 738**). 4. Dans une requête **englobante** ; dans la même requête, le WHERE s'exécute avant la fenêtre et le rang vaut 1 pour toutes les lignes restantes. 5. Deux tailles observées du même segment RFM selon l'exécution : la preuve que la frontière n'est pas reproductible. 6. Parce que le cumul est la somme des mois de l'année : s'il diffère, une ligne a été comptée deux fois ou manque. 7. NULL ; on **déclare** le trou dans la note de méthode. 8. « Ouaga 2000 réalise le plus gros chiffre d'affaires du réseau et un panier moyen de **107 374** FCFA, inférieur à celui de Kaya Marché (**111 447** FCFA) » ; interdit : « Ouaga 2000 vend mieux que les autres ». 9. **46** lignes

(16 libellés) contre **21** (7 familles). 10. Parce que la variation mensuelle va de $-12,3\%$ à $+27,1\%$ en cinq mois sur un socle stable : elle mesure le calendrier et le hasard plus que la tendance.

Module M11.C03 — Le temps en SQL : table calendrier, année sur année, mois glissants, et le mois qui manque

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5). **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 et C02 (fenêtres, cadres, LAG) ; M07.C02 (dates, WHERE), M07.C06 (CASE) ; M09.C03 (tendance, saisonnalité, bruit).

L'idée du chapitre. Comparer dans le temps a l'air simple — « cette année contre l'année dernière » — et c'est pourtant la famille de requêtes qui produit le plus de chiffres faux de toute la BI, parce que **le temps est incomplet** et que rien ne le signale : l'année en cours n'est pas finie, certains mois n'ont pas d'objectif, une moyenne mobile de douze mois calculée sur quatre mois reste une moyenne. Le socle du module donne à voir les trois cas d'un coup : **44** mois de ventes sans le moindre trou, mais **2** mois sans objectif (**magasin 4**, février et mars 2024) — et une comparaison naïve de 2026 à 2025 qui publie **-28,9 %** là où la comparaison à périmètre égal (huit mois contre huit mois) publie **+15,4 %**. Ce chapitre apprend à écrire le temps **explicitement** : un calendrier, un périmètre, un compteur de fenêtre.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql, rejoué en 0,4 s). Le socle contient une table `calendrier` (**1 339** jours, du 01/01/2023 au 31/08/2026, avec jours fériés et libellés), les **44** mois de ventes (**237 191** lignes nettes, **15 595 154 955** FCFA) et **218** lignes d'objectifs mensuels par magasin. Toutes les sorties publiées ici sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Fabriquer un axe de temps complet** — avec `generate_series` ou une table `calendrier` — et **prouver** qu'il couvre la période du rapport.
2. **Détecter un trou de série** : distinguer un mois sans vente (qui n'existe pas toujours) d'un mois sans référence en face (objectif, budget, effectif), et le rendre visible par une jointure **externe**.
3. **Comparer deux périodes à périmètre égal** : dire explicitement quels mois sont comparés, et pourquoi la comparaison « année complète contre année en cours » est interdite.
4. **Calculer une moyenne mobile** de 3 et de 12 mois, avec le **cadre** qui convient, et **compter les lignes** de la fenêtre pour savoir si elle est complète.
5. **Lire la saison** du socle : jour de la semaine (rapport **3,2** entre le meilleur et le pire jour) et mois calendaire (rapport **1,65** entre mars et juillet), et s'en servir pour interpréter — pas pour excuser.

2. Pourquoi cette notion est importante

Une entreprise ne produit pas deux fois la même semaine. Le socle le montre sans ambiguïté : le lundi rapporte **13 872 370** FCFA par jour de vente, le dimanche **4 322 830** — un rapport de **3,2**. Le mois de mars vaut en moyenne **433 645 399** FCFA par mois, celui de juillet **262 079**

192 — un rapport de **1,65**. Autrement dit : **comparer deux mois sans regarder le mois**, c’est comparer deux calendriers différents et appeler cela une tendance.

Par-dessus cette saison, la vraie difficulté est l’**incomplétude**. Trois situations, toutes présentes sur le socle :

- **La période en cours n’est pas finie.** Les ventes s’arrêtent au 31/08/2026 : 2026 compte **8** mois, 2025 en compte **12**. Comparer les deux totaux donne **−28,9 %** — une chute spectaculaire qui n’existe pas. À périmètre égal (janvier à août), la progression est de **+15,4 %**, cohérente avec les **+15,3 %** de 2025 et les **+18,8 %** de 2024 sur la même fenêtre.
- **La référence en face peut manquer.** Les objectifs couvrent **218** couples magasin-mois sur **220** : le magasin 4 n’a **pas d’objectif** pour février et mars 2024, alors qu’il y a réalisé **93 749 490** FCFA de ventes. Un rapport qui ne joint pas correctement publie soit une ligne à zéro (faux), soit rien du tout (invisible).
- **Une fenêtre peut être incomplète.** Une moyenne mobile de douze mois calculée sur un socle qui commence n’est complète qu’à partir du douzième mois. Sans compteur, janvier 2023 affiche « moyenne mobile 12 mois » d’un seul mois — et personne ne le voit.

3. Explication simple — l’agenda et les trous

Prenez un agenda papier de trois ans et demi. Vous y écrivez chaque soir le chiffre d’affaires du jour.

1. **Les pages existent même les jours sans écriture.** C’est tout l’intérêt d’un **calendrier** : il fournit les lignes que les ventes n’ont pas créées. Sans lui, un dimanche sans vente disparaît de la moyenne des dimanches — et la moyenne monte.
2. **Une page vide n’est pas un zéro.** Un jour sans vente vaut **0** FCFA ; un mois sans objectif vaut **inconnu**. Les deux se ressemblent dans un tableur et ne disent pas la même chose : le premier est une mesure, le second est une absence de mesure. La jointure externe (`LEFT JOIN`, `FULL OUTER JOIN`) est l’outil qui rend cette différence **visible**.
3. **Un agenda qui commence en janvier ne peut pas parler de décembre.** Toute comparaison « même mois, année précédente » a besoin de douze mois d’historique avant de pouvoir exister : sur le socle, seuls **32** des **44** mois ont un équivalent un an plus tôt. Les douze autres doivent afficher `NULL` et le dire.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
axe de temps	time axis	La liste complète des périodes du rapport, y compris celles sans donnée.
table calendrier	date dimension / calendar table	La table qui contient une ligne par jour (ou par mois) avec ses attributs : année, mois, trimestre, jour férié.
série fabriquée	generated series	Un axe de temps produit par le moteur (<code>generate_series</code>) plutôt que lu dans une table.
jointure externe	outer join	Une jointure qui conserve les lignes sans correspondance (<code>LEFT</code> , <code>FULL OUTER</code>) : le seul moyen de voir un trou.
trou de série	series gap	Une période de l’axe sans donnée : soit une absence réelle (pas de vente), soit une absence de mesure (pas d’objectif).
périmètre	scope / perimeter	L’ensemble des lignes comparées ; deux périodes ne se comparent qu’à périmètre égal.
période partielle	partial period	Une période en cours (mois courant, année courante) : sa comparaison à une période complète est fautive par construction.

Terme (français)	Équivalent anglais	Définition en une phrase
année sur année	year over year	Comparaison d'une période à la même période de l'année précédente (LAG(ca, 12) sur une série mensuelle).
moyenne mobile	moving average	Moyenne calculée sur une fenêtre qui avance (3 mois, 12 mois) : elle lisse le bruit sans supprimer la tendance.
saison	seasonality	La régularité calendaire d'une activité (jour de la semaine, mois, fêtes) : à connaître avant d'interpréter un écart.

Définition. Une **table calendrier** — date dimension — est la table de référence du temps : une ligne par jour (ici **1 339**), avec ses attributs calculés une fois pour toutes (année, mois, trimestre, libellé du jour, indicateur de férié). C'est **elle** qui rend les trous visibles : une jointure externe entre le calendrier et les ventes fait apparaître les jours sans vente, que les ventes seules ne peuvent pas montrer puisqu'elles ne contiennent que ce qui a eu lieu.

Définition. Le **périmètre** — scope — d'une comparaison est l'ensemble des lignes effectivement comparables : mêmes mois, même nombre de jours, mêmes magasins, mêmes exclusions. « 2026 contre 2025 » n'est pas un périmètre : c'est une phrase incomplète. « janvier à août 2026 contre janvier à août 2025 » en est un.

5. Cours approfondi

5.1 Fabriquer l'axe du temps

Deux façons d'obtenir un axe complet, et elles doivent **concorde** :

```
-- 1. la série fabriquée par le moteur
SELECT COUNT(*) FROM generate_series(DATE '2023-01-01', DATE '2026-08-31', INTERVAL 1 DAY);
-- 2. la table de référence du socle
SELECT COUNT(*) FROM calendrier;
```

Les deux rendent **1 339**. Ce contrôle n'est pas une coquetterie : il vérifie que la table de référence et la période réelle du socle parlent du même monde. Un écart signifie qu'un des deux est périmé — et c'est exactement le genre d'écart qui fait publier un rapport sur une période que la base ne couvre plus.

Pour un rapport mensuel, on agrège les ventes **par mois** puis on joint le calendrier des mois — ou l'on utilise directement `date_trunc('month', date_vente)` :

```
SELECT strftime(date_trunc('month', date_vente), '%Y-%m') AS mois, SUM(montant_ttc) AS ca
FROM ventes WHERE NOT est_retour GROUP BY 1 ORDER BY 1;
```

44 lignes. Le socle n'a **aucun** mois sans vente : c'est une chance, et cela doit être vérifié, pas supposé.

5.2 Le trou qu'on ne voit pas : l'objectif manquant

Les ventes couvrent les 44 mois ; les **objectifs** couvrent **218** couples magasin-mois sur les **220** possibles (5 magasins × 44 mois). Deux lignes manquent : le **magasin 4** en **février** et **mars 2024**.

Une jointure **interne** ne les voit pas — elle ne garde que ce qui existe des deux côtés, donc elle **supprime** silencieusement les deux mois. Une jointure **externe** les affiche :

```
WITH v AS (SELECT id_magasin, strftime(date_trunc('month', date_vente), '%Y-%m') am,
SUM(montant_ttc) ca
FROM ventes WHERE NOT est_retour GROUP BY 1, 2),
o AS (SELECT id_magasin, annee_mois am, ca_objectif_ttc obj FROM objectif_mois)
SELECT COALESCE(v.id_magasin, o.id_magasin) AS magasin, COALESCE(v.am, o.am) AS mois,
ROUND(v.ca) AS realise, o.obj AS objectif,
CASE WHEN o.obj IS NULL THEN 'objectif manquant'
WHEN v.ca IS NULL THEN 'vente manquante' ELSE 'complet' END AS etat
FROM v FULL OUTER JOIN o ON o.id_magasin = v.id_magasin AND o.am = v.am
WHERE o.obj IS NULL OR v.ca IS NULL;
```

magasin	mois	realise	objectif	etat
4	2024-02	42514372	NULL	objectif manquant
4	2024-03	51235118	NULL	objectif manquant

93 749 490 FCFA de ventes sans objectif en face. Le rapport de suivi a maintenant deux choix intellectuellement honnêtes : exclure ces deux mois **en le disant**, ou les afficher avec un taux de réalisation vide. Le choix malhonnête — et le plus fréquent — est de laisser la jointure interne les supprimer, puis d’annoncer « 218 mois suivis, 100 % de couverture ».

Attention. Une jointure interne sur une table de référence **réduit** le périmètre en silence. C’est le seul endroit du chapitre où la perte d’information ne produit ni **NULL**, ni erreur, ni ligne manquante : il n’y a simplement plus rien à voir. Règle de la maison : **toute jointure vers une table de référence se vérifie par un COUNT(*) avant et après**.

Définition. Un **trou de série** — series gap — est une période de l’axe de temps pour laquelle il n’existe **aucune** donnée : soit parce que rien ne s’est produit (un dimanche sans vente), soit parce que la mesure n’a pas été faite (un mois sans objectif). La jointure externe les rend **visibles** ; c’est sa seule fonction, et c’est suffisant pour la justifier.

5.3 Année sur année : le périmètre d’abord

Le socle s’arrête au 31/08/2026. Deux lectures du même chiffre :

Comparaison	Calcul	Résultat	Verdict
2026 (8 mois) contre 2025 (12 mois)	3 360 553 372 contre 4 724 700 286	−28,9 %	faux
janvier—août 2026 contre janvier—août 2025	3 360 553 372 contre 2 913 055 394	+15,4 %	défendable

La même année, la même base, deux conclusions opposées. La première n’est pas une erreur de calcul : c’est une erreur de **périmètre**. Elle est d’autant plus dangereuse qu’elle va dans le sens de la prudence (« l’activité baisse »), donc personne ne la conteste.

La bonne écriture est une fonction du périmètre, pas du calendrier :

```
WITH m AS (SELECT YEAR(date_vente) an, MONTH(date_vente) mo, SUM(montant_ttc) ca
FROM ventes WHERE NOT est_retour GROUP BY 1, 2)
SELECT an, ROUND(SUM(ca)) AS ca_total, ROUND(SUM(ca) FILTER (WHERE mo <= 8)) AS ca_8_mois
FROM m GROUP BY 1 ORDER BY 1;
```

2023	3443581547	2127742355
2024	4066319750	2527257254
2025	4724700286	2913055394
2026	3360553372	3360553372

La colonne **ca_8_mois** aligne les quatre années sur la même fenêtre — **8,8 %**, **+15,3 %**, **+15,4 %** — et cette fois 2026 se lit correctement. C’est aussi le contrôle que les trois croissances successives sont **cohérentes** entre elles.

5.4 Moyennes mobiles : la fenêtre pleine et son compteur

Un mois isolé est bruité : sur le socle, la variation mensuelle va de **-12,3 %** à **+27,1 %** en cinq mois, pour une tendance annuelle de **+16,8 %**. La moyenne mobile existe pour cela :

```
WITH m AS (SELECT date_trunc('month', date_vente) mm, SUM(montant_ttc) ca
            FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT strftime(mm, '%Y-%m') AS mois, ROUND(AVG(ca) OVER (ORDER BY mm
            ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) AS moyenne_3_mois,
COUNT(*) OVER (ORDER BY mm ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS lignes_3,
ROUND(AVG(ca) OVER (ORDER BY mm
            ROWS BETWEEN 11 PRECEDING AND CURRENT ROW)) AS moyenne_12_mois,
COUNT(*) OVER (ORDER BY mm ROWS BETWEEN 11 PRECEDING AND CURRENT ROW) AS lignes_12
FROM m ORDER BY mm LIMIT 4;
```

2023-01		284400304		1		284400304		1
2023-02		273791790		2		273791790		2
2023-03		293986146		3		293986146		3
2023-04		303978243		3		299083759		4

La cinquième colonne est la leçon : en janvier 2023, la « moyenne mobile de 12 mois » ne porte que sur **1** mois ; en avril, sur **4**. Sans le compteur, ces valeurs s’affichent avec la même autorité que celles des 32 mois complets — et la courbe a l’air de faire le travail pour lequel on l’a tracée.

Définition. Le **cadre de fenêtre** — window frame — est la partie de la partition réellement prise en compte autour de la ligne courante. `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` désigne trois lignes ; `ROWS BETWEEN 11 PRECEDING AND CURRENT ROW` en désigne douze. Le cadre **ne s’étend pas au-delà** des lignes existantes : quand il n’y en a qu’une, il n’en prend qu’une.

5.5 La saison, à connaître avant d’interpréter

Deux saisons structurent le socle.

Le jour de la semaine. Le lundi rapporte **13 872 370** FCFA par jour de vente, le dimanche **4 322 830** : rapport **3,2**. Un rapport hebdomadaire qui compare « ce lundi » à « ce samedi » mesure le calendrier, pas l’activité. Le contrôle se fait avec le calendrier du socle :

```
SELECT c.libelle_jour, ROUND(SUM(v.montant_ttc) / COUNT(DISTINCT v.date_vente)) AS ca_par_jour
FROM ventes v JOIN calendrier c ON CAST(c.date AS DATE) = v.date_vente
WHERE NOT v.est_retour GROUP BY 1 ORDER BY ca_par_jour DESC;
```

Le mois calendaire. Mars vaut en moyenne **433 645 399** FCFA par mois, juillet **262 079 192** : rapport **1,65**. Un premier trimestre et un troisième trimestre ne se comparent jamais directement.

Dans les faits. L’écart saisonnier (**×1,65** sur le mois, **×3,2** sur le jour) est **plus grand** que la croissance annuelle du socle (**+16,8 %**). Autrement dit : sur cette activité, le calendrier pèse davantage que la tendance. Toute conclusion tirée d’un mois isolé, sans référence au même mois de l’année précédente, a plus de chances de décrire le calendrier que l’entreprise.

5.6 Le mois partiel : la dernière ligne du tableau

Un mois n’est comparable à un autre que s’il est **complet**. Août 2026 l’est : **5 022** lignes de vente du 1er au 31, aucune journée manquante. Mais si le socle s’était arrêté au 15 août, le total du mois aurait été **environ deux fois** plus petit — et la moyenne mobile de 3 mois, la variation

annuelle et la croissance du trimestre auraient toutes été fausses **en même temps**, sans qu'aucune ne signale l'autre.

Attention. Le dernier point du graphique est presque toujours un mois **partiel**, et c'est celui qu'on regarde. Une moyenne mobile qui l'intègre baisse mécaniquement ; une variation annuelle qui l'utilise compare un bout de mois à un mois entier. La règle : le dernier point se publie **avec** son nombre de jours couverts, ou pas du tout — sur le socle, août 2026 est complet (5 022 lignes du 1er au 31), donc publiable ; un mois arrêté au 15 ne l'aurait pas été.

Le contrôle de complétude coûte une ligne :

```
SELECT COUNT(*) AS jours_de_vente, MIN(date_vente), MAX(date_vente)
FROM ventes WHERE NOT est_retour AND date_vente >= DATE '2026-08-01';
```

À retenir. Trois contrôles non négociables avant de publier une comparaison temporelle : la période est-elle **couverte** (calendrier), la référence est-elle **présente** (jointure externe), la fenêtre est-elle **pleine** (compteur de lignes). Les trois tiennent en trois colonnes, et les trois erreurs qu'ils évitent sont invisibles sans eux.

Conseil professionnel. Construisez la table calendrier **une fois** pour tout le service, et non une fois par rapport. Elle se vérifie en trois lignes (la période couverte, le nombre de jours, la présence des fériés) et se réutilise partout : comparaisons inter-annuelles, moyennes mobiles, jours ouvrés, découpages trimestriels. Un rapport qui fabrique son propre axe de temps le fabrique différemment des autres — et deux rapports qui ne partagent pas leur calendrier ne se réconcilient jamais.

5.7 Ce que ce chapitre ne couvre pas

La **rétenition** et les **cohortes** (le temps vu par client, et non par période) sont le sujet de C04. La **prévision** (prolonger une série) appartient à M19. Ici, on ne fait que rendre le temps complet, comparable et vérifiable.

6. Exemple concret — trois lectures de la même année

Un comité de direction demande « où en est l'année 2026 ? ». Le socle permet trois réponses, et une seule est utilisable :

Lecture	Requête	Résultat	Ce que ça vaut
Totale	SUM(montant_ttc) par année	3 360 553 372 FCFA	un chiffre, pas une réponse
Naïve	2026 contre 2025 complet	-28,9 %	faux (périmètre)
À périmètre égal	janvier—août 2026 contre janvier—août 2025	+15,4 %	la réponse

Et la réponse **s'accompagne** de sa saison : **+15,4 %** se compare à **+15,3 %** (2025) et **+18,8 %** (2024) sur la même fenêtre de huit mois. La conclusion tenable est donc : « la progression à périmètre égal est de **+15,4 %**, en très légère décélération par rapport aux **+18,8 %** de 2024 ». C'est une phrase qu'un décideur peut utiliser — parce qu'elle dit ce qui est comparé.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — compter l'axe du temps

```
SELECT COUNT(*) FROM generate_series(
  DATE '2023-01-01', DATE '2026-08-31', INTERVAL 1 DAY);
SELECT COUNT(*) FROM calendrier;
```

```
1339
1339
```

7.2 Étape 2 — le CA mensuel, et la preuve qu'il n'y a pas de trou

44 lignes, du 2023-01 au 2026-08, aucune manquante : la série mensuelle est **dense**.

7.3 Étape 3 — les objectifs, et leurs deux trous

C'est le `FULL OUTER JOIN` du §5.2 : 2 lignes d'état `objectif` manquant, magasin 4, février et mars 2024, pour **93 749 490** FCFA de ventes.

7.4 Étape 4 — année sur année à périmètre égal

Le tableau du §5.3 : **2 127 742 355 ; 2 527 257 254 ; 2 913 055 394 ; 3 360 553 372** FCFA sur les huit premiers mois, soit **+18,8 %**, **+15,3 %** et **+15,4 %**.

7.5 Étape 5 — moyennes mobiles avec compteurs

Les trois derniers mois du socle, avec des fenêtres **complètes** :

2026-06	351152152	moyenne 3 mois : 423242990	moyenne 12 mois : 421347035
2026-07	328841773	moyenne 3 mois : 366577753	moyenne 12 mois : 426175710
2026-08	368251515	moyenne 3 mois : 349415147	moyenne 12 mois : 431016522

La moyenne de 3 mois baisse (**423 242 990** → **349 415 147**) alors que la moyenne de 12 mois monte (**421 347 035** → **431 016 522**) : le court terme et le long terme ne racontent pas la même chose, et c'est normal. Une seule des deux courbes ne suffit jamais à décider.

7.6 Étape 6 — la saison, pour interpréter

Le jour (rapport **3,2**) et le mois (rapport **1,65**) du §5.5. À partir d'ici, chaque écart publié dans le module est comparé **au même mois de l'année précédente**, jamais au mois d'avant.

8. Erreurs fréquentes

1. **Comparer une période partielle à une période complète.** 2026 compte **8** mois, 2025 en compte **12** : la comparaison directe publiée **-28,9 %** au lieu de **+15,4 %** à périmètre égal. La faute est invisible parce qu'elle va dans le sens de la prudence.
2. **Joindre une table de référence en jointure interne.** Les **2** mois sans objectif du magasin 4 disparaissent du rapport : **93 749 490** FCFA de ventes s'évaporent sans un message, et la couverture annoncée devient « 100 % ».
3. **Publier une moyenne mobile incomplète.** En janvier 2023, une moyenne de 12 mois porte sur **1** mois ; en avril, sur **4**. Sans la colonne `COUNT(*) OVER` (même cadre), la courbe est lisse et fausse sur les douze premiers points.

4. **Remplacer un NULL par zéro.** Un mois sans objectif n'est pas un mois d'objectif nul : publier 0 produit un taux de réalisation infini ou nul selon la formule. NULL signifie « inconnu » — et « inconnu », dans un rapport, se déclare.
5. **Comparer deux mois sans regarder la saison.** Mars (433 645 399 FCFA par mois) contre juillet (262 079 192) donne **-40 %** sur un socle en croissance : l'écart est un écart de calendrier.

9. Bonnes pratiques professionnelles

1. **Vérifier la couverture de la période avant tout calcul.** Le MIN/MAX des dates, le nombre de jours ou de mois couverts, la comparaison au calendrier de référence : trois requêtes qui coûtent quelques secondes et qui conditionnent la validité de tout le reste.
2. **Écrire le périmètre dans le titre du rapport.** « Janvier—août 2026 contre janvier—août 2025 : +15,4 % » — le titre porte le périmètre, et personne ne peut plus confondre les deux lectures.
3. **Toujours joindre la saison à la comparaison.** Un écart annuel de **+15,4 %** se lit par rapport à la progression des années précédentes sur la même fenêtre (**+15,3 %**, **+18,8 %**), pas dans l'absolu.
4. **Compter les lignes de chaque fenêtre.** Un cadre (ROWS BETWEEN ...) et un COUNT(*) OVER avec le même cadre : la valeur et sa fiabilité dans la même ligne de résultat.
5. **Déclarer les trous dans la note de méthode.** Deux mois sans objectif, douze mois sans antériorité : ce sont des informations sur la base, et elles doivent figurer dans le rapport — pas être lissées.

10. Exercice guidé

Situation. Le directeur général reçoit un rapport intitulé « **2026 : l'activité recule de 28,9 %** », qui compare le chiffre d'affaires 2026 au chiffre d'affaires 2025. Il appelle le service analyse, très inquiet. Vous avez 15 minutes pour produire une réponse écrite avant son comité.

Consigne.

1. Reproduisez le chiffre du rapport. (2 pts)
2. Écrivez la comparaison à périmètre égal, avec le titre que vous proposeriez. (3 pts)
3. Ajoutez la référence des deux années précédentes sur la **même** fenêtre, et dites ce que la comparaison des trois progressions apporte. (3 pts)
4. Le DG demande : « pourquoi août 2026 peut-il être comparé à août 2025, alors qu'août 2025 avait **31** jours ? » Répondez en une phrase. (2 pts)
5. Citez la requête de contrôle qui prouve qu'aucun mois de ventes ne manque dans la base. (2 pts)

11. Exercices autonomes

E1 — Le tableau de suivi des objectifs, trous compris (40 min). Produisez, pour le magasin 4, un suivi mensuel 2024 : réalisé, objectif, taux de réalisation, et une colonne état qui distingue les trois cas (complet, objectif manquant, vente manquante). Contrainte : la jointure sera **externe**, et le rapport devra afficher les **2** mois sans objectif — pas les supprimer.

E2 — Deux moyennes mobiles, une conclusion (35 min). Pour le réseau entier, produisez la série mensuelle avec la moyenne mobile de 3 mois et celle de 12 mois, **chacune accompagnée du nombre de lignes de sa fenêtre**. Puis répondez en trois lignes : que disent les deux courbes sur le dernier trimestre, et pourquoi ne disent-elles pas la même chose ?

12. Correction détaillée

Exercice guidé.

1. **Reproduire le chiffre.** `SELECT YEAR(date_vente), SUM(montant_ttc) ... GROUP BY 1` donne **3 360 553 372 FCFA** pour 2026 et **4 724 700 286 FCFA** pour 2025, soit **-28,9 %**. Le calcul est juste ; la comparaison ne l'est pas.
2. **Le périmètre égal.** Même requête, avec `WHERE MONTH(date_vente) <= 8` avant d'agréger par année : **3 360 553 372** contre **2 913 055 394**, soit **+15,4 %**. Titre proposé : « **Janvier—août 2026 contre janvier—août 2025 : +15,4 %** » — le périmètre est dans le titre, la question de périmètre est close.
3. **Les deux années de référence.** Sur la même fenêtre : **+18,8 %** (2024/2023) puis **+15,3 %** (2025/2024) puis **+15,4 %** (2026/2025). La progression est **stable**, très légèrement décélérante au milieu de la période. C'est le seul énoncé qui survive à une question du comité.
4. **Les 31 jours d'août.** Parce que la comparaison porte sur **les deux années au même mois**, donc sur deux mois d'août complets de 31 jours : la longueur du mois est identique des deux côtés, comme le nombre de jours fériés.
5. **Le contrôle de complétude.** 44 lignes mensuelles attendues entre 2023-01 et 2026-08 ; la requête `SELECT COUNT(DISTINCT date_trunc('month', date_vente)) FROM ventes WHERE NOT est_retour` en rend **44** — et le croisement avec le calendrier (1 339 jours) confirme la couverture.

E1 (le suivi du magasin 4). La structure attendue est le `FULL OUTER JOIN` du §5.2, avec le `CASE` à trois états. Les deux lignes sans objectif portent **42 514 372** et **51 235 118 FCFA** de réalisé, et le taux de réalisation doit y être `NULL` — pas 0, pas « 100 % ». Total des ventes sans objectif : **93 749 490 FCFA**.

E2 (les deux mobiles). Au dernier mois du socle, la moyenne de 3 mois vaut **349 415 147 FCFA** et celle de 12 mois **431 016 522 FCFA** : la première baisse (elle a « oublié » un mois creux sorti du cadre), la seconde monte (elle lisse l'année entière). Les deux sont justes ; la première décrit le trimestre, la seconde la trajectoire. Un rapport qui n'en publie qu'une choisit à la place du lecteur — et le plus souvent celle qui confirme ce qu'il espérait.

13. Mini-projet de chapitre

La note de conjoncture d'un magasin, 2 pages. Choisissez un magasin du réseau et produisez sa note de conjoncture sur le dernier trimestre clos (juin—août 2026), avec le périmètre annoncé et la saison en regard.

Livrables et barème.

Critère	Ce qui est vérifié	Points
Le tableau mensuel	mois, CA, cumul, moyenne mobile de 3 mois, avec le compteur de fenêtre	6
La comparaison annuelle	les trois mois contre les trois mêmes mois de 2025, variations exactes	5
Le contrôle de complétude	la requête de couverture (calendrier, 1 339 jours) et sa sortie	4
La conclusion	une phrase, périmètre annoncé, saison citée, et une limite	3
Total		18

14. Résumé du chapitre

Le temps en SQL se traite en trois gestes : **fabriquer** l'axe (calendrier ou `generate_series`, ici **1 339** jours qui concordent), **joindre** les données à cet axe en **externe** pour que les trous apparaissent (les **2** mois sans objectif du magasin 4, **93 749 490 FCFA** de ventes sans référence), et

compter les lignes de chaque fenêtre pour savoir si elle est pleine. La comparaison temporelle exige un **périmètre** explicite : 2026 contre 2025 vaut **-28,9 %** ; janvier—août contre janvier—août vaut **+15,4 %** — même base, deux conclusions, une seule défendable. Les moyennes mobiles lissent le bruit mensuel (**-12,3 %** à **+27,1 %** d'un mois sur l'autre) mais ne valent qu'avec leur compteur de lignes. Enfin la **saison** doit être connue avant toute interprétation : **×3,2** entre le lundi et le dimanche, **×1,65** entre mars et juillet — plus grand que la croissance annuelle du socle (**+16,8 %**), ce qui dit l'essentiel : sur cette activité, le calendrier pèse plus que la tendance.

15. À retenir

1. **Un axe de temps complet est la condition de tout rapport temporel.** Sans calendrier, les périodes sans donnée n'existent pas — et l'absence se lit comme une absence de problème.
2. **Toute jointure vers une référence se fait en externe et se vérifie par un comptage.** La jointure interne supprime en silence.
3. **Aucune comparaison sans périmètre écrit.** « 2026 contre 2025 » n'est pas un périmètre ; « janvier—août contre janvier—août » en est un.
4. **Une fenêtre glissante se publie avec son compteur.** Une moyenne de 12 mois calculée sur 4 mois est une moyenne de 4 mois qui porte un mauvais nom.
5. **La saison se connaît avant l'interprétation.** Sur ce socle, le calendrier explique plus d'écart que la tendance.

À retenir. Le chiffre qui résume le chapitre n'est pas **+15,4 %** : c'est la paire **-28,9 % / +15,4 %** — le même socle, la même année, la même question, et une phrase dans le titre qui décide laquelle des deux est publiée.

À retenir. Un trou déclaré est une information ; un trou supprimé est une erreur. Les **2** mois du magasin 4 sont le seul endroit du socle où la réponse honnête est « je ne sais pas ».

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien de jours comptent le calendrier du socle, et combien en rend `generate_series` sur la même période ?
2. Combien de mois de ventes manque-t-il sur le socle ? Combien de mois d'objectifs ?
3. Que publie la comparaison du total 2026 au total 2025, et que publie la comparaison à périmètre égal ?
4. Pourquoi une jointure interne sur `objectif_mois` est-elle dangereuse ici ?
5. Que signifie `lignes_12 = 4` en avril 2023 dans la requête de moyenne mobile ?
6. Quel rapport y a-t-il entre le lundi et le dimanche sur ce socle ?
7. Quel rapport entre mars et juillet, et que faut-il en conclure pour un rapport trimestriel ?
8. Combien de mois du socle ont un équivalent un an plus tôt ?
9. Quelle est la progression annuelle moyenne sur ces mois comparables ?
10. Que doit contenir le titre d'un rapport de comparaison temporelle ?

Réponses. 1. **1 339** et **1 339** : les deux axes concordent. 2. Aucun mois de ventes (**0** sur **44**) ; **2** mois d'objectifs (**magasin 4**, février et mars 2024). 3. **-28,9 %** (2026 n'a que huit mois) ; **+15,4 %** à périmètre égal. 4. Parce qu'elle **supprime** les deux mois sans objectif — **93 749 490 FCFA** de ventes — sans erreur ni `NULL`. 5. Que la « moyenne de 12 mois » ne porte que sur **4** lignes : la

fenêtre n'est pas pleine. 6. **3,2** (**13 872 370** contre **4 322 830** FCFA par jour de vente). 7. **1,65** (**433 645 399** contre **262 079 192** FCFA par mois) : un premier trimestre ne se compare pas à un troisième sans correction. 8. **32** des **44** mois. 9. **+16,8 %**. 10. Le **périmètre** : « janvier—août 2026 contre janvier—août 2025 ».

Module M11.C04 — Cohortes, rétention et récurrence : RFM, et le client qui n'a pas de nom

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C03 (fenêtres, cadres, temps) ; M09.C04 (segmentation client) ; M08.C06 (indicateurs de fidélité).

L'idée du chapitre. Il existe deux façons de mesurer si les clients reviennent, et la plus répandue est la mauvaise. La lecture **calendaire** demande « combien des clients actifs en juin sont revenus en juillet ? » — sur le socle, **13,2 % (432 des 3 276 clients de juin)**. La lecture **par cohorte** demande « quels clients dont c'était le premier achat en juin sont revenus en juillet ? » — **22,7 %**, mais sur **22 clients** seulement. Les deux nombres prétendent décrire la même chose et n'ont ni la même base, ni la même signification : le premier est stable et mesure le **mélange** du fichier, le second est juste et trop petit pour être publié. Ce chapitre apprend à construire la **cohorte**, à lire la matrice en **triangle**, et à pondérer les taux avant de les commenter. Puis il assemble les trois mesures que tout le monde utilise sans les définir — récence, fréquence, montant — et montre ce qu'elles deviennent quand **18,3 %** du chiffre d'affaires est porté par un client qui n'a pas de nom.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). 23 497 clients de vente, 44 cohortes mensuelles, 237 191 lignes nettes, 15 595 154 955 FCFA, du 01/01/2023 au 31/08/2026. Toutes les sorties publiées ici sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Construire une cohorte** sur l'événement métier qui compte — la **première commande** — et non sur la date d'inscription du référentiel.
2. **Produire une matrice de rétention** cohorte × ancienneté, et **reconnaître sa forme** : un triangle, pas un rectangle.
3. **Calculer une rétention pondérée** par la taille des cohortes, et expliquer pourquoi la moyenne simple des taux est fausse.
4. **Distinguer trois questions voisines** : la rétention par cohorte, la récurrence calendaire et l'activité d'un mois.
5. **Segmenter en RFM** avec NTILE, départage explicite, et nommer les segments sans inventer de seuils « métier ».
6. **Traiter le client non identifié** : le mesurer, décider, et l'écrire dans la note de méthode.

2. Pourquoi cette notion est importante

Le socle permet de répondre à une question que le comité de direction posera tôt ou tard : **nos clients reviennent-ils ?** Il faut choisir un chiffre. Les candidats :

Ce qui est calculé	Chiffre	Ce que ça mesure vraiment
Clients actifs en juin 2025 revenus en juillet 2025	13,2 % (432 / 3 276)	le renouvellement du fichier actif , toutes cohortes mêlées
Clients dont juin 2025 est le premier achat, revenus en juillet	22,7 % (5 / 22)	la fidélité des nouveaux — sur un effectif trop faible pour conclure
Rétention moyenne pondérée au premier mois suivant, toutes cohortes	15,8 %	la fidélité de la base entière, chaque client pesant une fois

Le troisième chiffre est le seul qui soit à la fois **juste** et **publiable**. Le premier est juste aussi, mais il ne répond pas à la question posée : il dépend du **mélange** de cohortes du mois, et il resterait autour de **13 %** même si plus aucun nouveau client ne revenait jamais. Le deuxième est juste et inutilisable : **22** clients.

Ce chapitre existe parce que ces trois chiffres sortent de trois requêtes qui se ressemblent, que rien dans le résultat n’indique laquelle on a écrite, et que le plus gros des trois (le plus flatteur) est celui qu’on retient.

3. Explication simple — la promotion d’une année, et le verre à moitié plein

Une école veut savoir si ses élèves de première année poursuivent en deuxième. Deux méthodes.

La méthode fausse. En septembre, on compte tous les élèves présents : premières, deuxièmes, troisièmes années confondues. On recompte les mêmes en octobre. On annonce « **93 %** de nos élèves restent ». Le chiffre est exact et ne dit rien : la masse des troisièmes années, qui ne peuvent pas partir avant la fin du cycle, écrase le résultat. C’est la lecture calendaire.

La méthode juste. On prend **une** promotion, celle entrée en septembre. On la suit : combien sont là en octobre, en novembre, un an plus tard. On obtient une courbe. Cette courbe, comparée à celle des promotions entrantes des deux années précédentes, dit si l’école retient mieux qu’avant. C’est la cohorte.

Ce qu’aucune des deux ne donne. La lecture calendaire ne dit pas la qualité du service ; la lecture par cohorte ne dit presque rien pour la promotion entrée ce mois-ci, parce qu’elle est petite et récente. Ce qu’il faut, c’est **toutes les cohortes, pondérées** : chaque élève pèse une fois, quelle que soit son année d’entrée. Sur le socle, cela donne **15,8 %** — et ce chiffre-là ne bouge plus quand le mélange change.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
cohorte	cohort	Un groupe de clients partageant un événement de départ daté — ici le mois de leur première commande.
ancienneté	cohort age	Le nombre de mois écoulés depuis l’événement de départ ; noté M+1, M+3, M+12.
matrice rétention	de retention matrix	Le tableau cohortes × anciennetés : chaque ligne est une cohorte, chaque colonne une ancienneté.
rétention	retention	La part d’une cohorte revenue au moins une fois pendant l’ancienneté considérée.
récurrence	recurrence	La part des clients actifs un mois donné qui reviennent le mois suivant — toutes cohortes mêlées.
pondération	weighting	Donner à chaque cohorte un poids proportionnel à sa taille, et non le même poids à toutes.

Terme (français)	Équivalent anglais	Définition en une phrase
récence	recency	Le délai écoulé depuis le dernier achat d'un client.
fréquence	frequency	Le nombre d'achats (ou de tickets) d'un client sur la période.
montant	monetary value	Le chiffre d'affaires attribué au client sur la période.
RFM	recency, frequency, monetary	Les trois mesures réunies, chacune découpée en tiers, pour segmenter le fichier client.
segment	segment	Le croisement des trois découpages : ici 133, 233, 333 — trois chiffres, de 1 à 3.
client identifié	non anonymous customer / default member	Un identifiant technique qui agrège des achats sans porteur : ici le client 0 .

Définition. Une **cohorte** est un ensemble de clients réunis par un **événement daté** commun — et l'événement doit être choisi avec soin. Le référentiel du socle contient une date d'inscription qui commence en **2016**, alors que la première vente observée n'est jamais antérieure à janvier 2023 : les clients existaient avant les données. Une cohorte d'**inscription** décrirait donc un autre monde que la cohorte de **première commande**. Règle : la cohorte est datée sur l'événement que le rapport mesure.

Définition. La **rétenion** est la part d'une cohorte encore active après N périodes. Elle exige donc **deux** définitions explicites : ce qu'est « active » (ici : au moins un ticket non retourné dans le mois) et ce qu'est « N » (ici : des mois complets, pas des fenêtres de 30 jours). Deux rapports qui ne partagent pas ces deux définitions ne peuvent pas comparer leurs taux.

5. Cours approfondi

5.1 La cohorte d'entrée : 44 lignes, une seule par client

```
WITH p AS (SELECT id_client, date_trunc('month', MIN(date_vente)) AS co
           FROM ventes WHERE NOT est_retour GROUP BY 1)
SELECT co, COUNT(*) AS taille FROM p GROUP BY 1 ORDER BY 1;
```

44 lignes, du 2023-01 au 2026-08. La première cohorte contient **3 593** clients, la dernière (**2026-08**) exactement **3**. Ce déséquilibre n'est pas une anomalie de la base : c'est le résultat attendu d'une période d'observation qui commence. Tous les clients qui étaient déjà là en janvier 2023 sont comptés comme « entrants de janvier 2023 » — on dit que la série est **tronquée à gauche**.

Attention. La troncature à gauche ne se corrige pas : on ne peut pas inventer les achats antérieurs au 01/01/2023. Elle se **déclare**. Sans cette phrase dans la note de méthode, le lecteur croira que la base a gagné **3 593** clients en un mois et en a perdu **3 590** en trois ans et demi.

5.2 La matrice : un triangle, jamais un rectangle

La matrice croise chaque cohorte avec chaque ancienneté. Extrait des six premières cohortes, sur six anciennetés :

Cohorte	Taille	M+0	M+1	M+2	M+3	M+4	M+5
2023-01	3 593	100,0 %	14,6 %	18,3 %	17,5 %	15,0 %	12,9 %
2023-02	2 817	100,0 %	19,1 %	17,8 %	15,9 %	12,6 %	11,5 %
2023-03	3 137	100,0 %	16,2 %	15,0 %	11,3 %	10,5 %	13,7 %

Cohorte	Taille	M+0	M+1	M+2	M+3	M+4	M+5
2023-04	2 389	100,0 %	14,6 %	13,3 %	11,3 %	13,0 %	15,9 %
2023-05	1 653	100,0 %	11,6 %	11,4 %	13,3 %	16,0 %	18,4 %
2023-06	1 239	100,0 %	11,0 %	13,6 %	14,9 %	19,9 %	17,5 %

Deux choses sautent aux yeux, et aucune n'est un taux de fidélité.

Le triangle. Les **44** cohortes n'ont pas le même recul : **32** sont observables jusqu'à M+12, **20** jusqu'à M+24. La matrice a donc la forme d'un escalier, et la case M+24 de la cohorte de 2026-08 est **impossible** — pas vide par manque de données, mais par manque de **temps**. Un tableau qui la remplirait par zéro publierait une chute que personne n'a mesurée.

Le mélange des cohortes. Chaque ligne a sa taille : **3 593** en tête, **1 239** six mois plus tard. Une rétention « moyenne » calculée comme la moyenne des taux donnerait donc le même poids à ces deux populations — c'est la faute du \$5.3.

Retention par cohorte de première commande, en %													
	M+0	M+1	M+2	M+3	M+4	M+5	M+6	M+7	M+8	M+9	M+10	M+11	M+12
2023-01 (3593)	100	15	18	18	15	13	12	12	16	19	19	18	17
	100	19	18	16	13	12	13	16	18	18	18	16	15
	100	16	15	11	10	14	16	18	18	19	16	16	21
2023-04 (2389)	100	15	13	11	13	16	18	20	18	17	17	20	18
	100	12	11	13	16	18	18	18	16	15	19	21	15
	100	11	14	15	20	18	17	15	16	19	18	16	14
2023-07 (1017)	100	11	16	19	16	20	18	17	20	18	14	11	14
	100	14	17	19	18	17	17	20	18	17	14	12	14
	100	17	19	17	16	17	18	20	16	14	14	14	15
2023-10 (1074)	100	19	16	17	17	20	18	15	14	13	14	18	20
	100	21	18	14	20	19	17	12	14	13	17	18	20
	100	18	17	20	20	16	13	13	10	16	22	20	20
2024-01 (474)	100	19	23	22	14	12	15	16	17	20	23	17	18
	100	20	14	16	12	13	14	16	19	21	17	15	16
	100	16	12	14	11	14	17	22	20	20	17	17	23
2024-04 (324)	100	17	13	12	12	15	24	22	21	21	15	23	20
	100	13	15	16	12	17	24	20	21	18	19	19	15
	100	15	16	18	22	19	24	15	14	32	12	21	12
2024-07 (109)	100	10	16	22	18	23	13	17	19	24	17	10	10
	100	17	19	20	23	16	21	16	19	16	16	15	14
	100	26	12	20	26	15	22	18	21	12	11	10	18
2024-10 (148)	100	18	20	20	18	20	18	14	16	16	13	16	22
	100	17	21	18	21	22	17	17	10	11	25	22	20
	100	24	12	21	25	24	11	11	17	21	18	19	14
2025-01 (62)	100	13	16	19	14		13	13	18	13	29	24	14
	100	22	19	22		14	14	16	32	24	32	16	27
	100	16	14		16	20	16	16	26	22	16	31	22
2025-04 (43)	100	16	21	16	14	14	26	26	21	28	21	19	21
	100	16	12	12	12	12	31	34	19	19	19	28	25
	100	23	18		23	18	18	23	18	18	14	18	23
2025-07 (13)	100	31	15	23		23	15			31	38	15	
	100	25	38	25	31	19	12	31		12	19		25
	100	29		12	12	12	29	29	12	18	18	24	
2025-10 (9)	100	11	11	33	33	22	11			11			
	100	38	12	25	25	12	12		12				
	100	40	10	30	30	10	20	10	20				
2026-01 (8)	100	25	50	12	12	12		12					
	100	50	17		50		17						
	100			67		33							
2026-04 (4)	100												
	100												
	100												
2026-07 (1)	100												
	100	100											
	100												

Une matrice de retention s'arrete en triangle

5.3 Le piège des deux lectures

Le même mois, deux questions, deux réponses :

```
-- lecture calendaire : les actifs de juin, revenus en juillet
WITH a AS (SELECT DISTINCT id_client, date_trunc('month', date_vente) AS m
FROM ventes WHERE NOT est_retour)
SELECT COUNT(*) AS base_juin,
COUNT(*) FILTER (WHERE id_client IN (SELECT id_client FROM a WHERE m = DATE '2025-07-01'))
AS revenus
FROM a WHERE m = DATE '2025-06-01';
```

```
base_juin | revenus
3276      | 432
```

13,2 %. C'est un taux de **récence**, pas de rétention : il mesure la proportion des clients actifs d'un mois qui reviennent le mois suivant, quelle que soit leur date d'entrée. Il est utile (c'est la « respiration » du fichier) et il ne répond pas à « nos nouveaux clients reviennent-ils ? ».

Pour répondre à cette question-là, il faut la même paire de mois vue **par la cohorte** :

```
WITH p AS (SELECT id_client, date_trunc('month', MIN(date_vente)) AS co
  FROM ventes WHERE NOT est_retour GROUP BY 1),
  a AS (SELECT DISTINCT id_client, date_trunc('month', date_vente) AS m
  FROM ventes WHERE NOT est_retour),
  c AS (SELECT id_client FROM p WHERE co = DATE '2025-06-01')
SELECT (SELECT COUNT(*) FROM c) AS base,
  (SELECT COUNT(*) FROM c WHERE id_client IN
    (SELECT id_client FROM a WHERE m = DATE '2025-07-01')) AS revenus;
```

```
base | revenus
22   | 5
```

22,7 %, sur **22** clients. Le chiffre est juste et n'est pas publiable : à cette taille, un client de plus ou de moins le fait bouger de 4,5 points.

Le troisième chiffre est celui qu'on publie — la **moyenne pondérée de toutes les cohortes** : **15,8 %** au premier mois, chaque client pesant une fois. Le contrôle qui justifie la pondération tient en une requête : sur les **23** cohortes d'au moins **100** clients, la moyenne **simple** des taux vaut **16,4 %** et la moyenne **pondérée** **15,8 %**. L'écart n'est pas énorme, mais il existe — et il vient de là : dans la moyenne simple, une cohorte de **22** clients pèse autant qu'une cohorte de **3 593**.

Définition. La **pondération** consiste à donner à chaque cohorte un poids proportionnel à son effectif. Un taux moyen pondéré est la somme des clients revenus divisée par la somme des clients de départ — jamais la moyenne des taux. Le calcul direct est d'ailleurs plus simple : `SUM(actifs) / SUM(effectif)`.

Dans les faits. La différence entre **16,4 %** et **15,8 %** paraît faible, et elle est **signée** : les petites cohortes récentes ont des taux plus élevés (elles contiennent des primo-acheteurs encore chauds), donc la moyenne simple **surestime** systématiquement. Sur un fichier client en forte croissance, l'écart se creuse — et il va toujours dans le sens flatteur.

5.4 Une rétention qui ne baisse pas

Le résultat le plus important du chapitre est peut-être le plus décevant :

Ancienneté	M+1	M+3	M+6	M+12	M+24
Rétention pondérée	15,8 %	15,4 %	15,9 %	17,2 %	18,2 %

La courbe est **plate**, et elle remonte même légèrement. La cohorte de janvier 2023 vue dans son intégralité le confirme : de M+1 à M+43, elle reste entre **11,5 %** et **22,6 %**, sans tendance à la baisse, et se termine à **16,9 %**.

L'interprétation est un exercice de rigueur, pas d'enthousiasme. Le socle ne raconte **pas** une entreprise qui perd ses clients chaque mois. Il raconte une base composée de deux populations : une minorité d'acheteurs réguliers, présents presque tous les mois, et une majorité de clients de passage — le socle compte **3 316** clients à **8** tickets et **64** clients dont toute l'histoire tient dans **un seul** mois. Le taux de **15,8 %** est le poids de la première population dans le fichier.

Attention. « 15,8 % de rétention » ne signifie pas que **84 %** des clients sont perdus. Le taux compte comme active toute personne ayant acheté **au moins une fois** dans le mois : il ne mesure pas l'intensité, et un client régulier qui saute un mois compte comme sorti. Avant de parler de perte, il faut une deuxième mesure — la **récence** du \$5.6.

5.5 Le RFM : trois mesures, une décision

Le RFM réunit trois colonnes par client — **R**écence, **F**réquence, **M**ontant — et les découpe chacune en trois blocs de taille égale représentés par 1, 2, 3 :

```
WITH c AS (SELECT id_client, MAX(date_vente) AS derniere, COUNT(DISTINCT id_ticket) AS f,
                SUM(montant_ttc) AS m
            FROM ventes WHERE NOT est_retour GROUP BY 1),
    s AS (SELECT id_client,
                NTILE(3) OVER (ORDER BY derniere ASC, id_client) AS r,
                NTILE(3) OVER (ORDER BY f DESC, id_client) AS f3,
                NTILE(3) OVER (ORDER BY m DESC, id_client) AS m3,
                m
            FROM c)
SELECT CAST(r AS VARCHAR) || CAST(f3 AS VARCHAR) || CAST(m3 AS VARCHAR) AS segment, m FROM s;
```

NTILE(3) produit trois blocs de **7 833**, **7 832** et **7 832** clients sur les **23 497** du socle : c'est le découpage le plus égal possible, et il est exact. La lettre **1** va au **meilleur** tiers, donc la récence se trie en ordre **croissant** (dernier achat récent) alors que la fréquence et le montant se trient en ordre **décroissant**.

Trois résultats sur le socle :

- Le segment **133** — récents, peu fréquents, gros montants — réunit **2 151** clients et **4 937 755 609** FCFA, soit **31,7 %** du chiffre d'affaires net. C'est le plus gros segment par le montant, et il est **injouable** en l'état : il contient le client **0** (voir \$5.7).
- Le tiers des **récents** (dernier achat entre le **18/06/2026** et le **31/08/2026**) porte **48,7 %** du chiffre d'affaires, contre **23,1 %** pour le tiers des anciens (dernier achat entre le **31/01/2023** et le **13/03/2026**).
- Le fichier se répartit donc en **27** segments possibles ($3 \times 3 \times 3$), dont la plupart restent vides ou minuscules : c'est normal, et un rapport qui n'en nomme que quatre ou cinq fait le bon choix.

Définition. Le **RFM** n'a pas de frontières intrinsèques : les coupures en trois blocs égaux sont un choix de méthode (égalité des effectifs), pas une vérité métier. D'autres entreprises coupent à des seuils absolus (« plus de **12** mois sans achat », « au moins **5** commandes »). Ce qui rend un RFM défendable n'est jamais le découpage : c'est le fait de **l'écrire**.

Conseil professionnel. Nommez les segments avec les bornes qui les définissent, jamais avec un adjectif. « Récents à faible fréquence, dernier achat depuis le 18/06/2026 » se vérifie ; « clients premium » ne se vérifie pas — et se discute pendant trois réunions.

5.6 Fréquence, récurrence, récence : les trois chiffres qui tiennent

Trois mesures complètent le RFM et se calculent sans segmentation :

Mesure	Requête	Résultat sur le socle
Récurrence	écart entre deux tickets consécutifs du même client	médiane 96 jours, moyenne 137,2 jours, sur 171 737 paires
Récence	délai depuis le dernier achat, au 31/08/2026	moyenne 156 jours, médiane 122 jours

Mesure	Requête	Résultat sur le socle
Inactivité	clients sans achat depuis plus d'un an	1 848 clients, 4,3 % du CA net

Deux enseignements. D'abord, l'écart systématique entre médiane et moyenne (**96** contre **137,2** jours ; **122** contre **156**) est encore l'effet des gros clients : la moyenne décrit une minorité, la médiane décrit le client courant. Ensuite, la règle « pas d'achat depuis plus d'un an = client perdu » donne **1 848** clients — un effectif de campagne de réactivation, avec des noms et des adresses, et seulement **4,3 %** du chiffre d'affaires à la clé. C'est précisément le genre d'arbitrage qu'un rapport doit rendre possible : le coût d'une campagne se compare à **4,3 %** du CA, pas à une intuition.

5.7 Le client 0 : 18,3 % du CA sans porteur

Sur **23 497** identifiants clients, un seul n'existe pas dans le référentiel : le client **0**. Il porte **42 658** lignes, **38 984** tickets et **2 852 612 447** FCFA — **18,3 %** du chiffre d'affaires net du socle.

Le traiter « à part » n'est pas une option esthétique : c'est ce qui décide de la conclusion. La question posée par l'évaluation du module — « **10 %** des clients font-ils **60 %** du chiffre d'affaires ? » — se répond ainsi :

Périmètre	Part du CA du dixième supérieur	Nombre de clients
Tous clients	38,1 %	2 350
Client 0 exclu	24,2 %	2 349

Dans les deux cas, la réponse est **non** — mais elle a deux valeurs différentes, et la note de méthode doit dire laquelle est publiée. Le client **0** appartient au dixième supérieur dans la lecture « tous clients » : c'est **un** identifiant qui pèse à lui seul plus que les neuf dixièmes du fichier.

Il y a plus grave, et c'est une leçon de reproductibilité : sans clé de départage explicite, NTILE et ROW_NUMBER **ne sont pas déterministes**. Le même recoupement RFM, exécuté **15** fois le 24/09/2026 sans ORDER BY ..., id_client, a situé le segment principal entre **2 718** et **2 738** clients selon l'ordonnancement interne du moteur. La cause est mesurable : **3 316** clients partagent exactement **8** tickets (**14,1 %** du fichier), donc la frontière des tiers tombe au milieu d'une égalité massive et le moteur choisit qui passe. Ce n'est pas un bug, c'est une requête incomplète.

Attention. Un rapport dont le chiffre change d'une exécution à l'autre n'est pas un rapport. Toute utilisation de NTILE, RANK ou ROW_NUMBER doit porter une clé de départage — ici id_client — et **la même** dans toutes les requêtes du rapport. Le contrôle se fait en comparant deux exécutions, pas en relisant la requête.

5.8 Ce que ce chapitre ne couvre pas

La **valeur vie client** (somme actualisée des marges futures) et la **prédiction de départ** exigent des modèles, traités en M14 et M19. Ici, tout est **descriptif** : on mesure ce qui a eu lieu, avec des définitions écrites. C'est déjà suffisant pour occuper une réunion entière.

6. Exemple concret — trois façons de répondre à la même question

Un responsable marketing demande : « combien de nos clients nous restent d'une année sur l'autre ? »

Réponse	Calcul	Valeur	Utilisable ?
Activité	clients actifs en juin 2025 revenus en juillet 2025	13,2 %	non : mesure le fichier, pas la fidélité
Cohorte du mois	cohorte de juin 2025, revenue en juillet	22,7 % sur 22 clients	non : effectif trop faible
Rétention pondérée	toutes cohortes, à M+12	17,2 %	oui, avec sa définition

La réponse à envoyer est la troisième, **accompagnée** de la phrase qui la rend lisible : « **17,2 %** des clients d'une cohorte achètent de nouveau douze mois après leur premier achat ; le taux est **plat** de M+1 à M+24 (**15,8 %** à **18,2 %**), ce qui décrit un noyau d'acheteurs réguliers dans une base majoritairement composée de clients de passage ». Trois chiffres, une phrase, aucune ambiguïté sur ce qui est mesuré.

7. Démonstration pas à pas — cinq étapes sur le socle

7.1 Étape 1 — la cohorte d'entrée

44 cohortes, de **3 593** clients (janvier 2023) à **3** (août 2026). Contrôle : la somme des tailles redonne les **23 497** clients du socle, chacun dans une seule cohorte.

7.2 Étape 2 — la matrice et son triangle

Le tableau du §5.2. Contrôle de forme : **32** cohortes tiennent jusqu'à M+12, **20** jusqu'à M+24. Les cellules au-delà ne sont pas nulles, elles sont **inexistantes**.

7.3 Étape 3 — les deux lectures, côte à côte

13,2 % (**432** des **3 276** clients actifs en juin 2025) en lecture calendaire ; **22,7 %** (**5** sur **22**) en lecture par cohorte. Puis la version publiable : **15,8 %** pondéré, contre **16,4 %** en moyenne simple sur les **23** cohortes d'au moins **100** clients.

7.4 Étape 4 — le RFM, avec le client 0

Les trois blocs de **7 833**, **7 832** et **7 832** clients ; le segment **133** à **2 151** clients et **31,7 %** du CA ; le tiers des récents à **48,7 %** du CA. Puis le retrait du client **0** : dixième supérieur à **38,1 %**, puis **24,2 %**.

7.5 Étape 5 — les mesures de contrôle

Récurrence médiane **96** jours ; récence médiane **122** jours ; **1 848** clients inactifs depuis plus d'un an (**4,3 %** du CA) ; **64** clients à un seul mois d'activité. Ce sont ces quatre chiffres qui permettent de qualifier le taux de rétention de **15,8 %** — sans eux, il n'est qu'un nombre.

8. Erreurs fréquentes

1. **Dater la cohorte sur l'inscription.** Le référentiel porte des dates de création à partir de **2016**, les ventes commencent en **2023** : une cohorte d'inscription ne décrit pas le même univers. La cohorte se date sur l'événement mesuré.
2. **Compter des clients sur une jointure.** La jointure de la cohorte de janvier 2023 (**3 593** clients) avec les **44** mois d'activité rend **30 090** lignes. Un `COUNT(*)` au lieu d'un `COUNT(DISTINCT id_client)` multiplie la base par l'historique — sans erreur, sans avertissement.

3. **Moyenner des taux sans pondérer. 16,4 %** au lieu de **15,8 %** : la moyenne simple donne le même poids à la cohorte de juin 2025 (22 clients) qu'à celle de janvier 2023 (3 593).
4. **Classer sans clé de départage.** NTILE sur une fréquence qui compte 3 316 clients à 8 tickets fait bouger le segment principal entre 2 718 et 2 738 clients d'une exécution à l'autre.
5. **Lire un taux de rétention comme un taux de fidélité. 15,8 %** au premier mois, **18,2 %** à M+24 : la courbe est plate, elle ne décrit pas une fuite mais la part des acheteurs réguliers.

9. Bonnes pratiques professionnelles

1. **Écrire la définition de « actif » en tête du rapport.** Un ticket non retourné dans le mois, retours exclus : ce choix change le taux, et il doit être visible avant le premier chiffre.
2. **Publier la taille de la base à côté de chaque taux.** 22,7 % sur 22 clients et 15,8 % sur 23 483 clients ne se lisent pas avec le même crédit ; un taux sans effectif est une rumeur.
3. **Pondérer, jamais moyenner.** $\text{SUM}(\text{revenus}) / \text{SUM}(\text{effectif})$ en une seule requête, et non la moyenne de 44 taux.
4. **Traiter le client non identifié comme un cas, pas comme un client.** Il est mesuré (18,3 % du CA), nommé, et exclu des classements — ou conservé avec la mention explicite que le classement est faussé.
5. **Figer les classements par une clé de départage.** ORDER BY ..., id_client partout, et la preuve par deux exécutions comparées : c'est le seul moyen d'être reproductible.

10. Exercice guidé

Situation. La direction commerciale veut lancer une campagne de réactivation. Elle vous demande : « combien de clients avons-nous perdus, et combien cela nous coûte-t-il ? »

Consigne.

1. Écrivez la requête qui définit un client perdu — et justifiez la borne choisie. (2 pts)
2. Donnez le nombre de clients concernés et leur part du chiffre d'affaires. (3 pts)
3. Calculez la récurrence médiane et la récence médiane du fichier, et dites laquelle des deux vous utiliseriez pour fixer le délai de la campagne, et pourquoi. (3 pts)
4. Un collègue propose de joindre le référentiel clients pour filtrer les « vrais » clients. Que change cette jointure sur le total du socle ? (2 pts)

11. Exercices autonomes

E1 — La matrice de rétention complète, et sa lecture (45 min). Produisez la matrice des 44 cohortes, de M+0 à M+24, avec la taille de chaque cohorte en première colonne. Puis répondez en cinq lignes : les cohortes récentes retiennent-elles mieux ou moins bien que celles de 2023 ? Quelles cases de la matrice ne sont pas observables, et pourquoi ?

E2 — Deux segments RFM, deux politiques (35 min). Construisez le RFM avec départage explicite, puis comparez deux segments : le plus gros par le montant et celui des clients anciens à forte fréquence. Pour chacun : effectif, part du CA, plage de récence. Terminez par une phrase de politique commerciale par segment — une seule, et vérifiable par les chiffres que vous venez de produire.

12. Correction détaillée

Exercice guidé.

1. **La borne.** Un an sans achat est la borne la plus défendable sur ce socle, parce que la récurrence observée est longue (médiane **96** jours, moyenne **137,2** jours) : douze mois représentent plusieurs cycles d'achat manqués. La requête est un `MAX(date_vente)` par client comparé au **31/08/2026**.
2. **L'effectif.** **1 848** clients, soit **4,3 %** du chiffre d'affaires net — **663 520 490** FCFA. La bonne phrase est : « **4,3 %** du CA est à la clé, pas plus » ; c'est ce chiffre qui compare la campagne à son coût.
3. **Quelle mesure choisir.** La récurrence médiane (**96** jours) décrit le rythme entre deux achats, la récence médiane (**122** jours) la distance au dernier achat. Pour fixer un délai de campagne, on part de la **récurrence** : elle est stable et ne dépend pas du jour d'observation, contrairement à la récence, qui est une mesure « au 31/08/2026 » et vieillit d'un jour par jour.
4. **La jointure au référentiel.** Elle **retire** le client **0** (absent du référentiel), donc **2 852 612 447** FCFA et **18,3 %** du CA net du périmètre analysé. C'est un choix légitime, à condition d'être écrit : sans cette phrase, le total du rapport ne coïncide plus avec celui du compte de résultat.

E1 (la matrice). Les cohortes de 2023 et 2024 ne se distinguent pas nettement : les taux de M+1 restent dans une fourchette étroite (de **11,0 %** à **19,1 %** sur les six premières). Les cases non observables sont celles des cohortes récentes au-delà de leur recul : **32** cohortes tiennent à M+12, **20** à M+24, et les colonnes lointaines se remplissent de la droite vers la gauche à mesure que le temps passe. Une matrice de rétention se lit toujours **en triangle**, et sa lecture doit nommer la pointe comme « pas encore observable ».

E2 (deux segments). Le plus gros par le montant est **133** : **2 151** clients, **4 937 755 609** FCFA, **31,7 %** du CA — il contient le client **0** et doit être rejoué sans lui avant toute décision. Le tiers des **récents** (dernier achat entre le **18/06/2026** et le **31/08/2026**) porte **48,7 %** du CA, contre **23,1 %** pour le tiers des **anciens** (dernier achat entre le **31/01/2023** et le **13/03/2026**). Politique vérifiable : les récents ont un poids économique immédiat (relance de panier), les anciens un enjeu de réactivation borné par les **4,3 %** du \$5.6.

13. Mini-projet de chapitre

La note de fidélité de la cellule commerciale, 2 pages. À partir du socle, produisez la note de fidélité du parc client au 31/08/2026 : matrice de rétention, rétention pondérée à M+1, M+12 et M+24, répartition RFM, et la question du client non identifié.

Livrables et barème.

Critère	Ce qui est vérifié	Points
La matrice	44 cohortes, taille en première colonne, cases non observables marquées	6
La rétention pondérée	trois anciennetés, avec l'effectif de chaque calcul	5
Le RFM	trois blocs égaux, départage explicite, deux segments nommés avec leurs bornes	4
La note de méthode	définition d'« actif », traitement du client 0, troncature à gauche déclarée	3
Total		18

14. Résumé du chapitre

Mesurer la fidélité, c'est d'abord **choisir une cohorte** : l'événement de départ est la **première commande** — la date d'inscription du référentiel commence en **2016** et ne décrit pas le même

univers. La matrice de rétention se lit en **triangle** : **32** des **44** cohortes ont **12** mois de recul, **20** en ont **24**, et les cases au-delà ne sont pas vides, elles sont impossibles. Trois lectures se ressemblent et ne disent pas la même chose : la **récurrence** calendaire (**13,2 %**, **432** des **3 276** clients actifs en juin 2025), la rétention d'**une** cohorte (**22,7 %**, sur **22** clients) et la rétention **pondérée** (**15,8 %** à M+1, **17,2 %** à M+12, **18,2 %** à M+24) — seule la troisième est publiable. La courbe **plate** décrit un noyau d'acheteurs réguliers, pas une fuite. Le **RFM** répartit les **23 497** clients en trois blocs de **7 833**, **7 832** et **7 832**, place le tiers des récents à **48,7 %** du CA et fait apparaître le segment **133** à **31,7 %** — qui contient le client **0**, **18,3 %** du CA à lui seul. Et la leçon de reproductibilité se mesure : sans départage explicite, NTILE a fait varier le segment principal entre **2 718** et **2 738** clients sur **15** exécutions.

15. À retenir

1. **Une cohorte se date sur l'événement mesuré.** L'inscription n'est pas la première commande.
2. **Un taux sans effectif ne se publie pas.** **22,7 %** et **15,8 %** ne sont pas comparables sans dire qu'ils portent sur **22** et **23 483** clients.
3. **On pondère, on ne moyenne pas.** $\text{SUM}(\text{revenus}) / \text{SUM}(\text{effectif})$, jamais la moyenne des taux.
4. **Une courbe de rétention plate n'est pas une bonne nouvelle : c'est une description.** Le socle ne perd pas ses clients, il en garde un noyau d'environ **16 %**.
5. **Un classement sans départage n'est pas reproductible.** **3 316** clients à **8** tickets suffisent à rendre une frontière de tiers instable.
6. **Le client non identifié décide de la conclusion.** **18,3 %** du CA, **38,1 %** ou **24,2 %** selon qu'on le garde ou non.

À retenir. Le chapitre tient dans une comparaison : **13,2 %** de récurrence calendaire contre **15,8 %** de rétention pondérée. Le premier chiffre est plus facile à produire, plus stable, et il ne répond pas à la question. Le second demande une cohorte, une pondération et une phrase de méthode — et c'est le seul qu'on peut défendre devant un comité.

À retenir. Le socle raconte une entreprise dont **1 848** clients n'ont pas acheté depuis plus d'un an et qui ne représentent que **4,3 %** du chiffre d'affaires : la fidélité se décide sur des ordres de grandeur, et les ordres de grandeur s'obtiennent en pondérant, pas en devinant.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien de cohortes compte le socle, et quelle est la taille de la première et de la dernière ?
2. Pourquoi la matrice de rétention a-t-elle la forme d'un triangle ?
3. Que vaut la rétention pondérée à M+1, et que vaut la moyenne simple des taux sur les cohortes d'au moins 100 clients ?
4. Pourquoi **22,7 %** n'est-il pas le bon chiffre à publier, alors qu'il est juste ?
5. Que mesure la lecture calendaire, si elle ne mesure pas la fidélité ?
6. En combien de blocs NTILE(3) découpe-t-il **23 497** clients, et de quelles tailles ?
7. Quelle part du CA porte le tiers des clients récents ?
8. Pourquoi faut-il une clé de départage dans un NTILE, et quel signe le montre sur ce socle ?
9. Combien de clients n'ont pas acheté depuis plus d'un an, et quelle part du CA représentent-ils ?

10. Quelle est la part du chiffre d'affaires du dixième supérieur des clients, avec et sans le client 0 ?

Réponses. 1. **44** cohortes ; **3 593** clients en janvier 2023, **3** en août 2026. 2. Parce que les cohortes récentes n'ont pas le **recul** nécessaire : **32** cohortes atteignent M+12, **20** atteignent M+24. 3. **15,8 %** pondérée contre **16,4 %** en moyenne simple. 4. Parce qu'il porte sur **22** clients : l'effectif est trop faible pour qu'un taux soit stable. 5. La **récurrence** du fichier actif, toutes cohortes mêlées — **13,2 %** de juin à juillet 2025. 6. Trois blocs : **7 833**, **7 832** et **7 832** clients. 7. **48,7 %** du CA net. 8. Parce que **3 316** clients partagent exactement **8** tickets (**14,1 %** du fichier) : sans clé explicite, le segment principal a varié de **2 718** à **2 738** clients sur **15** exécutions. 9. **1 848** clients, **4,3 %** du CA net. 10. **38,1 %** tous clients confondus, **24,2 %** hors client 0.

Module M11.C05 — Requêtes avancées : CTE, tableau croisé, `UNNEST`, `QUALIFY`, récapitulatifs

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 4 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C04 (fenêtres, cadres, temps, cohortes) ; M07.C05 (jointures) ; M05.C04 (tableau croisé dans un tableur).

L'idée du chapitre. Une requête longue n'est pas une requête compliquée : c'est une requête dont on a nommé les étapes. Ce chapitre présente les six outils qui font passer d'un calcul à un **rapport** : la table d'expression commune (CTE) qui nomme, le **tableau croisé** qui met les mois en colonnes, `QUALIFY` qui classe et filtre dans la même requête, `UNNEST` qui inverse le grain, les **récapitulatifs multi-niveaux** qui rendent les sous-totaux dans une seule passe, et la **réursion**, précieuse pour les hiérarchies et inutile pour les calendriers. Et parce qu'aucun de ces outils ne remplace une donnée propre, le chapitre commence par l'exemple qui le prouve : le référentiel produits du socle écrit ses catégories de **16** façons pour **7** familles réelles, et un classement lancé sur les libellés bruts rend **16** lignes de réponse pour **7** rayons.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). 154 produits, 7 familles après repliement, 5 modes de paiement, 3 canaux, 145 212 tickets, 15 595 154 955 FCFA. Sorties de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Découper une requête longue en étapes nommées** avec `WITH`, et réutiliser la même étape plusieurs fois.
2. **Replier un libellé** sans perdre d'information : casse, espaces, accents, pluriel — et **mesurer** ce que chaque repliement change au classement.
3. **Croiser deux dimensions** avec `PIVOT` et avec l'agrégat conditionnel (`SUM(...)` `FILTER`), en sachant dans quel cas chacun est préférable.
4. **Classer et filtrer dans la même requête** avec `QUALIFY`, sans sous-requête enveloppante.
5. **Inverser un grain** avec `UNNEST`, en redonnant la définition de la ligne obtenue.
6. **Produire un récapitulatif multi-niveaux** (`ROLLUP`, `CUBE`, `GROUPING SETS`) et **distinguer un sous-total d'un zéro** avec `GROUPING()`.
7. **Employer la réursion** là où elle sert, et **écrire la version simple** là où elle ne sert pas.

2. Pourquoi cette notion est importante

Un rapport de direction ne demande pas « le chiffre d'affaires par mode de paiement » : il demande **le tableau** — les modes en colonnes, les années en lignes, les sous-totaux, et une ligne de total. En SQL brut, cette demande produit une requête d'une trentaine de lignes et de trois requêtes collées à la main dans un tableur. Les outils de ce chapitre font ce travail **dans le moteur**, où le contrôle est possible.

Le chapitre porte aussi un avertissement, et il est mesuré. Le référentiel produits du socle contient **154** produits répartis en **16** libellés de catégorie qui n'écrivent que **7** familles réelles :

la casse, les espaces finaux, les accents et le pluriel font le reste. Sur ces libellés bruts, un « meilleur produit par catégorie » rend **16** lignes, dont trois s'appellent Matériaux ; sur les familles repliées, il en rend **7**. Le même catalogue, la même question, deux rapports incompatibles — et c'est le repliement du libellé, une étape nommée dans une CTE, qui décide lequel est juste.

Enfin, ces outils ont un **coût de portabilité** : `QUALIFY` n'existe pas en PostgreSQL, `PIVOT` ne s'écrit pas de la même façon en SQL Server, `UNNEST` se nomme autrement selon les moteurs. Un rapport destiné à d'autres outils doit le savoir — et le dire.

3. Explication simple — la recette, le tableau, l'étagère

La CTE, c'est la recette étapes par étapes. Au lieu d'écrire une seule phrase de quinze lignes, on écrit : « d'abord je prépare la pâte, ensuite je la laisse reposer, enfin je la cuis ». Chaque étape porte un nom, et une étape peut servir deux fois — c'est même son principal intérêt.

Le tableau croisé, c'est le tableau à double entrée du cours de mathématiques. Une liste de 1 000 lignes « mois, mode de paiement, montant » devient un tableau de 4 lignes et 6 colonnes. Rien n'est ajouté, rien n'est perdu : c'est un changement de **forme**, pas de **fond**.

QUALIFY, c'est le tri suivi du filtre, en un geste. On numérote les lignes par famille, et on garde les numéros 1. En SQL classique, il faut deux requêtes imbriquées pour obtenir cela.

UNNEST, c'est renverser une étagère. Un ticket porte une liste d'articles ; `UNNEST` en fait une ligne par article. La base grossit d'un coup — **1,63** produit par ticket en moyenne — et le grain change.

Les récapitulatifs, ce sont les totaux du tableau. Pas en bas du tableau, collés à la main : **dans** la requête, avec `ROLLUP`, et une colonne `GROUPING()` qui dit ce qui est un détail et ce qui est un sous-total.

4. Vocabulaire essentiel

Terme (français)		Équivalent anglais	Définition en une phrase
table d'expression commune		CTE, common table expression	Un bloc <code>WITH nom AS (...)</code> qui donne un nom à une étape intermédiaire.
requête en escalier		chained / staged query	Une requête dont chaque étape est nommée et vérifiable isolément.
repliement d'un libellé		normalisation / folding	Ramener plusieurs graphies d'une même valeur à une seule, par étapes explicites.
famille de produits		product family	Le regroupement le plus large du catalogue, obtenu après repliement du libellé.
tableau croisé		pivot / cross-tab	Une sortie où une dimension passe des lignes aux colonnes.
agrégat conditionnel		conditional aggregate	Un agrégat limité par une condition : <code>SUM(montant) FILTER (WHERE mode = ...)</code> .
liste		list / array	Une valeur qui contient plusieurs valeurs, produite par <code>LIST</code> ou <code>ARRAY_AGG</code> .
éclatement		unnest / explode	Transformer une liste en plusieurs lignes.
récapitulatif multi-niveaux		rollup / cube / grouping sets	Un agrégat qui calcule le détail et les sous-totaux en une seule requête.
sous-total		subtotal	Une ligne qui agrège un sous-ensemble des lignes détaillées.
indicateur de regroupement		<code>grouping()</code>	La fonction qui distingue un sous-total d'une valeur réellement absente.

Terme (français)	Équivalent anglais	Définition en une phrase
réursion	recursive CTE	Une CTE qui se rappelle elle-même pour parcourir une hiérarchie ou une suite.

Définition. Une **table d'expression commune** — CTE — est un bloc `WITH nom AS (requête)` placé avant la requête principale. Elle ne stocke rien : elle **nomme**. Son intérêt n'est pas la performance, mais le fait qu'on peut exécuter chaque étape séparément, la vérifier, et la réutiliser dans plusieurs requêtes. Une requête de rapport lisible est presque toujours une suite de trois à six CTE.

Définition. Le **repliement d'un libellé** consiste à ramener plusieurs graphies d'une même valeur à une seule écriture. Il se fait par étapes **explicites** : casse (`UPPER`), espaces (`TRIM`), accents, et singulier/pluriel. Chaque étape se mesure — sur le socle, **16** libellés deviennent **9** écritures après casse et espaces, puis **7** familles après accents et pluriel. Une étape qu'on ne mesure pas est une étape qui a perdu des lignes sans le dire.

Définition. Un **récapitulatif multi-niveaux** est une agrégation qui produit, dans **une** requête, le détail et les sous-totaux. `ROLLUP (a, b)` rend le détail par couple, les sous-totaux de **a** et le total général ; `CUBE` rend en plus les sous-totaux de **b** seul ; `GROUPING SETS (...)` énumère exactement les niveaux voulus. Le nombre de lignes n'est plus celui du détail : il faut donc **nommer** les lignes de total, et c'est le rôle de `GROUPING()`.

Définition. Le **grain** d'un jeu de lignes est ce que représente **une** ligne. `UNNEST` le change sans prévenir : une ligne « ticket » devient une ligne « article de ticket ». Toute agrégation écrite après un `UNNEST` doit être relue à l'aune de ce nouveau grain — sinon `COUNT(*)` compte des articles en croyant compter des tickets.

5. Cours approfondi

5.1 La CTE : nommer les étapes, et réutiliser la bonne

Voici la forme que prend toute requête de rapport un peu longue :

```
WITH n AS (
    SELECT id_produit,
           CASE WHEN UPPER(TRIM(categorie)) IN ('MATERIAUX', 'MATÉRIAUX') THEN 'MATÉRIAUX'
                WHEN UPPER(TRIM(categorie)) IN ('PEINTURE', 'PEINTURES') THEN 'PEINTURE'
                WHEN UPPER(TRIM(categorie)) = 'ELECTRICITÉ' THEN 'ÉLECTRICITÉ'
                ELSE UPPER(TRIM(categorie)) END AS famille
    FROM produit
),
ca AS (SELECT n.famille, SUM(v.montant_ttc) AS ca
      FROM ventes v JOIN n USING (id_produit)
      WHERE NOT v.est_retour GROUP BY 1)
SELECT famille, ca, ROUND(100.0 * ca / SUM(ca) OVER (), 2) AS part_pct
FROM ca ORDER BY ca DESC;
```

Trois points à retenir. La CTE **n** ne fait qu'une chose : replier le libellé. La CTE **ca** ne fait qu'agréger. La requête finale ne fait qu'ajouter la part, avec une fenêtre sur l'agrégat. Aucune des trois n'est compliquée, et chacune se vérifie seule : `SELECT * FROM n` doit rendre **154** lignes, avec **7** familles distinctes.

C'est la **réutilisation** qui fait la valeur d'une CTE : la même CTE **n** sert ensuite à croiser les familles avec les quartiers, puis avec les années, sans que le repliement soit réécrit — donc sans risque qu'il soit réécrit **différemment**.

5.2 Le tableau croisé : PIVOT et l'agrégat conditionnel

Deux écritures produisent le même tableau. La première est DuckDB :

```
PIVOT (SELECT YEAR(date_vente) AS an, mode_paiement AS mp, montant_ttc AS m
       FROM ventes WHERE NOT est_retour)
ON mp USING SUM(m) GROUP BY an ORDER BY an;
```

an	Chèque	Créance 30 j	Espèces	Mobile Money	Virement
2023	623750816	273920936	1155573169	886824267	503512359
2024	723192393	321227804	1393160843	1047845325	580893385
2025	857090551	374566410	1612526524	1228076360	652440441
2026	593972283	262208583	1156476174	892367109	455529223

La seconde est du SQL de tous les moteurs :

```
SELECT YEAR(date_vente) AS an,
       SUM(montant_ttc) FILTER (WHERE mode_paiement = 'Espèces') AS especes,
       SUM(montant_ttc) FILTER (WHERE mode_paiement = 'Mobile Money') AS mobile,
       SUM(montant_ttc) FILTER (WHERE mode_paiement = 'Chèque') AS cheque
FROM ventes WHERE NOT est_retour GROUP BY 1 ORDER BY 1;
```

Les totaux sont identiques : pour 2023, la somme des cinq colonnes du PIVOT vaut **3 443 581 547 FCFA**, exactement le chiffre d'affaires de l'année. Le PIVOT est donc un **confort**, pas un moteur de calcul — et il a un piège de présentation : il range ses colonnes par ordre **alphabétique**.

Attention. PIVOT trie les colonnes produites par ordre alphabétique : Chèque, Créance 30 j, Espèces, Mobile Money, Virement — alors que l'ordre des données est Espèces, Mobile Money, Chèque, Virement, Créance 30 j. Un SELECT * publié tel quel donne donc un tableau dont l'ordre des colonnes n'est pas celui des libellés. Règle : on **nomme** les colonnes dans l'ordre voulu (SELECT Espèces, "Mobile Money", Cheque, ...) ou on garde l'agrégat conditionnel, dont l'ordre est écrit à la main.

5.3 QUALIFY : classer et filtrer dans la même requête

Le classement par groupe de C02 exigeait deux niveaux de requête : un pour le rang, un pour le filtre. QUALIFY (disponible dans DuckDB, Snowflake, BigQuery, Teradata) n'en exige qu'un :

```
WITH n AS (SELECT id_produit, CASE ... END AS famille FROM produit),
     v AS (SELECT n.famille, p.designation, ROUND(SUM(x.montant_ttc)) AS ca
          FROM ventes x JOIN n USING (id_produit) JOIN produit p USING (id_produit)
          WHERE NOT x.est_retour GROUP BY 1, 2)
SELECT famille, designation, ca FROM v
QUALIFY ROW_NUMBER() OVER (PARTITION BY famille ORDER BY ca DESC, designation) = 1
ORDER BY ca DESC;
```

MATÉRIAUX	Gravier 15/25 (m3) – réf 1	351651755
PLOMBERIE	Lavabo céramique – réf 2	349907568
BOIS & PANNEAUX	Panneau MDF 12 mm – réf 2	255397413
ÉLECTRICITÉ	Lampe solaire murale – réf 2	132272554
QUINCAILLERIE	Pince coupante 200 mm – réf 2	119995326
PEINTURE	Antirouille rouge brique 1 L – réf 2	110331930
CONSOMMABLES	Pelle carrée – réf 1	101865391

7 lignes, une par famille, et le départage est explicite (ca DESC, designation) — sans lui, deux produits à égalité de chiffre d'affaires changeraient de place d'une exécution à l'autre (leçon de C04).

Attention. Le **même** `QUALIFY` écrit sur les libellés bruts de `produit.categorie` rend **16** lignes — pour **7** familles réelles. Trois de ces lignes s'appellent Matériaux (`MATERIAUX`, `Materiaux`, `Matériaux`), et trois autres Quincaillerie. Le tableau est parfaitement exécuté, et il présente trois fois le même rayon : `QUALIFY` ne remplace pas la propreté des données, il la rend seulement plus visible.

5.4 UNNEST : inverser le grain

Un ticket contient une liste d'articles. Pour obtenir une ligne par article — le grain du détail — on éclate la liste :

```
WITH t AS (SELECT v.id_ticket, LIST(DISTINCT p.designation ORDER BY p.designation) AS lp
           FROM ventes v JOIN produit p USING (id_produit)
           WHERE NOT v.est_retour AND v.id_ticket = (SELECT MIN(id_ticket) FROM ventes)
           GROUP BY 1)
SELECT id_ticket, UNNEST(lp) AS designation FROM t;
```

T01-230101-000000 | Mètre ruban 5 m – réf 1

Le premier ticket du socle ne porte qu'une désignation, mais le maximum observé est de **4** produits (**1,63** en moyenne, **1** en médiane sur **145 212** tickets). L'opération est donc asymétrique : elle multiplie la base, et pas d'un facteur constant.

5.5 Les récapitulatifs : ROLLUP, CUBE, GROUPING SETS

Un tableau de direction comporte des sous-totaux. Sans récapitulatif, il faut trois requêtes et un recollage manuel ; avec, une seule :

```
WITH n AS (SELECT id_produit, CASE ... END AS f FROM produit),
b AS (SELECT ma.quartier AS q, n.f, SUM(v.montant_ttc) AS ca
      FROM ventes v JOIN magasin ma USING (id_magasin)
      JOIN n USING (id_produit) WHERE NOT v.est_retour GROUP BY 1, 2)
SELECT COALESCE(q, 'TOTAL') AS quartier, COALESCE(f, 'sous-total') AS famille,
       GROUPING(q) AS gq, GROUPING(f) AS gf, ROUND(SUM(ca)) AS ca
FROM b GROUP BY ROLLUP (q, f) ORDER BY gq, quartier, gf, famille;
```

41 lignes : **35** couples détail, **5** sous-totaux de quartier, **1** total général (**15 595 154 955** FCFA). Les deux colonnes `GROUPING()` disent, ligne par ligne, ce qui est du détail (0) et ce qui est un sous-total (1) — sans elles, un `NULL` de sous-total se confondrait avec une famille non renseignée.

`GROUPING SETS` répond à un autre besoin : quand on veut certains niveaux **sans** les autres. Sur le même socle, `GROUPING SETS ((quartier), (famille), (quartier, famille))` rend **47** lignes — **5** quartiers, **7** familles, **35** couples — chacune à son propre grain.

Dans les faits. Le tableau croisé des modes de paiement rend la période partielle **visible d'un coup d'œil** : les cinq colonnes progressent de 2023 à 2025, puis reculent toutes ensemble en 2026 — qui ne compte que **8** mois (C03, \$5.3). Aucun commentaire n'était nécessaire : la forme du tableau le dit.

5.6 La récursion : pour les hiérarchies, pas pour les calendriers

`WITH RECURSIVE` fait se rappeler une requête elle-même. Le cas d'école — la suite des mois — se résout plus simplement avec `generate_series` :

```
WITH RECURSIVE m AS (SELECT DATE '2023-01-01' AS d
                     UNION ALL SELECT d + INTERVAL 1 MONTH FROM m WHERE d < DATE '2026-08-01')
SELECT COUNT(*) AS mois FROM m;
```

44 — la même série que la table calendrier de C03, en plus verbeux et plus lent. La récursion mérite d’être écrite quand la **profondeur** est inconnue : arborescence d’organisation, nomenclature de produits, chaîne d’approbation. Sur un catalogue à deux niveaux (famille, sous-catégorie) et **29** couples, elle est un luxe ; sur un arbre d’articles imbriqué sur sept niveaux, elle est la seule solution.

5.7 Ce que chaque moteur sait faire

Élément	DuckDB (exécuté)	PostgreSQL (cité)	SQL Server (cité)
WITH (CTE)	oui	oui	oui
WITH RECURSIVE	oui	oui	oui, sans le mot RECURSIVE
QUALIFY	oui	non : sous-requête obligatoire	non : TOP ou ROW_NUMBER
PIVOT	oui, PIVOT (...) ON ... USING ...	non : crosstab (extension) ou agrégat conditionnel	oui, PIVOT (...) FOR ... IN (...)
UNNEST	oui	unnest()	OPENJSON / STRING_SPLIT selon la source
ROLLUP / CUBE / GROUPING SETS	oui	ROLLUP, CUBE, GROUPING SETS	idem, GROUPING() fourni

Attention. Ce tableau distingue trois colonnes qui ne pèsent pas pareil : DuckDB a été **exécuté** pour chaque ligne, PostgreSQL et SQL Server sont **cités** — aucune ligne de cette colonne n’a été jouée dans l’atelier. C’est la règle §1.5 du manuel, et elle vaut pour la syntaxe comme pour les chiffres : un rapport qui publie du code non exécuté le **déclare**.

Conseil professionnel. Écrivez le tableau croisé d’un rapport récurrent en **agrégat conditionnel**, même si votre moteur sait faire PIVOT. Trois raisons : les colonnes y sont nommées dans l’ordre voulu, une période sans donnée y sort à NULL au lieu de disparaître, et le code s’exécute partout. Gardez PIVOT pour l’exploration, là où la liste des colonnes n’est pas connue d’avance.

5.8 Ce que ce chapitre ne couvre pas

La **performance** de ces constructions — coût du PIVOT, effet d’un index, lecture d’un plan d’exécution — est le sujet de C06. Reste ici une règle : aucun de ces outils ne rend une requête rapide ; ils la rendent **lisible**, ce qui est la condition pour pouvoir un jour la rendre rapide.

6. Exemple concret — le tableau que la direction avait demandé

La demande : « le chiffre d’affaires par quartier et par famille, avec les sous-totaux, pour les quatre années ». La réponse tient en deux étapes nommées et un ROLLUP. Ce que la direction reçoit :

- **41** lignes, dont **5** sous-totaux de quartier et **1** total général de **15 595 154 955** FCFA ;
- un détail par couple quartier × famille, où les familles se nomment MATÉRIAUX, PLOMBERIE, ÉLECTRICITÉ, QUINCAILLERIE, BOIS & PANNEAUX, CONSOMMABLES et PEINTURE — **7** familles, pas **16** libellés ;
- et les deux familles de tête à **3 535 582 421** et **3 452 082 064** FCFA, soit **0,5** point d’écart.

Ce dernier point est le plus utile au comité : sur ce catalogue, savoir laquelle de MATÉRIAUX et de PLOMBERIE arrive en tête dépend **du repliement des libellés**. La réponse honnête n'est pas « Matériaux est premier », c'est « les deux sont à égalité, à 0,5 point près ».

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — le repliement mesuré

`SELECT COUNT(DISTINCT categorie) FROM produit` → **16**. Après `UPPER(TRIM(...))` → **9**. Après repliement des accents et du pluriel → **7**. Trois nombres, trois définitions du catalogue.

7.2 Étape 2 — la CTE réutilisée

La CTE `n` du §5.1 sert trois fois : classement par famille, croisement avec les quartiers, croisement avec les années. Une seule définition du repliement, donc un seul classement possible.

7.3 Étape 3 — le tableau croisé

Le `PIVOT` des modes de paiement par année (§5.2) et sa vérification : la somme des cinq colonnes de 2023 égale **3 443 581 547** FCFA, le chiffre d'affaires de l'année. Puis la même table en agrégat conditionnel, pour constater que les colonnes y sont dans l'ordre métier.

7.4 Étape 4 — `QUALIFY` avec et sans repliement

7 lignes avec la CTE de repliement, **16** sans. C'est la démonstration la plus courte du chapitre : un outil puissant posé sur des données sales produit une réponse fausse **plus vite**.

7.5 Étape 5 — `UNNEST` et le grain

Le premier ticket du socle (T01-230101-000000) ne porte qu'une désignation ; le maximum est de **4** produits par ticket. Après éclatement, la base ne compte plus **145 212** tickets mais des lignes d'articles : tout `COUNT(*)` écrit ensuite compte **des articles**.

7.6 Étape 6 — le récapitulatif

41 lignes en `ROLLUP` (35 + 5 + 1), **47** en `GROUPING SETS` (5 + 7 + 35). Le contrôle tient en une requête : le total général du récapitulatif doit être **égal** au chiffre d'affaires net du socle (**15 595 154 955** FCFA). S'il diffère, un `NULL` a été compté comme un zéro, ou un `GROUPING` a été oublié.

8. Erreurs fréquentes

1. **Croire que la CTE accélère.** Elle nomme. Un moteur peut la recalculer plusieurs fois ; c'est un excellent découpage de lecture et pas une optimisation. La performance se mesure (C06).
2. **Publier un `PIVOT` en `SELECT *`.** Les colonnes sortent par ordre alphabétique — Chèque avant Espèces — et le tableau publié n'a plus l'ordre du métier.
3. **Classer sur les libellés bruts.** **16** lignes pour 7 familles : le même rayon apparaît trois fois sous trois graphies (Matériaux, matériaux, MATÉRIAUX).
4. **Agréger après un `UNNEST` sans redéfinir le grain.** Les tickets deviennent des articles (**1,63** par ticket en moyenne) : un `COUNT(*)` compare alors des articles à des tickets.
5. **Confondre un `NULL` de sous-total et une valeur absente.** Sans `GROUPING()`, la ligne de total général ressemble à une ligne dont la dimension est vide — et un lecteur pressé la somme comme un détail.

6. **Recopier une syntaxe non exécutée.** `QUALIFY` en PostgreSQL et `PIVOT ... ON ... USING` en SQL Server n'existent pas : un rapport portable s'écrit en agrégat conditionnel.

9. Bonnes pratiques professionnelles

1. **Une étape nommée par intention.** `n` pour le repliement, `ca` pour l'agrégat, `rg` pour le rang : une CTE qui s'appelle `tmp` empêche la relecture.
2. **Mesurer chaque repliement.** `16 → 9 → 7` : chaque étape publie son compte avant d'être utilisée dans un rapport.
3. **Nommer les colonnes d'un tableau croisé.** Jamais `SELECT *` sur un `PIVOT` ; l'ordre des colonnes fait partie de la donnée.
4. **Écrire le départage de tout classement.** `ORDER BY ca DESC, designation` : la leçon de C04 vaut pour `QUALIFY` comme pour `ROW_NUMBER`.
5. **Vérifier le total des récapitulatifs** contre le total de référence (**15 595 154 955 FCFA**), et déclarer les niveaux calculés. Un sous-total non vérifié est un chiffre de plus à défendre.

10. Exercice guidé

Situation. La direction commerciale veut un tableau unique : les **7** familles en lignes, les **4** années en colonnes, le total par famille à droite, le total par année en bas, et la part de chaque case.

Consigne.

1. Écrivez la CTE de repliement des libellés, et donnez ses trois comptes : libellés bruts, écritures, familles. (2 pts)
2. Produisez le tableau croisé des familles par année, en **agrégat conditionnel**, avec les colonnes dans l'ordre des années. (3 pts)
3. Ajoutez les sous-totaux de ligne et de colonne, et dites par quelle construction ils sont produits. (3 pts)
4. Expliquez, en deux phrases, pourquoi il ne faut **pas** écrire ce tableau en `SELECT *` sur un `PIVOT`. (2 pts)

11. Exercices autonomes

E1 — Trois façons, un seul tableau (40 min). Produisez le croisement quartiers × familles : (a) en `PIVOT`, (b) en agrégat conditionnel, (c) en `GROUPING SETS`. Puis répondez en quatre lignes : quel tableau porte des sous-totaux, lequel a les colonnes dans l'ordre métier, et lequel faudrait-il publier si le rapport partait vers un moteur qui ignore `PIVOT`.

E2 — Le panier du premier ticket, et sa généralisation (35 min). À partir de la CTE qui éclate un ticket en lignes, calculez la composition moyenne d'un ticket par famille de produits. Contrainte : le grain doit être redit — une ligne par article — et le total des articles doit être confronté aux **145 212** tickets du socle.

12. Correction détaillée

Exercice guidé.

1. **Les trois comptes.** **16** libellés bruts, **9** écritures après `UPPER(TRIM(...))`, **7** familles après repliement des accents (`MATÉRIAUX/MATERIAUX`, `ELECTRICITÉ`) et du pluriel (`PEINTURE/PEINTURES`). La CTE écrit ces trois règles une fois pour toutes.

2. **Le tableau.** `SUM(montant_ttc) FILTER (WHERE YEAR(date_vente) = 2023) AS ca_2023`, puis 2024, 2025, 2026, avec `GROUP BY famille` sur la CTE de repliement. Colonnes dans l'ordre chronologique, comme dans le rapport.
3. **Les sous-totaux.** `GROUP BY ROLLUP (famille, annee)` : les cellules, les sous-totaux de famille, les sous-totaux d'année et le total général dans la **même** requête — **41** lignes pour le croisement quartier × famille, et la même structure ici. `GROUPING(famille)` et `GROUPING(annee)` nomment les lignes de total.
4. **Pourquoi pas SELECT *.** Parce que `PIVOT` produit ses colonnes par ordre alphabétique : les années seraient bien ordonnées (2023, 2024...), mais des libellés comme Chèque et Espèces se retrouveraient dans l'ordre inverse de celui du métier — et le tableau publié serait, à la lettre, un autre tableau.

E1 (trois façons). (a) Le `PIVOT` est le plus court à écrire et le moins contrôlable : colonnes alphabétiques, aucune notion de sous-total. (b) L'agrégat conditionnel donne l'ordre métier des colonnes et fonctionne partout, mais les sous-totaux demandent un `ROLLUP` supplémentaire ou une requête d'union. (c) Le `GROUPING SETS` est le seul qui produit les trois niveaux en une passe, au prix d'une lecture plus difficile. Pour un rapport qui part vers un moteur sans `PIVOT` : l'agrégat conditionnel, éventuellement enveloppé dans un `GROUPING SETS`.

E2 (le panier éclaté). Après éclatement, la base compte les lignes d'articles, pas les tickets : le nombre de lignes dépasse nécessairement la médiane de **1** produit par ticket, et le total des articles doit être comparé aux **145 212** tickets — jamais confondu avec eux. La composition moyenne par famille se lit alors par ticket et non par ligne d'article : le dénominateur est le nombre de **tickets**, pas le nombre de lignes éclatées.

13. Mini-projet de chapitre

Le tableau de bord d'une famille de produits, 2 pages. Choisissez une famille du catalogue et produisez son tableau de bord : ventes par année, par quartier et par mode de paiement, avec sous-totaux et parts, plus le top 3 des désignations.

Livrables et barème.

Critère	Ce qui est vérifié	Points
La CTE de repliement	les trois comptes (16 / 9 / 7) publiés, une seule définition	5
Le tableau croisé	deux dimensions croisées, colonnes nommées dans l'ordre métier	5
Les sous-totaux	produits par <code>ROLLUP</code> ou <code>GROUPING SETS</code> , <code>GROUPING()</code> utilisé, total vérifié	5
Le top 3	<code>QUALIFY</code> avec départage explicite, une ligne par désignation	3
Total		18

14. Résumé du chapitre

Six outils, une même idée : rendre une requête longue **lisible et vérifiable**. La **CTE** nomme les étapes et se réutilise — c'est elle qui porte le repliement des libellés (**16** → **9** → **7**). Le **tableau croisé** change la forme sans changer le fond, et l'agrégat conditionnel le fait partout, avec des colonnes dans l'ordre métier — là où `PIVOT` trie alphabétiquement. `QUALIFY` classe et filtre en une passe : **7** lignes sur les familles repliées, **16** sur les libellés bruts, et c'est la donnée qui décide, pas l'outil. `UNNEST` inverse le grain (**1,63** produit par ticket en moyenne, **4** au maximum) et oblige à redire ce qu'est une ligne. Les **récapitulatifs** rendent le détail et les sous-totaux ensemble — **41** lignes en `ROLLUP`, **47** en `GROUPING SETS` — à condition que `GROUPING()` distingue un total d'un `NULL`. Et la **réursion** sert les hiérarchies : pour les **44** mois du socle, `generate_series` fait le travail en une ligne.

15. À retenir

1. **Une requête longue se découpe en étapes nommées.** Une étape qu'on ne peut pas exécuter seule ne peut pas être vérifiée.
2. **Un repliement de libellé se mesure à chaque étape. 16 libellés, 9 écritures, 7 familles :** trois nombres à publier avant tout classement.
3. **PIVOT range ses colonnes par ordre alphabétique.** Le tableau publié doit nommer ses colonnes.
4. **QUALIFY ne nettoie pas les données, il les rend visibles. 16 lignes pour 7 rayons.**
5. **UNNEST change le grain.** Après éclatement, la ligne est un article, plus un ticket.
6. **GROUPING() sépare un sous-total d'une absence.** Sans lui, un total général se lit comme un détail.
7. **La récursion est faite pour les hiérarchies, pas pour égrener 44 mois.**

À retenir. Le chiffre qui résume le chapitre n'est pas un total : c'est la paire **16 / 7**. Seize façons d'écrire sept rayons, et un outil avancé qui publie seize lignes sans broncher. Le nettoyage des libellés n'est pas une étape préparatoire qu'on repousse : c'est la première CTE du rapport.

À retenir. Les deux familles de tête du catalogue sont à **0,5** point l'une de l'autre (**3 535 582 421** et **3 452 082 064** FCFA). Un rapport qui annonce « Matériaux est notre première famille » énonce donc moins une vérité qu'une conséquence de son propre repliement.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien de libellés de catégorie compte le référentiel produits, combien d'écritures après casse et espaces, combien de familles après accents et pluriel ?
2. Que produit un « meilleur produit par catégorie » écrit sur les libellés bruts, et pourquoi est-ce un problème ?
3. Dans quel ordre PIVOT range-t-il ses colonnes, et pourquoi est-ce gênant pour un rapport ?
4. Quelle écriture du tableau croisé fonctionne dans tous les moteurs, et pourquoi ?
5. Que change UNNEST au grain d'une table, et quel chiffre du socle le rappelle ?
6. Combien de lignes rend ROLLUP (quartier, famille), et comment se décomposent-elles ?
7. À quoi sert GROUPING(), et que se passe-t-il si on l'oublie ?
8. Combien de lignes rend GROUPING SETS ((quartier), (famille), (quartier, famille)), et de quoi sont-elles faites ?
9. Quand la récursion est-elle préférable à generate_series, et pourquoi ?
10. Quelles sont les deux familles de tête du catalogue, et de combien d'écart ?

Réponses. 1. **16** libellés, **9** écritures, **7** familles. 2. **16** lignes, dont trois graphies du même rayon : le classement dépend du libellé et non du produit. 3. Par ordre **alphabétique** (Chèque, Créance 30 j., Espèces...), alors que l'ordre métier est Espèces, Mobile Money, Chèque, Virement, Créance 30 j. 4. **L'agrégat conditionnel** (SUM(...) FILTER (WHERE ...)) : colonnes nommées dans l'ordre voulu, NULL préservé, et syntaxe portable. 5. Il fait passer d'une ligne par ticket à une ligne par article — **1,63** produit par ticket en moyenne, **4** au maximum. 6. **41** lignes : **35** couples, **5** sous-totaux de quartier, **1** total général. 7. À distinguer un sous-total d'une valeur absente ; sans lui, la ligne de total général se lit comme une ligne de détail. 8. **47** lignes : **5** quartiers, **7** familles, **35** couples. 9. Quand la profondeur de la hiérarchie n'est pas connue d'avance — un arbre d'organisation, pas une suite de **44** mois. 10. **MATÉRIAUX** (**3 535 582 421** FCFA) et **PLOMBERIE** (**3 452 082 064** FCFA), à **0,5** point près.

Module M11.C06 — Le rapport lent : vues, index, plans d'exécution, `SELECT *`, partitions

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 5 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C05 (fenêtres, cohortes, requêtes avancées) ; M07.C06 (volumétrie) ; M09.C05 (lecture d'un plan d'exécution).

L'idée du chapitre. Un rapport juste et lent n'est pas un rapport. Sur le socle du module, la même question — « combien de lignes de retour ? » — coûte **142 ms** posée sur la vue et **1 ms** posée sur la même table recopiée en mémoire : le rapport entre les deux est **×142**, pour un résultat identique au chiffre près. Plus frappant encore, `SELECT *` coûte **581 ms** là où trois colonnes nommées coûtent **96 ms (×6,1)** : le plan annonce **18** colonnes lues contre **3**, et l'horloge confirme. Ce chapitre apprend à **lire** ces nombres avant de les subir : ce qu'une vue relit, ce qu'un `SELECT *` transporte, ce qu'un index change vraiment (mesuré : presque rien, en DuckDB), comment un plan se lit de bas en haut, et pourquoi une mesure de performance se **fige** dans un fichier daté au lieu de se relire à l'horloge.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (03_exercices/dossier_M11/socle_m11.sql). Les temps cités sont des **médianes de 5 exécutions**, produites par `tools/perf_M11.py` et **gelées** le **24/09/2026** dans `03_exercices/dossier_M11/PERF_M11.json`. Le chapitre ne relit jamais l'horloge : il relit ce fichier.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Distinguer une vue d'une table** dans ses conséquences pratiques, et dire ce qu'une vue **relit** à chaque appel.
2. **Mesurer un protocole** : médiane de plusieurs exécutions, instrument nommé, valeurs **figées et datées**.
3. **Chiffrer le coût d'un `SELECT *`** à l'horloge **et** dans le plan d'exécution, sur une table et sur une vue.
4. **Lire un plan d'exécution** : les étages, leur ordre, le nombre de colonnes lues, et ce qu'un `SEQ_SCAN` annonce du travail à venir.
5. **Décider de l'utilité d'un index sur une mesure**, pas par réflexe — et savoir pourquoi un même index change tout dans un moteur et rien dans un autre.
6. **Arbitrer entre lisibilité et performance** quand les deux se contredisent (§5.5).
7. **Écrire la note de méthode d'un rapport** : périmètre, protocole, valeurs gelées, ce qui n'a pas été mesuré.

2. Pourquoi cette notion est importante

Le module M11 produit **12** rapports. Chacun d'eux lit le même socle de **240 000** lignes de ventes. Si le rapport type met **841 ms** au lieu de **277 ms** — c'est-à-dire écrit sur la vue et en `SELECT *` au lieu de la table et de trois colonnes — l'écart paraît dérisoire. Multiplié par douze rap-

ports, par les rafraîchissements quotidiens et par les relances de chaque utilisateur, il devient une salle d'attente.

Mais le vrai enjeu du chapitre n'est pas la vitesse : c'est la **confiance**. Un rapport dont la note de méthode dit « médiane de 5 exécutions, DuckDB 1.5.5, valeurs figées le 24/09/2026 dans PERF_M11.json » est un rapport **reproductible**. Un rapport dont les temps ont été copiés une fois, sans protocole, est un rapport qu'on ne peut ni vérifier ni défendre. La performance, en BI, se **documente** avant de s'optimiser.

Enfin, le chapitre désarme un réflexe coûteux : sur ce socle, ajouter un index fait passer une recherche par client de **0,6 ms** à **0,5 ms** — et le plan continue d'annoncer **SEQ_SCAN**. L'index n'est pas une bonne pratique à appliquer partout : c'est une réponse à un problème que le **plan** doit avoir montré.

3. Explication simple — le classeur, le sommaire et le chariot

La vue, c'est un raccourci vers le classeur. Chaque fois qu'on ouvre le raccourci, il faut ressortir le classeur et le relire. La table, c'est le tableau recopié sur le bureau : on l'a payé une fois (la copie) et on le consulte gratuitement ensuite. Sur le socle : **142 ms** à chaque appel de la vue, **1 ms** pour la table équivalente.

Le `SELECT *`, c'est le chariot trop large. Demander toutes les colonnes d'une table large revient à sortir du magasin les **18** rayons pour n'utiliser que **3** produits. Le plan le dit avant l'exécution : **18** colonnes lues au lieu de **3**, et l'horloge confirme (**581 ms** contre **96**).

Le plan d'exécution, c'est la fiche de préparation du cuisinier. Avant de cuisiner, il annonce : « je vais chercher les fichiers, les joindre, les regrouper, projeter les colonnes ». Trois étages sur le socle (`HASH_GROUP_BY`, `PROJECTION`, `HASH_JOIN`), lus **de bas en haut**. On ne discute pas une performance sans cette fiche.

L'index, c'est l'index alphabétique d'un dictionnaire. Utile pour trouver **une** page précise, inutile pour lire le dictionnaire entier. Sur DuckDB — un moteur en colonnes — la recherche d'un client reste un **SEQ_SCAN** : le moteur lit la colonne entière, et cela lui coûte moins que de naviguer dans un index. Sur SQLite — un moteur en lignes — le même index **remplace** le balayage complet : le plan passe de « `SCAN ventes_t` » à « `SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)` ».

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
vue	view	Une requête nommée ; elle ne contient pas de données et relit sa source à chaque appel.
matérialisation	materialization	Le fait de copier le résultat d'une vue dans une table, pour ne le calculer qu'une fois.
table en mémoire	in-memory table	Une table dont les données sont copiées dans la mémoire du moteur : rapide, mais rien n'est persisté.
plan d'exécution	execution plan	Le programme que le moteur se propose d'exécuter : les opérateurs, leur ordre et les colonnes lues.
étage	operator / stage	Un opérateur du plan : <code>SEQ_SCAN</code> , <code>HASH_JOIN</code> , <code>HASH_GROUP_BY</code> , <code>PROJECTION</code> , <code>ORDER_BY</code> .
balayage séquentiel	sequential scan	Lire toute une colonne, ligne après ligne : le mode normal d'un moteur en colonnes.

Terme (français)	Équivalent anglais	Définition en une phrase
index	index	Une structure de recherche par valeur : indispensable en moteur en lignes, souvent ignorée en colonnes.
projection	projection	La liste des colonnes réellement lues et transmises aux étages suivants.
coût du SELECT *	star-select overhead	Le travail de lecture de colonnes inutilisées, visible dans la projection du plan.
médiane de 5	median of 5 runs	Le protocole de mesure retenu : cinq exécutions, on publie la valeur médiane.
valeur gelée	frozen measurement	Un temps machine écrit dans un fichier daté, que la prose relit au lieu de relancer l'horloge.
partition	partition	Un découpage physique des données (par date, par magasin) qui permet de ne pas lire ce dont on n'a pas besoin.

Définition. Le **plan d'exécution** est la description, par le moteur, de ce qu'il va faire : les opérateurs, leur ordre d'enchaînement et les colonnes lues. Il se demande par **EXPLAIN avant** l'exécution — et c'est ce qui en fait l'outil d'un analyste : il annonce le coût sans faire le travail. Sur le socle, le croisement magasin / chiffre d'affaires se prépare en **3** étages : **HASH_GROUP_BY**, **PROJECTION**, **HASH_JOIN**.

Définition. Une **vue** ne stocke pas de données : c'est une requête à laquelle on a donné un nom. Sa conséquence pratique est immense et souvent ignorée : **chaque appel relit la source**. Sur le socle, la vue `ventes` relit le CSV des ventes ; compter les retours coûte **142** ms sur la vue contre **1** ms sur la table équivalente — soit **×142** pour un résultat identique.

Définition. Un **index** est une structure de recherche par valeur, qui évite de parcourir toutes les lignes. Il n'est **pas** universel : son utilité dépend du moteur (lignes ou colonnes), de la requête (recherche ponctuelle ou agrégat) et de la sélectivité. La seule façon honnête de trancher est de lire le plan et de mesurer : sur ce socle, l'index ne change ni le plan de DuckDB ni ses temps.

Définition. Le **coût du SELECT *** est le travail de lecture des colonnes qui ne serviront pas à la sortie. Il se lit dans la **projection** du plan — **18** colonnes lues pour un **SELECT ***, **3** pour trois colonnes nommées — et se vérifie à l'horloge (**581** ms contre **96** ms). C'est le gaspillage le plus fréquent des rapports, et le plus simple à corriger.

5. Cours approfondi

5.1 La vue : ce qu'elle relit, à chaque fois

Le socle du module est une suite de vues posées sur des CSV. C'est un choix assumé (aucun fichier de base à versionner, voir §5.6), mais il a un prix :

```
-- sur la vue : le CSV est relu à chaque appel
SELECT COUNT(*) FROM ventes WHERE est_retour;
-- sur la table équivalente, copiée en mémoire
CREATE OR REPLACE TABLE ventes_m AS SELECT * FROM ventes;
SELECT COUNT(*) FROM ventes_m WHERE est_retour;
```

Mesure	Sans matérialisation	Avec matérialisation	Rapport
Comptage des retours	142 ms (vue)	1 ms (table)	×142
SELECT *	841 ms (vue)	581 ms (table)	×1,4
3 colonnes nommées	277 ms (vue)	96 ms (table)	×2,9

La dernière ligne mérite attention : sur la **vue**, `SELECT *` coûte **×3,0** de plus que trois colonnes (**841** contre **277**) ; sur la **table**, l'écart est de **×6,1** (**581** contre **96**). Le gaspillage des colonnes est donc plus visible quand la lecture de fichier ne le masque pas.

5.2 `SELECT *` : le lire dans le plan, pas seulement au chronomètre

Deux requêtes, un seul chiffre d'affaires attendu :

```
EXPLAIN SELECT * FROM ventes_m;
EXPLAIN SELECT date_vente, id_magasin, montant_ttc FROM ventes_m;
```

Le premier plan annonce **18** colonnes lues, le second **3**. Le rapport est de **×6** — et il est **écrit dans le plan**, vérifiable sans exécuter. À l'horloge : **581** ms contre **96** ms, soit **×6,1**. Le plan avait raison avant la mesure, et c'est exactement l'intérêt de le lire : on peut refuser une requête avant de payer son temps.

Dans les faits. Le plan annonce **×6** de colonnes, l'horloge mesure **×6,1**. C'est la démonstration la plus utile du chapitre pour un débutant : la lecture d'un plan n'est pas un rituel de spécialiste, c'est un **devis** — proposé gratuitement par le moteur avant chaque requête.

5.3 Lire un plan : trois étages, de bas en haut

```
EXPLAIN SELECT m.nom, SUM(v.montant_ttc)
FROM ventes_m v JOIN magasin m USING (id_magasin) GROUP BY 1;
```

```
HASH_GROUP_BY
PROJECTION
HASH_JOIN
```

3 étages : le moteur joint les ventes et les magasins (`HASH_JOIN`), projette les colonnes utiles (`PROJECTION`), puis regroupe (`HASH_GROUP_BY`). La lecture se fait **de bas en haut** : les données remontent l'arbre, chaque étage transmettant son résultat à celui du dessus. Trois questions suffisent pour auditer un plan :

1. **Combien de colonnes** chaque lecture annonce-t-elle ? (les projections, §5.2)
2. **Quel opérateur domine** ? Un `SEQ_SCAN` sur une table large précède presque toujours un plan lent.
3. **Y a-t-il un étage inutile** ? Une jointure ou un tri qui ne servent à aucune réponse du rapport.

5.4 L'index : mesuré, puis contrôlé dans un second moteur

Sur le socle matérialisé, on cherche un client précis et on agrège par magasin :

Requête	Sans index	Avec index	Effet
Recherche par client (15676)	0,6 ms	0,5 ms	aucun effet mesurable
Agrégat par magasin	1,5 ms	1,5 ms	aucun effet
Plan de la recherche	<code>SEQ_SCAN</code>	<code>SEQ_SCAN</code>	l'index n'est pas utilisé

DuckDB est un moteur **en colonnes** : lire une colonne entière de **240 000** valeurs compactées coûte moins que parcourir un index. Le contrôle croisé dans un moteur **en lignes** renverse la conclusion :

```
SQLite, 240 000 lignes chargées en mémoire
sans index : SCAN ventes_t
avec index : SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)
réponse    : 14 lignes et 5 578 688 FCFA pour le client 15676
```

Le même index, la même requête, le même résultat — et un plan qui change du tout au tout. La leçon n'est pas « l'index est inutile », elle est « l'index est une réponse à un plan » : on regarde le plan, on voit un balayage complet qui coûte, on indexe, on remesure.

Attention. Une colonne indexée dans un moteur en lignes peut ralentir les **écritures** : chaque insertion met l'index à jour. Sur un rapport de lecture, l'arbitrage est simple ; sur une table alimentée en continu, il se mesure des deux côtés — et se documente.

5.5 Lisibilité contre performance : le cas de la fenêtre

Toutes les écritures lisibles ne sont pas les plus rapides. Le module l'a mesuré sur la question la plus fréquente de C01 — la part d'une ligne dans son ensemble :

Écriture	Temps	Avantage
Fonction de fenêtre (SUM(...) OVER ())	150 ms	une seule requête, une seule passe à écrire, robuste au filtre ajouté
Sous-requête scalaire	292 ms	aucune

La fenêtre est **×1,9** plus rapide **et** plus lisible : ici, les deux critères vont dans le même sens. Ce n'est pas toujours le cas — une CTE bien nommée peut être recalculée plusieurs fois. La règle du chapitre n'est donc pas « préférer le plus lisible » : c'est **mesurer avant de choisir**, puis écrire le choix.

Attention. Un temps machine n'est pas reproductible au bit près : le premier appel paie la lecture du fichier, les suivants non, et la machine hôte varie. C'est pourquoi le protocole est **publié** (médiane de 5 exécutions) et pourquoi les valeurs sont **gelées** dans un fichier daté — `PERF_M11.json`, figé le **24/09/2026** par `tools/perf_M11.py --figer`. Un rapport qui relit l'horloge à chaque exécution publie un chiffre différent chaque matin, et ne peut plus être comparé à lui-même.

5.6 Matérialiser ou non : le plancher d'un fichier

On pourrait écrire le socle matérialisé sur le disque une fois pour toutes. Le coût de ce confort est mesurable : un fichier DuckDB qui ne contient **qu'une** table de **154** lignes pèse **536 576** octets. Ce plancher fixe interdit de versionner la base du module — c'est pour cela que le socle M11 est un **script** (`socle_m11.sql`) qui reconstruit les vues et les tables de référence en une fraction de seconde, et que les démonstrations de volume se font **en mémoire** (`ouvrir(materialiser=True)` dans `03_exercices/dossier_M11/connexion.py`).

Conseil professionnel. Mesurez **avant** d'optimiser, et écrivez la mesure **dans** le rapport. Une note de trois lignes — « ce rapport lit la vue (donc le CSV) et dix-huit colonnes ; en nommant trois colonnes il passe de **581** ms à **96** ms ; l'index n'apporte rien ici » — vaut mieux qu'un rapport rapide et muet. Six mois plus tard, c'est cette note qui empêchera un collègue d'ajouter un index inutile ou de croire que le rapport était lent « pour de bonnes raisons ».

5.7 Les partitions

Le **partitionnement** découpe physiquement les données — par année, par magasin — pour qu'une requête ne lise que les partitions concernées. Trois familles d'usage : le partitionnement **Hive** sur des fichiers (dossier `annee=2026/mois=08/`), les tables partitionnées de PostgreSQL et SQL Server, et les filtres de partition dans les moteurs analytiques.

Attention. Le partitionnement est **cité, non exécuté** dans ce chapitre : le socle de l'atelier n'a ni le volume ni le disque pour qu'une mesure soit interprétable. Aucun plan de partition n'est donc publié ici — un chapitre qui montre un plan qu'il n'a pas lu fabrique une preuve, ce que ce manuel interdit depuis le premier module (§1.5).

5.8 La note de méthode d'un rapport de performance

Tout rapport de performance doit pouvoir répondre à cinq questions, et les cinq tiennent en quatre lignes :

1. **Quel moteur et quelle version ?** DuckDB 1.5.5, sur l'atelier.
2. **Quel protocole ?** Médiane de 5 exécutions, requête complète, résultat consommé.
3. **Quand ?** Le 24/09/2026.
4. **Où sont les valeurs ?** `03_exercices/dossier_M11/PERF_M11.json`, produit par `tools/perf_M11.py`.
5. **Qu'est-ce qui n'a pas été mesuré ?** Les partitions, PostgreSQL et SQL Server — cités, non exécutés.

6. Exemple concret — la facture d'un rapport mal écrit

Un rapport quotidien lit le socle et publie le chiffre d'affaires par magasin avec sa part. Deux écritures, mesurées :

Version	Écriture	Temps	Sur 12 rapports × 3 exécutions
A	vue + <code>SELECT *</code> + sous-requête scalaire	≈ 841 ms + 292 ms	≈ 41 s
B	table matérialisée + 3 colonnes + fenêtre	≈ 96 ms + 150 ms	≈ 9 s

Le même contenu, la même justesse, et un facteur 4,5 — sans changer une seule ligne de logique métier, seulement la **forme** de la lecture. C'est le résultat le plus utile du chapitre : la première optimisation d'un rapport n'est presque jamais un index, c'est de **lire ce qu'on utilise**.

7. Démonstration pas à pas — six étapes sur le socle

7.1 Étape 1 — le protocole d'abord

Cinq exécutions, on publie la médiane, et l'instrument est nommé : `tools/perf_M11.py`. Sans protocole, un temps ne vaut rien.

7.2 Étape 2 — la vue contre la table

142 ms contre 1 ms (×142) pour compter les retours. La vue relit le CSV ; la table est en mémoire.

7.3 Étape 3 — le plan du `SELECT *`

18 colonnes annoncées contre **3**, **×6** dans le plan et **×6,1** à l'horloge (**581** ms contre **96** ms). Le devis du moteur suffisait à trancher.

7.4 Étape 4 — l'index, mesuré puis contrôlé

0,6 ms contre **0,5** ms, agrégat inchangé, plan toujours `SEQ_SCAN` : en DuckDB, aucun effet. Le contrôle croisé SQLite montre un plan qui passe de `SCAN ventes_t` à `SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)` — et le même résultat, **14** lignes et **5 578 688** FCFA pour le client **15676**.

7.5 Étape 5 — fenêtre contre sous-requête

150 ms contre **292** ms (**×1,9**) pour la même part : ici, la forme la plus lisible est aussi la plus rapide.

7.6 Étape 6 — le plancher de matérialisation

536 576 octets pour une table de **154** lignes : rien ne justifie de versionner une base là où un script de reconstruction suffit.

8. Erreurs fréquentes

1. **Optimiser sans mesurer.** Ajouter un index par réflexe n'a **aucun** effet mesurable sur ce socle (**0,6** ms contre **0,5** ms) et le plan reste `SEQ_SCAN` : le travail n'est pas là où on le croyait. C'est mesurer d'abord, écrire ensuite.
2. **Publier un temps sans protocole.** Un chiffre isolé, sans version de moteur, sans nombre d'exécutions et sans date, n'est pas une mesure : c'est une anecdote. Le protocole tient en une ligne (**médiane de 5**, DuckDB 1.5.5, **24/09/2026**, `PERF_M11.json`).
3. **Laisser `SELECT *` dans un rapport.** **581** ms contre **96** ms, **18** colonnes contre **3** : le gaspillage est dans le plan, donc évitable sans exécuter.
4. **Confondre vue et table dans l'estimation d'un coût.** Le même comptage vaut **142** ms ou **1** ms selon la source : annoncer « ce rapport coûte 1 ms » en l'ayant mesuré sur la table, pour un rapport qui lit la vue, est un chiffre faux.
5. **Recopier les temps d'un autre moteur.** Un index qui transforme un plan SQLite (`SEARCH`) ne change rien à un plan DuckDB (`SEQ_SCAN`) : les conclusions ne se transportent pas.
6. **Relire l'horloge dans un rapport.** Deux exécutions doivent donner le même document ; un temps mesuré « en direct » fait varier le rapport d'un jour à l'autre sans qu'aucune donnée n'ait changé.

9. Bonnes pratiques professionnelles

1. **Écrire le protocole avant le chiffre.** Médiane de **5** exécutions, version du moteur, date, instrument nommé : la même phrase que celle du fichier `PERF_M11.json`.
2. **Geler les valeurs, dater le fichier.** `tools/perf_M11.py --figer` écrit le JSON ; la prose le relit. C'est ce qui rend deux relevés identiques sur une machine plus lente.
3. **Lire le plan avant d'optimiser.** Le devis est gratuit : **18** contre **3** colonnes annoncées, pour **581** contre **96** ms mesurées.
4. **Nommer ses colonnes.** `SELECT date_vente, id_magasin, montant_ttc` plutôt que `SELECT *` : même réponse, **×6** de travail en moins.

5. **Déclarer ce qui n'a pas été mesuré.** Les partitions, PostgreSQL et SQL Server sont **cités** ici ; c'est écrit dans la note de méthode et dans le corps du chapitre.

10. Exercice guidé

Situation. Un rapport de suivi met **841** ms sur la machine de l'analyste. Le collègue en conclut : « il faut indexer la table des ventes ».

Consigne.

1. Reproduisez la mesure et donnez le protocole complet (moteur, nombre d'exécutions, date, instrument). (2 pts)
2. Montrez, plan à l'appui, ce que la requête lit réellement, et quelle correction simple divise son coût. (3 pts)
3. Mesurez l'effet de l'index proposé par le collègue, et rédigez la réponse à lui faire en deux phrases. (3 pts)
4. Ajoutez la ligne de note de méthode que ce rapport doit porter. (2 pts)

11. Exercices autonomes

E1 — La même question, deux sources (40 min). Mesurez le comptage des retours sur la vue puis sur la table matérialisée, cinq fois chacune, et publiez la médiane. Puis répondez en quatre lignes : de combien le rapport est-il plus rapide, et que faut-il écrire dans la note de méthode pour que ce gain soit vérifiable par quelqu'un d'autre.

E2 — L'index, dans les deux moteurs (40 min). Reproduisez la mesure d'index en DuckDB, puis le contrôle croisé en SQLite : relevez les deux plans et le résultat identique. Terminez par trois conseils destinés à un analyste qui doit décider d'indexer — trois phrases, une par critère (moteur, requête, écriture).

12. Correction détaillée

Exercice guidé.

1. **La mesure.** Comptage des retours sur la vue : médiane de **5** exécutions, DuckDB 1.5.5, atelier, **24/09/2026**, instrument `tools/perf_M11.py` — **841** ms si la requête prend aussi `SELECT *`, et **142** ms pour le seul comptage sur la vue.
2. **Le plan.** `EXPLAIN SELECT * FROM ventes_m` annonce **18** colonnes lues, contre **3** pour la version nommée : la correction est d'**écrire les colonnes**, ce qui ramène la mesure de **581** ms à **96** ms (**×6,1**).
3. **L'index.** Sans index **0,6** ms, avec index **0,5** ms, agrégat **1,5** ms dans les deux cas, et le plan reste un `SEQ_SCAN` : « l'index n'est pas le problème : le rapport relit la vue et lit dix-huit colonnes pour en utiliser trois ; corrigeons la lecture, puis remesurons. »
4. **La note de méthode.** « Médiane de **5** exécutions, DuckDB 1.5.5, atelier, valeurs figées le **24/09/2026** dans `03_exercices/dossier_M11/PERF_M11.json` ; partitions, PostgreSQL et SQL Server cités, non exécutés. »

E1 (deux sources). **142** ms contre **1** ms, soit **×142** : le gain est réel mais **conditionnel** — il suppose que la table soit déjà matérialisée, ce qui a un coût (mémoire) et une durée de vie (le temps de la session). La note de méthode doit donc dire **quand** la table est créée et **ce qui se passe** si elle ne l'est pas : un gain non déclaré se transforme en régression chez le collègue qui exécute le rapport sans matérialisation.

E2 (deux moteurs). Trois critères : le **moteur** (DuckDB en colonnes ignore l'index, SQLite en lignes l'utilise) ; la **requête** (recherche ponctuelle ou agrégat complet : un agrégat ne profite pas d'un index) ; l'**écriture** (chaque index ralentit les insertions — à mesurer sur une table alimentée). Le contrôle croisé donne le même résultat dans les deux moteurs — **14** lignes et **5 578 688** FCFA pour le client **15676** — ce qui est la moindre des choses : un index ne change pas la réponse, il change le **chemin**.

13. Mini-projet de chapitre

La note de performance d'un rapport, 2 pages. Choisissez l'un des **12** rapports du projet de module. Mesurez-le sur la vue et sur la table, avec et sans `SELECT *`, et produisez sa note de performance.

Livrables et barème.

Critère	Ce qui est vérifié	Points
Le protocole	médiane de 5 exécutions, moteur et version, date, instrument nommé	5
Les mesures	deux sources × deux façons d'écrire la projection, valeurs publiées	5
Le plan	lecture commentée du plan de la requête (étages, colonnes lues)	4
La note de méthode	ce qui a été mesuré, ce qui ne l'a pas été, et la décision d'optimisation	4
Total		18

14. Résumé du chapitre

La performance d'un rapport se **lit** avant de se mesurer, et se **mesure** avant de s'optimiser. Sur le socle du module : une vue relit sa source et coûte **142** ms là où la table équivalente coûte **1** ms (**×142**) ; `SELECT *` lit **18** colonnes au lieu de **3**, ce que le plan annonce (**×6**) et que l'horloge confirme (**581** ms contre **96** ms, **×6,1**) ; un index ne change **rien** en DuckDB (**0,6** ms contre **0,5** ms, plan toujours `SEQ_SCAN`) mais transforme le plan SQLite en `SEARCH`, pour la même réponse (**14** lignes, **5 578 688** FCFA) ; une fenêtre coûte **150** ms là où la sous-requête scalaire en coûte **292** (**×1,9**), alliant ici lisibilité et vitesse ; et matérialiser a un plancher (**536 576** octets pour **154** lignes), ce qui justifie un socle en script plutôt qu'en fichier. Enfin, aucun de ces chiffres ne serait publiable sans protocole : médiane de **5** exécutions, DuckDB 1.5.5, valeurs gelées le **24/09/2026** dans `PERF_M11.json`.

15. À retenir

1. **Une vue relit sa source à chaque appel.** **142** ms contre **1** ms pour le même comptage.
2. **Le `SELECT *` se voit dans le plan avant de coûter.** **18** colonnes annoncées contre **3**.
3. **Le plan se lit de bas en haut**, et trois questions suffisent : colonnes lues, opérateur dominant, étage inutile. Sur le socle : **3** étages.
4. **L'index est une réponse à un plan, pas une habitude.** Aucun effet mesurable en DuckDB ; plan transformé en SQLite.
5. **Un temps sans protocole n'est pas une mesure.** Médiane de **5** exécutions, moteur, date, instrument.
6. **Les valeurs de performance se gèlent dans un fichier daté.** Deux exécutions du relevé doivent donner le même document.
7. **La première optimisation est de lire ce qu'on utilise** — pas d'ajouter un index.

À retenir. Le chiffre qui résume le chapitre est un écart entre deux façons d'écrire la même question : **×6,1**. Rien n'a changé dans le métier — seul le nombre de colonnes lues a changé. C'est l'optimisation la moins coûteuse et la plus souvent oubliée.

À retenir. Le protocole fait partie du résultat. « **0,5 ms** » sans protocole ne vaut rien ; « médiane de **5** exécutions, DuckDB 1.5.5, **24/09/2026**, PERF_M11.json » se vérifie, se compare et se défend.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Combien coûte le comptage des retours sur la vue, et sur la table équivalente ?
2. Combien de colonnes le plan annonce-t-il pour un `SELECT *`, et combien pour trois colonnes nommées ?
3. Quels sont les trois étages du plan du croisement magasin / chiffre d'affaires ?
4. Quels temps et quels plans donnent la recherche par client, avec et sans index, en DuckDB ?
5. Que répond le plan SQLite de la même recherche, sans index puis avec ?
6. Quel résultat commun les deux moteurs renvoient-ils pour le client 15676 ?
7. Que coûtent la fenêtre et la sous-requête scalaire sur la même part, et lequel des deux est le plus lisible ?
8. Combien pèse un fichier DuckDB contenant une table de 154 lignes, et qu'en déduit-on pour le socle du module ?
9. Que contient une note de méthode de performance, en cinq points ?
10. Pourquoi les valeurs de performance du module sont-elles gelées dans un fichier, et de quelle date ?

Réponses. 1. **142 ms** sur la vue, **1 ms** sur la table (**×142**). 2. **18** contre **3** (rapport **×6** dans le plan, **×6,1** à l'horloge : **581 ms** contre **96 ms**). 3. `HASH_GROUP_BY`, `PROJECTION`, `HASH_JOIN`. 4. **0,6 ms** sans index, **0,5 ms** avec, plan `SEQ_SCAN` dans les deux cas. 5. `SCAN ventes_t` puis `SEARCH ventes_t USING INDEX idx_ventes_client (id_client=?)`. 6. **14** lignes et **5 578 688 FCFA**. 7. **150 ms** pour la fenêtre, **292 ms** pour la sous-requête scalaire (**×1,9**) : la fenêtre est aussi la plus lisible. 8. **536 576** octets — un plancher fixe : le socle reste un script plutôt qu'un fichier de base. 9. Moteur et version, protocole, date, emplacement des valeurs, et ce qui n'a pas été mesuré. 10. Pour que deux exécutions du relevé donnent le même document ; elles sont gelées le **24/09/2026** dans `PERF_M11.json`.

Module M11.C07 — La requête comme livrable : style, convention, tests de non-régression

Outils : DuckDB 1.5.5 (exécuté), SQLite (contrôle croisé, exécuté), PostgreSQL et SQL Server (cités, non exécutés — règle §1.5. **Durée indicative :** 3 h. **Niveau :** N3 → N4. **Pré-requis :** M11.C01 à C06 ; M07.C08 (qualité, contrôles) ; M05.C07 (documentation d'un classeur).

L'idée du chapitre. Une requête juste qu'on ne peut pas relire est une requête qu'on ne peut pas corriger. Ce dernier chapitre du module traite les requêtes comme des **livrables** : un en-tête qui dit la question métier, un nommage qui se relit six mois plus tard, et des **tests de non-régression** qui refusent qu'un chiffre publié change sans que personne ne s'en aperçoive. Le module en fournit la preuve la plus concrète : `tools/controle_sql_M11.py` rejoue **18** requêtes et confronte chaque résultat au fichier publié — **18** contrôles, **18** accords. Et il porte un avertissement : un test qui **réécrit** la requête ne teste plus la même chose. Sur le socle, un classement des clients par nombre de tickets classé en ordre croissant au lieu de décroissant donne **23 497** au lieu de **23 444** — même données, même requête « à peu près », deux chiffres.

Matériel de l'atelier — DuckDB 1.5.5 · socle M11 (`03_exercices/dossier_M11/socle_m11.sql`) · **instrument** `tools/controle_sql_M11.py`. Les chiffres de chapitre sont ceux du module, gelés dans `chiffres_cites.json` ; les sorties montrées sont celles de l'atelier, exécutées le 24/09/2026.

1. Objectifs du chapitre

À la fin de ce chapitre, vous savez :

1. **Écrire un en-tête de requête** qui porte la question métier, le périmètre, la date et l'auteur.
2. **Nommer** les étapes, les alias et les colonnes de sortie pour qu'une relecture soit possible.
3. **Appliquer les conventions du manuel** : mots-clés en majuscules, une requête pour une question, pas de `SELECT *`, unités déclarées, apostrophes échappées.
4. **Écrire un test de non-régression** qui rejoue une requête et compare son résultat à une valeur publiée.
5. **Construire un contrôle croisé** entre deux sources indépendantes, et **expliquer un écart** au lieu de le masquer.
6. **Rejouer une requête à l'identique** dans un test — et savoir ce qu'on mesure quand on la réécrit.
7. **Livrer un rapport SQL** : requêtes, tests, note de méthode, et la liste de ce qui n'a pas été vérifié.

2. Pourquoi cette notion est importante

Le module M11 publie **12** rapports, **20** requêtes de référence et **18** requêtes auto-testées. Chaque chiffre de ces rapports se retrouve un jour dans une réunion, puis dans un budget. Le mode de défaillance n'est pas l'erreur grossière : c'est le chiffre qui change **sans que personne ne le remarque** — un libellé corrigé dans le référentiel, une ligne de retour ajoutée, un filtre déplacé d'une clause à l'autre.

Les tests de non-régression existent pour cela, et ils coûtent étonnamment peu : **18** requêtes rejouées, comparées à un fichier publié, un verdict par ligne et un code de sortie. Le jour où un chiffre bouge, la cause est identifiée en une minute au lieu d'une réunion.

Le chapitre porte aussi un piège de méthode, mesuré sur le socle. Un contrôle sur le nombre de tickets par client donne **23 444** si la requête source classe en ordre **décroissant** — et **23 497** si le test la classe en ordre **croissant**. Personne n'a touché aux données : c'est le test qui a changé de question. Un test de non-régression qui ne rejoue pas la requête **à l'identique** ne protège rien ; il fabrique des alertes.

3. Explication simple — la fiche de recette et le contrôle qualité

Une requête livrée, c'est une fiche de recette. En haut : le nom du plat, pour combien de personnes (le périmètre), la date, qui l'a écrite. Ensuite les étapes, numérotées et nommées. En bas : ce qui peut rater. Une requête sans en-tête oblige le suivant à deviner la question — et à la deviner faux.

Un test de non-régression, c'est le contrôle qualité du rayon. On ne regoûte pas le plat chaque matin ; on vérifie que l'étiquette dit la même chose que le contenu. Sur le module : **18** étiquettes, **18** contenus, et l'accord est vérifié à chaque exécution.

Un contrôle croisé, c'est peser la farine sur deux balances. Deux instruments différents, la même grandeur : si les deux donnent **15 595 154 955** FCFA, on avance ; s'ils diffèrent, on cherche **lequel** a bougé avant de publier. Le module compare ainsi une mesure DuckDB et une mesure pandas.

4. Vocabulaire essentiel

Terme (français)	Équivalent anglais	Définition en une phrase
en-tête de requête	query header	Le bloc de commentaires qui précède une requête : question, périmètre, source, date, auteur.
convention de nommage	naming convention	Les règles partagées sur les noms de tables, de colonnes et d'étapes intermédiaires.
test de non-régression	regression test	Un test qui rejoue une requête et compare son résultat à une valeur attendue.
valeur attendue	expected value	La valeur publiée à laquelle le test compare le résultat — ici, celle du fichier <code>chiffres_cites.json</code> .
contrôle croisé	cross-check	La vérification d'une même grandeur par deux chemins indépendants (deux moteurs, deux méthodes).
code de sortie	exit code	Le signal rendu par un contrôle : zéro si tout passe, non nul sinon — c'est ce qui l'automatise.
jeu de tests	test set	L'ensemble des contrôles d'un module ; ici, 18 requêtes rejouées.
requête l'identique	à verbatim replay	La règle d'or d'un test : rejouer exactement la requête publiée, sans la réécrire.
écart expliqué	explained gap	La différence entre deux socles, documentée (périmètre, période, retours) et jamais additionnée.

Définition. Un **test de non-régression** est un contrôle qui rejoue une requête **publiée** et compare son résultat à une **valeur attendue**. Sa valeur n'est pas de trouver des erreurs : elle est de **signaler un changement**. Un test qui échoue ne dit pas « le rapport est faux », il dit « quelque chose a bougé, et il faut savoir quoi avant de livrer ». C'est une alerte, pas un verdict.

Définition. Un **contrôle croisé** vérifie une même grandeur par deux chemins **indépendants** : deux moteurs (DuckDB et SQLite), deux outils (SQL et pandas), ou deux socles (le socle M11 et celui de M09). Il porte une conclusion plus forte qu'un simple test, à une condition : les deux chemins ne doivent pas partager la même source d'erreur.

Définition. Le **code de sortie** d'un contrôle est le signal qu'il rend au système : zéro si tous les contrôles passent, non nul sinon. C'est ce détail — et non l'affichage — qui rend un jeu de tests utilisable dans une chaîne automatique : un rapport qui se construit sans que les contrôles aient tourné n'est pas un rapport vérifié.

5. Cours approfondi

5.1 L'en-tête de requête : la question avant le code

Le manuel impose un en-tête court, en commentaires, à toute requête livrée :

```
-- Q07 · Part du chiffre d'affaires des 10 % de clients les plus gros
-- Périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, 5 magasins, client 0
includ
-- Source   : 03_exercices/dossier_M11/socle_m11.sql (vue ventes)
-- Sortie   : une ligne par centile, plus la part cumulée
-- Publiée  : 38,1 % (voir chiffres_cites.json, clé m11_concentration_top10_pct)
```

Ces cinq lignes répondent aux cinq questions qu'un relecteur se pose — et elles sont aussi ce dont un test de non-régression a besoin pour vérifier la **bonne** valeur : une requête sans périmètre écrit ne permet pas de savoir si les **38,1 %** publiés et les **24,2 %** hors client non identifié parlent du même ensemble.

Conseil professionnel. Écrivez l'en-tête **avant** la requête, pas après. Tant que le périmètre n'est pas rédigé, la requête n'est pas finie : c'est en écrivant la ligne « Périmètre » qu'on découvre qu'on hésite entre ventes brutes et ventes nettes — et cette hésitation est exactement ce qui fait publier deux chiffres différents du même indicateur.

5.2 Nommer : les étapes, les alias, les colonnes

Trois règles, et elles suffisent :

1. **Une étape, un nom qui dit son rôle.** `nets`, `ca_mois`, `rang_magasin` — jamais `tmp1`, `tmp2`, `t3`.
2. **Un alias qui se lit à voix haute.** `SUM(montant_ttc) AS ca_net`, et non `AS s`. Une colonne de sortie est lue par quelqu'un qui n'a pas écrit la requête.
3. **Un nom de colonne de sortie en français**, comme le reste du manuel : `part_pct`, `rang`, `mois`.

Attention. Un alias muet est un défaut de livraison, pas un détail esthétique. `SELECT COUNT(*) AS n1, SUM(x) AS s` sort une table publiée avec deux colonnes nommées `n1` et `s` : l'analyste qui reçoit ce fichier doit revenir demander ce que contiennent les colonnes — et il posera la question à quelqu'un qui ne se souvient plus.

5.3 Un style qui se relit

Les conventions du manuel, appliquées dans tous les chapitres du module :

Règle	Exemple
Mots-clés en majuscules, identifiants en minuscules	<code>SELECT id_magasin FROM ventes WHERE NOT est_retour</code>
Une requête par question ; une question par requête	Le titre de l'en-tête est une phrase, pas un thème
Pas de <code>SELECT *</code> dans un rapport	colonnes nommées (C06 : 581 ms contre 96 ms)
Alias de table courts mais lisibles	<code>ventes v JOIN magasin m USING (id_magasin)</code>
Unités écrites dans le nom de la colonne	<code>ca_net_fcfa</code> , <code>part_pct</code> , <code>duree_jours</code>
Apostrophes échappées dans les filtres	<code>mode_paiement = 'Créance 30 j'</code> — et non des guillemets doubles
Découpage en CTE plutôt qu'en requêtes imbriquées	une étape nommée par intention (C05)

5.4 Les trois tests de non-régression du module

Le module en compte 3, et chacun protège autre chose :

Test	Ce qu'il protège	Instrument
Rejeu des 18 requêtes de référence contre les valeurs publiées	les chiffres du module	<code>tools/controle_sql_M11.py</code>
Comparaison de deux relevés du socle	la stabilité du fichier de valeurs	<code>chiffres_manuel.py</code> M11 (deux exécutions, <code>diff = 0</code> ligne)
Comparaison des deux socles (M11 et M09)	le périmètre des rapports	rapport R12 du projet

Le premier est exécuté ici, dans son intégralité :

```
=== controle SQL M11 : 18 requetes, une source par cote ===
cote SQL : socle_m11.sql rejoue par DuckDB 1.5.5
cote publie: chiffres_cites.json (265 cles M11)

| 1 | m11_lignes | 240000 | 240000 | OK |
| 2 | m11_lignes_hors_retours | 237191 | 237191 | OK |
| 3 | m11_retours | 2809 | 2809 | OK |
| 4 | m11_tickets | 145212 | 145212 | OK |
| 5 | m11_clients_vente | 23497 | 23497 | OK |
| 6 | m11_magasins | 5 | 5 | OK |
| 7 | m11_produits | 154 | 154 | OK |
| 8 | m11_ca_2023 | 3443581547 | 3 443 581 547 FCFA | OK |
| 9 | m11_ca_8_mois_2026 | 3360553372 | 3 360 553 372 FCFA | OK |
|10 | m11_client0_lignes | 42658 | 42658 | OK |
|11 | m11_client0_ca | 2852612447 | 2 852 612 447 FCFA | OK |
|12 | m11_c02_max_rank | 23444 | 23444 | OK |
|13 | m11_c02_max_dense_rank | 21 | 21 | OK |
|14 | m11_c03_ratio_saison | 1.65 | 1.65 | OK |
|15 | m11_c04_rfm_blocs | 7833 | 7 833 / 7 832 / 7 832 | OK |
|16 | m11_c04_oneshot | 64 | 64 | OK |
|17 | m11_c05_grouping_sets_lignes | 47 | 47 | OK |
|18 | m11_calendrier_jours | 1339 | 1339 | OK |

18 controles sur 18 : OK – le socle et le fichier publie disent la meme chose.
```

Ce tableau est un **contrôle croisé**, au sens strict : le côté gauche est une requête exécutée à l'instant, le côté droit un fichier écrit plus tôt — parfois par une autre exécution, parfois sur une autre machine (le fichier de valeurs a été produit par `chiffres_manuel.py`). Les deux chemins ne partagent ni le moment, ni le code : c'est ce qui donne sa valeur à l'accord.

Définition. La **valeur attendue** d'un test est la valeur **publiée** — ici, celle du fichier `chiffres_cites.json` — et non la valeur qu'on vient de lire à l'écran. C'est cette distinction qui fait la force du contrôle : le test ne compare pas le socle à lui-même, il compare le socle **aujourd'hui** à ce que le module affirmait **avant**. Sans valeur attendue écrite quelque part, un test ne peut que constater que « ça tourne ».

5.5 Le contrôle entre deux sources : expliquer un écart, jamais l'additionner

Le module travaille sur **2** socles. Le socle M11 — celui des chapitres — compte **240 000** lignes de ventes. Le socle de M07/M09, construit pour la quincaillerie, en compte **50 008** et affiche **7 908 259 732** FCFA bruts, **7 876 320 165** hors retours. Les additionner n'aurait aucun sens : ils ne mesurent ni la même période, ni le même périmètre.

Le rapport R12 du projet fait le contraire : il **confronte** les deux et **explique** l'écart — lignes, retours, magasins, période. C'est le troisième test du module, et le plus formateur : il n'y a pas de « bonne » réponse à trouver, il y a une différence à documenter. Un rapport qui publie un écart sans l'expliquer laisse à son lecteur le soin d'imaginer une erreur — et le lecteur imaginera la pire.

5.6 Un test rejoue la requête à l'identique

Voici la même question posée à deux ordres de tri près :

```
-- version publiée (ordre décroissant) : rang maximum 23444
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1),
      r AS (SELECT RANK() OVER (ORDER BY t DESC) rk FROM c) SELECT MAX(rk) FROM r;
-- version « équivalente » écrite dans un test (ordre croissant) : rang maximum 23497
WITH c AS (SELECT id_client, COUNT(DISTINCT id_ticket) t FROM ventes WHERE NOT est_retour GROUP BY 1),
      r AS (SELECT RANK() OVER (ORDER BY t) rk FROM c) SELECT MAX(rk) FROM r;
```

23 444 contre **23 497**. La première valeur est celle que le module publie (avec **21** comme plus grand `DENSE_RANK`), la seconde est le nombre de clients — un résultat qui a l'air correct et qui ne répond pas à la question posée. Un test qui aurait « réécrit proprement » la requête aurait donc signalé un écart inexistant, ou pire : validé **23 497** et fait diverger le module de son propre fichier.

Attention. Un test de non-régression recopie la requête **caractère pour caractère** — ordre de tri, filtre de retour, périmètre, jointure. Toute « amélioration » du texte dans un test est une nouvelle requête, donc un nouveau résultat : ce n'est plus un test, c'est une source de bruit. Le corollaire est agréable : quand un test échoue, la première chose à vérifier n'est pas la donnée, c'est **la requête du test**.

Dans les faits. Le contrôle du module n'a pas seulement validé **18** chiffres : il en a **corrigé** un. Le contrôle du client **0** a d'abord compté **43 161** lignes, là où le module publie **42 658** — l'écart venait de la requête de contrôle, qui comptait aussi les lignes de retour, alors que la valeur publiée est en ventes nettes. La donnée était bonne, la mesure était fausse : c'est exactement ce qu'un contrôle croisé doit détecter, et il vaut mieux qu'il le fasse sur un contrôle que sur un rapport.

5.7 Ce que ce chapitre ne couvre pas

Les tests d'**intégration** et la chaîne d'automatisation (exécuter les contrôles à chaque modification, refuser la livraison en cas d'échec, notifier l'équipe) relèvent de l'ingénierie et non de l'analyse : ils sont cités ici et traités dans l'option M17. Le principe utile à retenir reste accessible : un contrôle qui ne s'exécute pas tout seul finira par ne plus s'exécuter du tout.

6. Exemple concret — la revue d'un rapport avant envoi

Un rapport est prêt. La revue tient en dix minutes et cinq questions, dans cet ordre :

#	Question	Réponse attendue
1	L'en-tête dit-il la question, le périmètre et la source ?	cinq lignes, pas zéro
2	Les colonnes de sortie portent-elles leurs unités ?	ca_net_fcfa, part_pct
3	Reste-t-il un <code>SELECT *</code> ?	non (C06 : 581 ms contre 96 ms)
4	Les classements ont-ils une clé de départage ?	oui (C04 : 3 316 clients à 8 tickets)
5	Les chiffres publiés passent-ils les contrôles ?	18 sur 18

Cinq questions, une seule exige un outil, et aucune ne demande de relire la logique métier : c'est le propre d'une revue utile.

7. Démonstration pas à pas — cinq étapes sur le socle

7.1 Étape 1 — écrire l'en-tête

La question, le périmètre, la source, la sortie, la valeur publiée : cinq lignes avant la première ligne de SQL. Sur le socle, la question la plus sensible du module — la concentration — porte deux valeurs selon le traitement du client non identifié (**38,1 %** ou **24,2 %**) : sans en-tête, on ne sait pas laquelle est publiée.

7.2 Étape 2 — nommer les étapes

La CTE `c` (le client et son nombre de tickets) et la CTE `r` (le rang) du §5.6 : deux étapes, deux noms, et une requête qui se relit.

7.3 Étape 3 — rejouer les 18 contrôles

`python3 tools/controle_sql_M11.py` : **18** contrôles, **18** accords, code de sortie nul. Le tableau du §5.4 est la sortie réelle de l'atelier.

7.4 Étape 4 — comparer les deux socles

240 000 lignes contre **50 008**, **15 595 154 955** FCFA contre **7 908 259 732** bruts : l'écart s'explique (période, périmètre, retours) et ne s'additionne pas.

7.5 Étape 5 — documenter ce qui n'a pas été testé

PostgreSQL, SQL Server, les partitions, la montée en charge : **cités**, non exécutés. Une note de méthode qui ne dit que ce qui marche est une publicité, pas une méthode.

8. Erreurs fréquentes

1. **Publier une requête sans en-tête.** Le relecteur ne peut ni vérifier le périmètre ni reproduire le chiffre (**38,1 %** ou **24,2 %** selon le traitement du client non identifié).
2. **Réécrire la requête dans le test.** **23 444** contre **23 497** pour un ordre de tri inversé : le test signale alors un écart qui n'existe pas, et la confiance dans les contrôles en sort abîmée.
3. **Additionner deux socles.** **240 000** lignes et **50 008** lignes ne mesurent pas la même chose : un total commun n'a aucun sens, et il est impossible à défendre.
4. **Chercher les erreurs, pas les changements.** Un test de non-régression ne dit pas « c'est faux » : il dit « quelque chose a bougé ». Confondre les deux fait ignorer les alertes utiles.
5. **Laisser un contrôle sans code de sortie.** Un contrôle qui affiche « OK » sans rendre de code ne s'intègre à rien : il dépend d'un humain qui le lance et qui lit.
6. **Publier des alias muets.** `n1`, `s`, `col3` : le rapport est juste et inutilisable.

9. Bonnes pratiques professionnelles

1. **L'en-tête d'abord.** Cinq lignes : question, périmètre, source, sortie, valeur publiée.
2. **Un test recopie la requête à l'identique.** Aucune reformulation, aucun « nettoyage » : le test est un témoin, pas une réécriture.
3. **Comparer plutôt qu'additionner.** Deux socles se confrontent et s'expliquent (rapport R12) ; ils ne se somment jamais.
4. **Rendre un code de sortie.** Un jeu de tests s'utilise dans une chaîne automatique ou ne sert à rien.
5. **Documenter ce qui n'a pas été vérifié.** Moteurs cités, partitions citées, mesures non faites : la même honnêteté que pour les chiffres (§1.5).

10. Exercice guidé

Situation. Vous reprenez une requête écrite par un collègue : elle calcule la part du chiffre d'affaires des **10 %** de clients les plus gros, sans en-tête, avec des alias `a`, `b`, `c`, et un `SELECT *` dans une CTE intermédiaire. Elle donne **38,1 %**.

Consigne.

1. Écrivez l'en-tête complet de la requête, périmètre compris. (3 pts)
2. Renommez les étapes et les colonnes de sortie, et justifiez deux de vos choix. (2 pts)
3. Écrivez le contrôle de non-régression associé à cette requête, dans l'esprit de `tools/controle_sql_M11.py`. (3 pts)
4. Le même calcul hors client non identifié donne **24,2 %** : comment l'en-tête doit-il le dire, et pourquoi ? (2 pts)

11. Exercices autonomes

E1 — Cinq requêtes, cinq en-têtes (40 min). Reprenez cinq requêtes des chapitres C01 à C06 (celles que vous avez écrites aux exercices) et ajoutez-leur un en-tête complet. Puis relisez-les une par une et notez ce que l'en-tête vous a obligé à préciser que la requête laissait implicite.

E2 — Le jeu de tests d'un rapport (40 min). Choisissez un des **12** rapports du projet, listez les valeurs qu'il publie, et écrivez pour chacune un contrôle : la requête rejouée à l'identique, la clé de la valeur attendue, et ce que l'échec signifierait. Terminez par la phrase qui doit figurer dans la note de méthode, et par celle qui dit ce que vous n'avez pas testé.

12. Correction détaillée

Exercice guidé.

1. **L'en-tête.** Question : « part du chiffre d'affaires net des 10 % de clients les plus gros ». Périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, 5 magasins, **client 0 inclus**. Source : `socle_m11.sql`. Sortie : une ligne par centile, avec part cumulée. Publié : **38,1 %** (`m11_concentration_top10_pct`).
2. **Les noms.** `clients_ca` (le chiffre d'affaires par client) et `deciles` (le découpage en centiles) au lieu de `a` et `b` : le nom dit ce que contient l'étape, et la relecture ne demande plus de reconstruire le raisonnement. Les colonnes de sortie portent leurs unités : `ca_net_fcfa`, `part_pct`.
3. **Le contrôle.** Rejeu de la requête à **l'identique**, comparaison à la clé publiée, verdict par ligne, code de sortie non nul en cas d'écart — la structure de `tools/controle_sql_M11.py` (qui en compte 18).
4. **Le client non identifié.** L'en-tête doit dire que le client **0** est inclus — et donner la valeur hors client non identifié (**24,2 %**) dans la note, puisque le retrait est une décision de périmètre : sans cette phrase, deux services publieront deux chiffres différents du même indicateur.

E1 (les en-têtes). Les points que l'en-tête fait apparaître sont toujours les mêmes : le **périmètre** (retours inclus ou non), le **grain** (client, ticket, ligne), la **période**, et le **traitement des cas particuliers** (client **0**, magasin sans objectif, cohorte incomplète). C'est la liste des quatre erreurs les plus fréquentes du module, et l'en-tête les rend visibles **avant** la publication.

E2 (le jeu de tests). Un bon jeu de tests couvre trois natures de valeurs : un **volume** (les **240 000** lignes, les **145 212** tickets), un **total** (**15 595 154 955** FCFA, **3 443 581 547** pour 2023) et un **taux** (**38,1 %**, **15,8 %** de rétention à M+1). Trois natures, trois façons de casser : un volume qui bouge signale un changement de filtre ou de source, un total un changement de périmètre, un taux un changement de dénominateur. Et la phrase de méthode : « les contrôles rejouent les requêtes publiées à l'identique ; les moteurs cités et non exécutés ne sont pas testés ».

13. Mini-projet de chapitre

La remise en forme d'un rapport existant, 2 pages. Prenez l'un des **12** rapports du projet de module, réécrivez-le selon les conventions du chapitre (en-tête, nommage, colonnes nommées) et livrez son jeu de tests.

Livrables et barème.

Critère	Ce qui est vérifié	Points
L'en-tête	question, périmètre, source, sortie, valeur publiée	5
Le style	étapes nommées, alias lisibles, unités dans les noms, aucun <code>SELECT *</code>	4
Les tests	requêtes rejouées à l'identique , valeurs attendues citées, code de sortie	6
La note de méthode	ce qui est vérifié, ce qui ne l'est pas, et la limite du rapport	3
Total		18

14. Résumé du chapitre

Une requête livrée sans en-tête est une requête qu'on ne peut pas vérifier : cinq lignes — question, périmètre, source, sortie, valeur publiée — suffisent à la rendre reproductible. Le style n'est pas une coquetterie : les étapes nommées, les alias lisibles et les unités dans les noms de

colonnes font la différence entre un rapport qu'on relit et un rapport qu'on refait. Les **3** tests de non-régression du module couvrent les chiffres (**18** contrôles rejoués contre le fichier publié), la stabilité du fichier de valeurs (deux relevés, `diff = 0` ligne) et le périmètre des rapports (comparaison des deux socles, **240 000** lignes contre **50 008**). Deux règles portent tout le reste : un test recopie la requête à l'**identique** — l'ordre de tri inversé donne **23 497** au lieu de **23 444**, et ce n'est pas une erreur de donnée mais une erreur de test — et un contrôle qui ne rend pas de **code de sortie** ne sert à rien.

15. À retenir

1. **L'en-tête d'abord** : la question, le périmètre, la source, la sortie, la valeur publiée.
2. **Un nom dit un rôle**. `clients_ca`, `rang_magasin`, jamais `tmp`, `t3`, `n1`.
3. **Un test rejoue la requête à l'identique**. Réécrire la requête du test, c'est changer de question.
4. **Un test signale un changement, il ne prouve pas une erreur**. C'est une alerte, pas un verdict.
5. **Deux socles se comparent, ils ne s'additionnent pas** : **240 000** lignes contre **50 008**.
6. **Un contrôle rend un code de sortie**, sinon il dépend d'un humain qui se souvienne de le lancer.
7. **La note de méthode dit aussi ce qui n'a pas été testé**.

À retenir. Le chiffre qui résume le chapitre est une paire : **23 444 / 23 497**. Même requête, même socle, même question — à un ordre de tri près. Un test qui n'aurait pas recopié la requête publiée aurait signalé une fausse alerte, ou validé un chiffre faux. C'est le seul chapitre du module où la rigueur porte sur **le contrôle lui-même**.

À retenir. **18** contrôles, **18** accords, et un code de sortie nul : voilà à quoi ressemble un module qu'on peut livrer. Le jour où l'un des **18** passe au rouge, on saura **quoi** regarder — et c'est tout ce qu'on demande à un test.

16. Évaluation formative

Répondez sans machine, puis vérifiez avec l'atelier (5 minutes) :

1. Que contient l'en-tête d'une requête livrée, et pourquoi le périmètre y figure-t-il en premier ?
2. Combien de tests de non-régression compte le module, et que protège chacun ?
3. Combien de contrôles rejoue `tools/controle_sql_M11.py`, et contre quelle source ?
4. Que donne le rang maximum des clients par nombre de tickets en ordre décroissant, et en ordre croissant ?
5. Pourquoi un test de non-régression ne doit-il jamais réécrire la requête qu'il rejoue ?
6. Que fait un contrôle croisé de plus qu'un simple test, et à quelle condition ?
7. Combien de lignes comptent les deux socles du module, et pourquoi ne pas les additionner ?
8. Pourquoi un contrôle doit-il rendre un code de sortie ?
9. Citez trois natures de valeurs qu'un jeu de tests doit couvrir.
10. Que doit dire la note de méthode d'un rapport, en plus de ce qu'il vérifie ?

Réponses. 1. Question, périmètre, source, sortie, valeur publiée — le périmètre d'abord, parce que c'est lui qui décide de la valeur publiée (**38,1 %** ou **24,2 %** selon le client non identifié). 2. **3** : les chiffres du module, la stabilité du fichier de valeurs, et le périmètre des rapports entre les deux socles. 3. **18** contrôles, contre les valeurs publiées de `chiffres_cites.json` (côté SQL : le socle rejoué). 4. **23 444** en ordre décroissant, **23 497** en ordre croissant. 5. Parce que toute ré-

écriture est une autre requête : on testerait un résultat qui n’a jamais été publié. 6. Il vérifie une même grandeur par deux chemins indépendants — à condition que les deux ne partagent pas la même source d’erreur (ici : une requête exécutée contre un fichier écrit plus tôt). 7. **240 000** lignes et **50 008** : périodes, périmètres et retours différents — les additionner produirait un total indéfendable. 8. Pour pouvoir l’enchaîner dans un traitement automatique, sans dépendre d’un humain qui lit la sortie. 9. Un volume, un total, un taux. 10. Ce qui n’a **pas** été vérifié ou exécuté (moteurs cités, partitions, montée en charge).

Projet M11.P — « Les 12 rapports SQL de la cellule commerciale »

Module M11 · projet noté sur 20 · seuil 13 · durée imposée 4 h (chronomètre en marche) · rendu : 4 livrables — P1 les 12 requêtes (8 points), P2 le jeu de tests (5 points), P3 la note de lecture (4 points), P4 la fiche de reprise (3 points).

Ce que le projet évalue. Non pas « la requête la plus savante », mais la **bibliothèque de rapports** : douze requêtes autonomes, **versionnées**, **testées** et **commentées en français**, chacune répondant à une question écrite. Vous recevez la commande de la cellule commerciale et le socle du module : **240 000** lignes de vente, **237 191** hors retours, **145 212** tickets, **23 497** clients, **5** magasins, **44** mois. La règle du projet est celle du module : **un rapport dont le total diffère d'un rapport à l'autre est rejeté**, même si chaque requête « fonctionne » — et un rapport que son lecteur ne peut pas rejouer ne vaut rien.

1. Énoncé

La commande

La cellule commerciale d'une enseigne de quincaillerie (**5** magasins, **44** mois, **240 000** lignes) commande sa **bibliothèque de rapports** : **12** requêtes autonomes, chacune répondant à une question qu'elle s'est posée cette année. Elle les veut **en français**, **datées**, **testées**, et livrées avec leur **mode d'emploi** — pas un dossier de requêtes, un **outil de travail**.

Le matériel

Élément	Chemin	Ce que c'est
Le socle	03_exercices/dossier_M11/socle_m11.sql	les 5 tables de référence et les 2 vues de ventes (3 595 octets)
La connexion	03_exercices/dossier_M11/connexion.py	ouvrir(materialiser=False) — la matérialisation en mémoire pour les mesures
Les mesures de performance	03_exercices/dossier_M11/PERF_M11.json	les temps gelés du chapitre C06 (24/09/2026 , médianes de 5 exécutions)
Le contrôle automatique	tools/controle_sql_M11.py	18 requêtes rejouées et confrontées au fichier publié — le calque de votre livrable P2
Les valeurs publiées	01_socle_donnees/data/reference/chiffres_cites.json	clés m11_*, m11p_*, m11e_* : 301 clés pour le module
Les 7 chapitres	02_modules/M11_C01...C07	fenêtres, rangs, temps, cohortes, requêtes avancées, plans, style et tests

Les six totaux de contrôle

Ces six nombres se recalculent depuis le socle — ils ne se recopient pas. Un rapport qui publie un total différent doit **dire pourquoi** (périmètre plus étroit, période partielle, client non identifié exclu) : c'est la seule tolérance admise, et elle s'écrit dans l'en-tête.

Contrôle	Valeur attendue
Chiffre d'affaires net (retours exclus)	15 595 154 955 FCFA
Lignes de vente (retours exclus)	237 191

Contrôle	Valeur attendue
Tickets de vente	145 212
Clients identifiés dans les ventes	23 497
Magasins	5
Mois couverts	44

2. Les 12 questions métier (le catalogue imposé)

Chaque rapport répond à **une** question, citée telle quelle dans son en-tête. Le chapitre indiqué est celui qui fournit la méthode ; la colonne « sortie » dit ce que le rapport doit **contenir**, pas la façon de l'obtenir.

Code	Question de la cellule	Chap.	Sortie attendue
R01	Combien avons-nous vendu chaque mois, et où en sommes-nous cette année ?	C01, C02	mois, CA, cumul, part du mois dans son année
R02	Quels magasins vendent le plus — et lesquels ont le plus gros panier ?	C01, C02	deux classements côte à côte, chacun avec son rang
R03	Quels sont nos 3 meilleurs produits par famille ?	C02	3 lignes par famille, rang, part dans la famille
R04	Quelle part du chiffre d'affaires font nos plus gros clients ?	C01, C04	parts cumulées, top 1 %, top 5 %, top 10 %, top 20 %
R05	L'activité progresse-t-elle, à périmètre comparable ?	C03	année sur année à mois constants, moyenne des variations
R06	La tendance de fond monte-t-elle ?	C03	moyennes mobiles de 3 et 12 mois, avec leur compteur de fenêtre
R07	Tenons-nous nos objectifs ?	C03	réalisé, objectif, taux — trous compris et déclarés
R08	Quel jour de la semaine travaillons-nous le mieux ?	C03	CA par jour de vente, écart entre le meilleur et le pire
R09	Nos clients reviennent-ils ?	C04	rétenion pondérée à M+1, M+3, M+12, avec les effectifs
R10	Quels clients appeler en priorité, et pour quoi ?	C04	segments RFM, effectif, part du CA, plage de récence
R11	Où vend-on quoi ?	C05	tableau croisé quartier × famille avec sous-totaux
R12	Nos deux socles racontent-ils la même histoire ?	C07	comparaison documentée des deux socles, écarts expliqués

Douze questions, douze rapports, une seule vérité de total : les 12 rapports publient le même chiffre d'affaires net quand ils parlent de la même période — **15 595 154 955 FCFA** sur le socle complet, ou un sous-ensemble dont le périmètre est écrit noir sur blanc.

Le format d'en-tête, imposé

Chaque rapport s'ouvre sur ces six lignes — c'est ce qui distingue une bibliothèque d'un fichier de brouillons :


```
-- Rxx – <question de la cellule, recopiée telle quelle>
-- Hypothèses : <ce qu'on suppose : retours exclus, client 0 inclus, période complète...>
-- Grain : <une ligne = un mois / un client / un couple quartier-famille>
-- Source : <socle M11, vue ventes, référence produit>
-- Total de contrôle : <le nombre que le rapport doit rendre>
-- Date : <le jour de rédaction>
```

3. Les 4 livrables

P1 — Les 12 requêtes (8 points)

Une requête par question, dans l'ordre R01 → R12, chacune avec **son en-tête complet**, un ORDER BY **explicite** — avec sa clé de départage quand deux lignes peuvent être à égalité —, et son **total de contrôle**.

Ce qui est vérifié à la correction :

- R02 : les deux classements sont donnés avec leur **rang** ; le rapport signale que l'ordre n'est pas le même (**Kaya** a le plus gros panier, **Ouagadougou** le plus gros chiffre d'affaires).
- R03 : le classement porte sur les **7** familles réelles, pas sur les **16** libellés du référentiel ; le rapport publie les **deux** comptes (**46** lignes contre **21**).
- R04 : la part du top 10 % est publiée **avec** et **sans** le client non identifié, et la définition du décile est écrite.
- R05 : aucune comparaison entre périodes de longueurs différentes ; la période partielle est annoncée.
- R06 : chaque moyenne mobile porte le nombre de lignes de sa fenêtre ; les douze premiers mois sont marqués comme **non comparables**.
- R07 : les **2** couples magasin-mois sans objectif apparaissent dans le rapport, avec un état explicite et **93 749 490** FCFA de ventes sans référence.
- R09 : la rétention est **pondérée** par la taille des cohortes ; le tableau porte les effectifs.
- R10 : le traitement du client **0** est écrit ; tous les classements sont départagés.
- R11 : les sous-totaux sont produits dans la **même** requête (ROLLUP ou GROUPING SETS), et le total général égale le chiffre d'affaires net.

P2 — Le jeu de tests (5 points)

Chaque requête est livrée avec **au moins un test de non-régression**, dans l'esprit de `tools/controle_sql_M11.py` : une valeur attendue (le total, le nombre de lignes, une valeur extrême), la requête rejouée à l'**identique**, le verdict, et un **code de sortie** utilisable en chaîne.

Ce qui est vérifié :

- **au moins un** test par rapport, soit **12** au minimum, plus la sortie de l'instrument du module (**18** lignes) publiée ;
- les tests **recopient** la requête du rapport caractère pour caractère : la plus petite réécriture change la réponse — un classement des clients par nombre de tickets rend **23 444** en ordre décroissant et **23 497** en ordre croissant ;
- un écart produit un **code de sortie non nul** : un test qui se contente d'afficher « échec » ne protège rien.

Attention. Un test qui « améliore » la requête qu'il rejoue ne teste plus le chiffre publié : il fabrique une alerte, ou pire, il valide une valeur qui n'apparaît nulle part dans le rapport.

P3 — La note de lecture (4 points · 2 pages)

Deux pages, pour la personne qui n'écrira jamais une ligne de SQL : ce que les **12** rapports disent **ensemble**, et les **3** précautions à connaître avant de citer un chiffre :

1. le client **0** — **18,3 %** du chiffre d'affaires, absent du référentiel, premier de tout classement ;
2. les **retours** — **2 809** lignes négatives : un total « net » qui les compte n'est pas un total ;
3. les **grains** — ligne, ticket, client : trois nombres pour la même question (**146 161** tickets au total, **145 212** avec au moins une vente, **2 797** qui ne portent que des retours).

La note se termine par **la question à laquelle les 12 rapports ne répondent pas** — par exemple pourquoi un client achète, ou ce qui se passera au prochain trimestre : un rapport décrit, il n'explique pas.

P4 — La fiche de reprise (3 points)

Un pair — qui n'a pas écrit les rapports — en reprend **3**, exécute leurs tests, et note **ce qui manque** pour qu'il puisse travailler sans vous : hypothèse non écrite, colonne ambiguë, test absent, nom de table supposé connu. La fiche publiée est la **sienne**, pas la vôtre ; votre annotation de son retour tient en trois lignes.

Conseil professionnel. Écrivez l'en-tête **avant** la requête. Tant que la ligne « Hypothèses » n'est pas rédigée, la requête n'est pas finie : c'est en l'écrivant qu'on découvre qu'on hésite entre ventes brutes et ventes nettes — et cette hésitation est exactement ce qui fait publier deux chiffres différents du même indicateur.

4. Barème

Livrable	Critère	Points
P1 · les 12 requêtes	12 requêtes rendues, dans l'ordre R01 → R12	4
	Les 12 en-têtes complets (question, hypothèses, grain, source, total, date)	2
	ORDER BY explicite et clé de départage partout où une égalité est possible	1
	Total de contrôle présent et exact	1
P2 · le jeu de tests	Au moins 1 test par rapport (12 au minimum)	2
	Requêtes rejouées à l'identique (aucune réécriture)	1,5
	Sortie de <code>tools/controle_sql_M11.py</code> publiée (18 lignes)	1
	Code de sortie non nul en cas d'écart	0,5
P3 · la note de lecture	Les 3 précautions (client 0 , retours, grains), chacune chiffrée	2
	Ce que les 12 rapports disent ensemble , en une page	1
	La question à laquelle ils ne répondent pas	1
P4 · la fiche de reprise	Les 3 rapports repris par un pair, tests exécutés	2
	Le compte rendu de ce qui manque, et votre réponse en trois lignes	1
Total		20

Seuil de validation : 13/20. Un livrable non rendu vaut zéro : il n'y a pas de points de présence.

5. Ce qui fait rejeter un livrable

1. **Un total qui change d'un rapport à l'autre** sans explication de périmètre : la règle du projet, et la seule faute éliminatoire.
2. **Un classement sans clé de départage.** 3 316 clients partagent exactement 8 tickets (14,1 % du fichier) : la frontière des tiers a compté de 2 718 à 2 738 clients selon l'exécution, sans qu'aucune donnée n'ait changé.
3. **Une jointure interne vers une table de référence** qui supprime en silence les 2 mois sans objectif du magasin 4.
4. **Une comparaison de périodes de longueurs différentes** présentée comme une évolution : 2026 (8 mois) contre 2025 (12 mois) donne **-28,9 %** ; à périmètre égal, **+15,4 %**.
5. **Un rapport dont les chiffres ne sont pas dans le fichier publié** : toute valeur citée se retrouve dans `chiffres_cites.json`, ou n'est pas publiée.
6. **Un test qui réécrit la requête qu'il contrôle** : ce n'est plus le chiffre du rapport qui est vérifié.
7. **Un moteur annoncé sans l'être** : PostgreSQL et SQL Server sont **cités** dans le module, jamais exécutés ; un rapport qui publie une sortie « PostgreSQL » non produite est faux.

6. Déroulé conseillé (4 séances, environ 8 h)

Séance	Objet	Livrable visé
1	Lire la commande, rejouer le socle, recalculer les 6 totaux, écrire les en-têtes des 12 rapports	P1 (en-têtes)
2	Écrire R01 → R06, avec sorties réelles et totaux de contrôle	P1
3	Écrire R07 → R12 (trous d'objectif, cohortes pondérées, RFM départagé, tableau croisé, contrôle croisé)	P1
4	Le jeu de tests, la note de lecture, l'échange de relecture avec un pair	P2, P3, P4

7. Comment vous corriger vous-même

1. **Les totaux** : `python3 tools/controle_sql_M11.py` doit rendre **18** lignes au vert sur le socle livré ; vos **12** tests s'ajoutent à la suite, dans le même format.
2. **Les sorties** : recopiez les sorties réelles de votre atelier, jamais une sortie approximative ; un tableau publié avec des valeurs arrondies « à la main » perd le contrôle.
3. **La relecture croisée** : échangez votre P4 avec un camarade. Le test est simple : peut-il **rejouer** trois de vos rapports et retrouver vos chiffres sans vous demander d'explication ? Si oui, la bibliothèque est bonne — c'est la définition du projet.

Le critère de réussite du projet. Un lecteur qui n'a jamais vu le socle ouvre votre dossier, lit l'en-tête d'un rapport, exécute la requête, obtient votre chiffre, lance le test qui le vérifie, et comprend en une phrase à quoi le rapport sert. Douze fois de suite.

Évaluation M11 — « SQL avancé pour la BI »

Module M11 · 30 h · niveau N3 → N4 · 7 chapitres. Total : 70 points — quiz 20 (seuil 14/20 §B.6), exercices à rendre 20 (dont 2 de type E4, « correction d'erreurs »), auto-test SQL 10, étude de cas 20 (seuil 12). Durée conseillée : 1 h pour le quiz, 2 h pour les exercices, 45 min pour l'auto-test, 1 h 30 pour l'étude de cas. **Matériel autorisé** : 03_exercices/dossier_M11/ (socle et connexion), 01_socle_donnees/, 01_socle_donnees/data/reference/chiffres_cites.json (clés m11_*, m11p_*, m11e_*) et l'instrument tools/controle_sql_M11.py. **Toute réponse chiffrée doit être mesurable** : un chiffre qui ne se retrouve pas dans le socle ne rapporte pas de point.

Ce que cette évaluation vérifie. Que vous savez **interroger** un socle de vente et **défendre** vos chiffres. Les questions ne demandent pas d'écrire des requêtes savantes : elles demandent de **dire** quelle construction répond à une question, de **prédire** la sortie d'une requête, de **diagnostiquer** un total faux, et de nommer ce qu'on manipule. Deux exercices (E4 et E5) portent sur des **requêtes erronées** : la justification pèse plus que la correction.

A · Questions de récupération (non notées)

Répondez de mémoire, en une ligne chacune. Elles ne rapportent rien : elles disent ce qu'il faut relire.

1. Qu'est-ce qui distingue une fonction de fenêtre d'un `GROUP BY` sur le **nombre de lignes** rendues ?
2. Que font respectivement `RANK`, `DENSE_RANK` et `ROW_NUMBER` quand deux lignes sont à égalité ?
3. Qu'est-ce qu'un **cadre de fenêtre**, et que change `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` ?
4. Qu'est-ce qu'une **cohorte**, et sur quel événement se date-t-elle ?
5. Que fait `QUALIFY` que `WHERE` ne peut pas faire ?
6. Que change `UNNEST` au **grain** d'un jeu de lignes ?
7. Que lit un plan d'exécution, et dans quel sens se lit-il ?

B · Quiz (20 questions · 20 points · 1 point par question)

Quatre familles de 5 questions : trouver la requête, prédire le résultat, diagnostiquer, nommer.

Bloc 1 — Quelle requête pour quelle question (C01—C05) (Q1 à Q5)

Q1. La cellule demande « quelle part de l'année représente ce mois ? ». Quelle construction répond ? a) un `GROUP BY` sur l'année ; b) `SUM(ca_mois) OVER (PARTITION BY annee)` ; c) `LAG(ca_mois) OVER (ORDER BY mois)` ; d) `NTILE(12) OVER (ORDER BY mois)`.

Q2. « Le 3^e meilleur produit de chaque famille » : quelle construction donne la bonne réponse **et** son rang ? a) `GROUP BY famille HAVING COUNT(*) <= 3` ; b) `RANK() OVER (PARTITION BY famille ORDER BY ca DESC)` puis un filtre `<= 3`, avec `id_produit` en clé de départage ; c) `LIMIT 3` ; d) `NTILE(3) OVER (PARTITION BY famille)`.

Q3. « Les clients qui ont acheté en janvier et pas en février » : quelle construction ne se trompe pas de sens ? a) `INNER JOIN` sur le mois ; b) `EXCEPT` (ou une jointure externe gauche avec test de nullité) ; c) `UNION ALL` ; d) `GROUP BY id_client` sans filtre de mois.

Q4. Pour dater la **cohorte** d'un client, quel événement prend-on ? a) sa date d'inscription au référentiel ; b) la date de sa **première commande** dans les ventes ; c) sa dernière commande ; d) la première commande de l'année en cours.

Q5. « Le chiffre d'affaires par quartier, **avec ses sous-totaux**, dans un seul résultat » : a) trois requêtes à recoller ; b) `GROUP BY ROLLUP (quartier, famille)` ; c) `PIVOT` ; d) `UNION` des détails et des totaux.

Bloc 2 — Prédisez le résultat (Q6 à Q10)

Q6. Que rend `SELECT id_client, COUNT(*) FROM ventes WHERE est_retour = 0 GROUP BY 1 HAVING COUNT(*) = 8` ? a) 8 clients ; b) 3 316 clients ; c) 8 lignes de vente ; d) aucun client.

Q7. Une requête filtre d'abord le mois de **janvier 2023**, puis calcule `SUM(montant_ttc) OVER (PARTITION BY annee_de_vente)`. Quelle part du chiffre d'affaires de 2023 le rapport publiera-t-il pour ce mois ? a) **8,3 %**, la vraie part ; b) **100,0 %**, parce que le filtre a été appliqué **avant** la fenêtre ; c) **12,5 %** ; d) une erreur d'exécution.

Q8. Sur les clients classés par **nombre de tickets**, que valent `MAX(RANK())` et `MAX(DENSE_RANK())` ? a) **23 444** et **23 444** ; b) **23 444** et **21** ; c) **23 497** et **21 444** ; d) **21** et **21**.

Q9. Combien de lignes rend `GROUP BY ROLLUP (quartier, famille)` sur ce socle, et de quoi sont-elles faites ? a) **35** lignes, les couples ; b) **41** lignes : **35** couples, **5** sous-totaux de quartier et **1** total général ; c) **47** lignes ; d) **44** lignes, une par mois.

Q10. Une moyenne mobile de **12** mois est calculée au **3^e** mois du socle. Combien de lignes portent réellement dans sa fenêtre ? a) **12** ; b) **3** ; c) **0** ; d) **1**.

Bloc 3 — Diagnostic (Q11 à Q15)

Q11. Un suivi des objectifs joint les ventes au référentiel des objectifs avec un `INNER JOIN` : le magasin 4 perd deux mois. Combien manque-t-il de lignes, et combien de ventes sont-elles concernées ? a) **2** couples, **93 749 490** FCFA ; b) **44** couples, **6 685 260 000** FCFA ; c) **2** couples, **5 317 736 710** FCFA ; d) aucun, la jointure interne ne perd rien.

Q12. Un rapport compare 2026 (**8** mois) à 2025 (**12** mois) et annonce **-28,9 %**. Que faut-il publier ? a) **-28,9 %**, le chiffre est exact ; b) **+15,4 %**, la comparaison à **périmètre égal** ; c) **0 %** ; d) **+18,8 %**, la croissance de 2024.

Q13. Un rapport de rétention publie **13,2 %** (**432** clients revenus sur **3 276** actifs), alors qu'une lecture par cohorte donne **22,7 %** sur **22** clients. Quelle est la cause ? a) une panne du moteur ; b) deux **définitions** différentes de la rétention — **calendaire** contre **par cohorte** — et un dénominateur qui ne mesure pas la même population ; c) un filtre de date oublié ; d) une erreur d'arrondi.

Q14. Un « meilleur produit par catégorie » écrit sur les libellés bruts du référentiel rend **46** lignes ; sur les familles repliées, il en rend **21**. Que révèle l'écart ? a) une erreur de la requête ; b) un **défaut de normalisation** du référentiel : **16** libellés bruts pour **9** écritures et **7** familles réelles ; c) une différence de période ; d) une limite mémoire du moteur.

Q15. Un rapport qui fait `SELECT *` sur la vue coûte **581** ms, la même requête sur trois colonnes nommées **96** ms. Qu'annonce le plan **avant** toute mesure ? a) rien d'utile ; b) la lecture de **18** colonnes contre **3** : le coût est dans la projection ; c) qu'il faut un index ; d) que la vue est corrompue.

Bloc 4 — Vocabulaire (Q16 à Q20)

Q16. Qu'appelle-t-on le **grain** d'un résultat ? a) sa taille en octets ; b) ce que représente **une ligne** (un mois, un client, un ticket, un couple quartier-famille) ; c) le nombre de colonnes ; d) la clé de tri.

Q17. Qu'appelle-t-on une **clé de départage**, et que se passe-t-il sans elle ? a) la clé primaire de la table ; b) la seconde colonne de tri qui rend l'ordre **déterministe** : sans elle, un classement peut changer de réponse entre deux exécutions ; c) un index ; d) un filtre.

Q18. Que fait **QUALIFY** ? a) il filtre **avant** le calcul de la fenêtre ; b) il filtre **après** le calcul de la fenêtre, ce que **WHERE** ne peut pas faire ; c) il trie ; d) il crée une vue.

Q19. Pourquoi un rapport coûte-t-il **142** ms sur la **vue** et **1** ms sur la table équivalente ? a) la table est plus petite ; b) une **vue** ne stocke rien : elle **relit** sa source à chaque appel ; c) la vue est mal écrite ; d) les deux requêtes ne sont pas les mêmes.

Q20. Que doit contenir la **note de méthode** d'un rapport de performance ? a) seulement les temps mesurés ; b) les temps, la version du moteur, le protocole, la date, et ce qui **n'a pas** été exécuté ; c) une capture d'écran ; d) la liste des index de la base.

C · Exercices à rendre (20 points · 5 exercices, dont 2 de type E4)

E1 — Le top 3 par famille, deux fois (niveau 3 · 3 points)

Produisez les **3** meilleurs produits de chaque famille du catalogue, une première fois sur les libellés tels quels, une seconde fois après repliement (casse, espaces, accents, pluriel). Donnez le **nombre de lignes** rendu par chaque version, et expliquez l'écart en deux phrases.

E2 — Les deux périmètres de la concentration (niveau 3 · 3 points)

Calculez la part du chiffre d'affaires du dixième supérieur des clients, **avec** puis **sans** le client non identifié. Écrivez la phrase qui doit accompagner la publication de chacune des deux valeurs.

E3 — Le suivi des objectifs, trous compris (niveau 3 · 4 points)

Produisez le suivi mensuel 2024 du magasin 4 : réalisé, objectif, taux de réalisation, et une colonne **etat** à trois valeurs (complet, objectif manquant, vente manquante). Le rapport doit faire apparaître les mois sans objectif, avec le montant de ventes concerné — **aucune ligne ne doit disparaître**.

E4 — « Ce rapport annonce un chiffre d'affaires net » (type E4 · niveau 4 · 5 points)

Voici la requête telle qu'elle a été livrée :

```
SELECT COUNT(*) AS lignes, ROUND(SUM(montant_ttc)) AS ca_net
FROM ventes;
```

Elle rend **240 000** lignes et **15 419 985 157** FCFA, publiés comme « chiffre d'affaires net ».

1. **Nommez l'erreur** en une phrase, sans écrire de requête.
2. **Corrigez** la requête et donnez les deux valeurs attendues.
3. **Mesurez l'écart** en FCFA et en pourcentage, et dites dans quel sens il joue.
4. Dites ce que ce total ferait à un rapport de suivi des objectifs, dont l'objectif cumulé est de **6 685 260 000** FCFA.
5. Écrivez la ligne d'en-tête qui aurait empêché l'erreur.

E5 — « Ce classement change de réponse à chaque exécution » (type E4 · niveau 4 · 5 points)

Voici la requête telle qu'elle a été livrée :

```
SELECT id_client, NTILE(3) OVER (ORDER BY nb_tickets) AS segment
FROM (SELECT id_client, COUNT(*) AS nb_tickets FROM ventes GROUP BY 1) t;
```

Relancée **15** fois le **24/09/2026**, elle a compté **2 718, 2 720, 2 728** puis **2 738** clients dans son segment principal — sans qu'aucune donnée ne change.

1. **Nommez la cause**, en citant le fait du socle qui la rend possible.
2. **Corrigez** la requête, et donnez la taille des trois tiers une fois l'ordre fixé.
3. Expliquez pourquoi ce défaut est **pire** qu'une requête lente.
4. Écrivez le **test de non-régression** qui aurait détecté le problème (valeur attendue, requête rejouée, verdict).

D · Auto-test SQL (10 points · 18 requêtes · auto-corrigé)

L'instrument `tools/controle_sql_M11.py` rejoue **18** requêtes et confronte chaque résultat aux valeurs publiées du module. Exécutez-le et **rendez sa sortie** (les **18** lignes de verdict et la ligne de bilan).

Barème : **18** contrôles au vert = **10** points ; **14** à **17** = **7** points ; **10** à **13** = **4** points ; moins de **10** = **0** point, avec un mot d'explication obligatoire sur les écarts.

Attention. Ne modifiez **aucune** requête de l'instrument : un contrôle n'est un contrôle que s'il rejoue la requête **publiée**. Si un écart apparaît, cherchez d'abord ce qui a changé dans le socle ou dans le fichier publié — et notez-le, c'est la réponse attendue.

E · Étude de cas — « 10 % des clients font-ils 60 % du CA ? » (20 points · seuil 12)

La commande

La direction commerciale a entendu, en séminaire, que « **10 %** des clients font **60 %** du chiffre d'affaires ». Elle vous demande de **vérifier la formule sur son propre fichier** et de lui dire, en une page, si elle peut la reprendre à son compte.

Les quatre productions attendues

1. **Le chiffre vrai** — la part du chiffre d'affaires portée par le dixième supérieur des clients, avec le **périmètre** annoncé (ventes nettes, période, magasins) et la **définition du décile** retenue.
2. **Le classement des clients, avant et après retrait du client non identifié** — ce que change le retrait, et le mot qui l'explique.
3. **La conclusion tenable** — trois lignes pour la direction, chiffres compris, sans formule recopiée sans vérification.
4. **La question suivante à poser** — celle que votre rapport ne tranche pas, et que la direction devra poser à son tour.

Le résultat attendu (à retrouver, pas à recopier)

Sur ce socle, la réponse est **non**, et elle est stable : le dixième supérieur pèse **38,1 %** du chiffre d'affaires net, **24,2 %** une fois le client **0** écarté, **51,0 %** pour le top **20 %**, **30,0 %** pour le top **5 %** et **21,6 %** pour le top **1 %**.

Attention. Le résultat dépend de la **définition** du dixième : `NTILE(100) <= 1` retient **2 350** clients et donne **38,1 %** ; un `ROW_NUMBER()` sur **10 %** du fichier en retient **2 349** et donne **38,0 %**. Un rapport qui publie un pourcentage sans publier sa définition de décile produit un chiffre que personne ne peut reproduire.

Barème de l'étude de cas

Critère	Points
1. Le chiffre vrai : périmètre annoncé, définition du décile écrite, valeur exacte	5
2. Le classement avant / après retrait du client 0 , et le mot qui l'explique	5
3. La conclusion tenable : trois lignes, utilisables, sans surinterprétation	5
4. La question suivante, formulée comme une question à laquelle on peut répondre	5
Total	20

Correction détaillée

A • Réponses de récupération

1. Une fenêtre **conserve** les lignes (elle ajoute des colonnes) ; `GROUP BY` **réduit** la table aux groupes.
2. `RANK` laisse des trous après une égalité, `DENSE_RANK` n'en laisse pas, `ROW_NUMBER` attribue des rangs uniques — d'où la nécessité d'une clé de départage.
3. Le cadre est la portion de la partition prise en compte autour de la ligne courante ; `2 PRECEDING` retient trois lignes et ne s'étend pas au-delà des lignes existantes.
4. Un groupe de clients partagé par un événement daté — ici la **première commande**, et non la date d'inscription du référentiel.
5. `QUALIFY` filtre **après** le calcul de la fenêtre : il permet de garder le rang 1 par groupe dans la même requête.
6. `UNNEST` passe d'une ligne « ticket » à une ligne « article de ticket » : le grain change, et `COUNT(*)` ne compte plus la même chose.
7. Le plan décrit les opérateurs et les colonnes lues avant l'exécution ; il se lit **de bas en haut**.

B • Corrigé du quiz (1 point par question)

Question	Réponse	Pourquoi
Q1	b	La part se calcule par une fenêtre partitionnée par année (C01).
Q2	b	<code>RANK</code> partitionné, filtré à 3 , avec <code>id_produit</code> en départage (C02).
Q3	b	<code>EXCEPT</code> (ou jointure externe gauche + nullité) : le <code>INNER JOIN</code> répond à une autre question (C05).
Q4	b	La cohorte se date sur la première commande (C04).
Q5	b	<code>ROLLUP</code> produit les 41 lignes en une requête (C05).

Question	Réponse	Pourquoi
Q6	b	3 316 clients partagent exactement 8 tickets (14,1 %).
Q7	b	Le filtre avant la fenêtre ne laisse qu'une ligne : 100,0 % au lieu de 8,3 % (C01).
Q8	b	23 444 en RANK contre 21 en DENSE_RANK (C02).
Q9	b	35 couples + 5 sous-totaux + 1 total général (C05).
Q10	b	Au 3 ^e mois, la fenêtre ne porte que sur 3 lignes (C03).
Q11	a	2 couples manquants (magasin 4, 2024-02 et 2024-03) pour 93 749 490 FCFA (C03).
Q12	b	À périmètre égal : +15,4 % (C03).
Q13	b	Rétention calendaire contre rétention par cohorte : deux définitions, deux populations (C04).
Q14	b	16 libellés bruts, 9 écritures, 7 familles : 46 lignes contre 21 (C05).
Q15	b	Le plan annonce 18 colonnes contre 3 : le coût se lit avant l'exécution (C06).
Q16	b	Le grain est ce que représente une ligne (C01, C05).
Q17	b	La clé de départage rend l'ordre déterministe ; sans elle, le rapport change (C02, C07).
Q18	b	QUALIFY filtre après la fenêtre (C05).
Q19	b	Une vue relit sa source : 142 ms contre 1 ms (C06).
Q20	b	Temps, moteur, protocole, date, et ce qui n'a pas été exécuté (C06, C07).

C · Corrigé des exercices à rendre

E1. Les libellés bruts rendent **46** lignes pour **16** catégories ; après repliement des graphies, le même top 3 rend **21** lignes pour **7** familles. L'écart est la trace exacte du défaut de normalisation du référentiel, pas un problème de requête.

E2. Avec le client **0** : **38,1** % ; sans lui : **24,2** %. Les deux phrases attendues : « le dixième supérieur des **2 350** clients pèse **38,1** % du chiffre d'affaires net » et « hors client non identifié, ce dixième pèse **24,2** % — l'écart est porté par un seul identifiant, non rattaché à un client du référentiel ».

E3. Structure FULL OUTER JOIN avec CASE à trois états ; les **2** lignes sans objectif portent **42 514 372** et **51 235 118** FCFA, soit **93 749 490** FCFA de réalisé sans référence. Le taux de réalisation y vaut NULL — ni **0**, ni **100** %.

E4 — les cinq attendus. 1. La requête ne filtre pas les retours : elle compte les **2 809** lignes de retour dans un total annoncé « net ». 2. Correction : WHERE est_retour = 0 → **237 191** lignes et **15 595 154 955** FCFA. 3. Écart : **-175 169 798** FCFA, soit **-1,1** % — le total faux est **plus bas** que le vrai, puisque les retours sont négatifs, alors que le nombre de lignes est **plus haut** de **1,2** % (**240 000** comptées contre **237 191**) : l'erreur se lit à l'envers selon l'indicateur. 4. Face à un objectif cumulé de **6 685 260 000** FCFA, le total faux afficherait **230,7** % au lieu de **233,3** % : l'erreur reste invisible dans le vert de l'indicateur. 5. En-tête : « Hypothèses : ventes **nettes** de retours (est_retour = 0) ; les **2 809** lignes de retour sont exclues, jamais sommées. »

E5 — les quatre attendus. 1. L'ORDER BY n'a **pas** de clé de départage : **3 316** clients partagent exactement **8** tickets (**14,1** % du fichier), et le moteur est libre de placer les ex æquo comme il veut. 2. Correction : ORDER BY nb_tickets, id_client → les trois tiers comptent **7 833**, **7 832** et **7 832** clients, stables d'une exécution à l'autre. 3. Une requête lente se corrige et se mesure ; une requête **instable** publie des chiffres contradictoires d'un jour à l'autre, et le rapport perd sa crédibilité avant qu'on ait trouvé pourquoi. 4. Le test attendu rejoue **à l'identique** la requête corrigée et compare la taille du premier tiers à **7 833** : tout autre résultat fait échouer le contrôle et sort en code non nul.

D · Corrigé de l'auto-test

La sortie attendue comporte **18** lignes de verdict au vert, du comptage des lignes (**240 000, 237 191** hors retours) au ratio de saison (**1,65**), et se termine par « **18** controles sur 18 : OK — le socle et le fichier publie disent la meme chose. »

E · Corrigé de l'étude de cas — les quatre attendus

1. **Le chiffre vrai.** **38,1 %** du chiffre d'affaires net, périmètre : ventes nettes (retours exclus), 01/01/2023 au 31/08/2026, **5** magasins, client **0 inclus** ; définition du décile : `NTILE(100) <= 1`, soit **2 350** clients (l'autre définition usuelle, un `ROW_NUMBER` sur **10 %** du fichier, retient **2 349** clients et donne **38,0 %**).
2. **Le classement, avant et après.** Le client **0** pèse **2 852 612 447** FCFA — **18,3 %** du chiffre d'affaires — sur **42 658** lignes et **38 984** tickets, sans exister dans le référentiel. Il occupe donc toujours la première place d'un classement par montant : une fois écarté, le dixième supérieur tombe à **24,2 %**, et le classement des têtes change intégralement. Le mot à employer est **périmètre** : retirer le client **0** retire **18,3 %** du chiffre d'affaires, c'est un choix, et il s'écrit.
3. **La conclusion tenable.** Trois lignes, du type : « sur notre fichier, les **10 %** de clients les plus gros pèsent **38 %** du chiffre d'affaires, et **24 %** une fois le client non identifié écarté — la formule des **60 %** ne s'applique pas ici ; la concentration est réelle mais plus faible, et le top **20 %** pèse **51,0 %**. »
4. **La question suivante.** « Quelle part du chiffre d'affaires de chaque **cohorte** est portée par des clients non identifiés ? » Tant que le client **0** représente **18,3 %** du chiffre d'affaires, toute lecture de la rétention se fait sur un cinquième du total qu'on ne sait pas attribuer — c'est la question qui manque aux **12** rapports du projet, et elle se pose avec les outils du chapitre C04.

Barème global et seuils

Partie	Points	Seuil
B · Quiz (20 questions)	20	14/20 (§B.6)
C · Exercices à rendre (5 , dont 2 E4)	20	—
D · Auto-test SQL (18 requêtes)	10	—
E · Étude de cas	20	12/20 (§B.6)
Total	70	49/70 pour la validation du module

Épreuve de palier. Cette évaluation prépare l'épreuve **P4** de la phase 4 (§B.6) : les quatre compétences qu'elle mesure — écrire une requête qui répond à une question posée, prédire ce qu'une fenêtre va changer, diagnostiquer un total faux, publier un taux avec son effectif et sa définition — sont celles que l'épreuve de palier reprend, sur un socle différent.